ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/360410918

INTRODUCTION TO BIG DATA ANALYTICS

Book · May 2022

CITATIONS
0

READS
2,408

2 authors:



John.T. Mesiah Dhas
T John Institute of Technology Bangalore
35 PUBLICATIONS **46** CITATIONS
SEE PROFILE



Shiny Angel T.S
SRM Institute of Science and Technology
47 PUBLICATIONS **61** CITATIONS
SEE PROFILE

All content following this page was uploaded by John.T. Mesiah Dhas on 06 May 2022.

The user has requested enhancement of the downloaded file.

Introduction to Big Data Analytics

Dr. John T Mesia Dhas

Dr. T. S. Shiny Angel

Dr. Adarsh T K

The Palm Series



Introduction to Big Data Analytics

Dr. John T Mesia Dhas

Dr. T. S. Shiny Angel

Dr. Adarsh T K

The Palm Series



Title: INTRODUCTION TO BIG DATA ANALYTICS

Author: Dr. John T Mesia Dhas, Dr. T. S. Shiny Angel, Dr. Adarsh T. K.

Publisher: Self-published by Dr. John T Mesia Dhas

Copyright © 2022 Dr. John T Mesia Dhas

All rights reserved, including the right of reproduction in whole or in part or any form

Address of Publisher: No-1, MGR Street, Charles Nagar, Pattabiram

Chennai – 600072

India

Email: jtmdhasres@gmail.com

Printer: The Palm

Mogappair West

Chennai -600037

India

ISBN: 

Chapter	Topic	Page
1. BIG DATA		
1	1. Introduction	1
	2. Classification of Big Data	3
	3. Types of Big Data Analytics	8
	4. Characteristics	9
	5. Major Challenges	14
	6. Traditional Approach of Storing and Processing	17
2. HADOOP		
2	1. Introduction	27
	2. Important Features	30
	3. How Hadoop Works?	33
	4. Hadoop Eco System and Components	36
3. HADOOP DISTRIBUTED FILE SYSTEMS		
3	1. Introduction to HDFS	48
	2. HDFS Daemons	52
	3. HADOOP Architecture	58
	4. Read Operation in HDFS	73
	5. Write Operation in HDFS	74
	6. Hadoop Installation Process	85
	7. Exploring HADOOP Commands	98
	8. Rack Awareness in Hadoop HDFS	111
4. MAP REDUCE		
4	1. Map Reduce Architecture	115
	2. Map Reduce jobs	118
	3. Shuffle and sort on Map and reducer side	125
	4. Map Reduce Types	129
	5. Input formats	132
	6. Output formats	135

Chapter	Topic	Page
	7. Map side and Reduce side joins	137
	8. Map Reduce Programs	145
5. BIG DATA ANALYTICS – CASE STUDIES		
5	1. Netflix on AWS	153
	2. AccuWeather on Microsoft Azure	154
	3. China Eastern Airlines on Oracle Cloud	154
	4. Etsy on Google Cloud	155
	5. mLogica on Sap Hana Cloud	156

CHAPTER 1

BIG DATA

CONTENTS

- Introduction
- Classification
- Characteristics
- Major Challenges
- Traditional Approach of Storing and Processing

1. INTRODUCTION

The 20th Century

The first major data project is created in 1937 and was ordered by the Franklin D. Roosevelt's administration in the USA. After the Social Security Act became law in 1937, thegovernment had to keep track of contribution from 26 million Americans and more than 3 million employers. IBM got the contract to develop punch card-reading machine for this massive book keeping project.

The first data-processing machine appeared in 1943 and was developed by the British to decipher Nazi codes during World War II. This device, named Colossus, searched for patterns in intercepted messages at a rate of 5.000 characters per second. Thereby reducing the task from weeks to merely hours.

In 1952 the National Security Agency (NSA) is created and within 10 years contract more than 12.000 cryptologists. They are confronted with information overload during the Cold War as they start collecting and processing intelligence signals automatically.

In 1965 the United Stated Government decided to build the first data center to store over 742 million tax returns and 175 million sets of fingerprints by transferring al those records onto magnetic computer tape that had to be stored in a single location. The project was later dropped out of fear for 'Big Brother', but it is generally accepted that it was the beginning of the electronic data storage era.

In 1989 British computer scientist Tim Berners-Lee invented eventually the World Wide Web. He wanted to facilitate the sharing of information via a 'hypertext' system. Little could he know at the moment the impact of his invention.

John Mashey is the **father** of the term **Big Data** might well be John Mashey, who was thechief scientist at Silicon Graphics in the 1990s.

As of the '90s the creation of data is spurred as more and more devices are connected to the

internet. In 1995 the first super-computer is built, which was able to do as much work in a second than a calculator operated by a single person can do in 30.000 years.

The 21st Century

In 2005 Roger Mougalas from O'Reilly Media coined the term Big Data for the first time, only a year after they created the term Web 2.0. It refers to a large set of data that is almost impossible to manage and process using traditional business intelligence tools.

2005 is also the year that Hadoop was created by Yahoo! built on top of Google's MapReduce. Its goal was to index the entire World Wide Web and nowadays the open- source Hadoop is used by a lot organizations to crunch through huge amounts of data.

As more and more social networks start appearing and the Web 2.0 takes flight, more and more data is created on a daily basis. Innovative startups slowly start to dig into this massive amount of data and also governments start working on Big Data projects. In 2009 the Indian government decides to take an iris scan, fingerprint and photograph of all of these 1.2 billion inhabitants. All this data is stored in the largest biometric database in the world.

In 2010 Eric Schmidt speaks at the Techonomic conference in Lake Tahoe in California andhe states that "there were 5 exabytes of information created by the entire world between the dawn of civilization and 2003. Now that same amount is created every two days."

In 2011 the McKinsey report on Big Data *The next frontier for innovation, competition, and productivity,* states that in 2018 the USA alone will face a shortage of 140.000 – 190.000 datascientist as well as 1.5 million data managers.

In the past few years, there has been a massive increase in Big Data startups, all trying to dealwith Big Data and helping organizations to understand Big Data and more and more companies are slowly adopting and moving towards Big Data. However, while it looks like Big Data is around for a long time already, in fact Big Data is as far as the internet was in 1993. The large Big Data revolution is still ahead of us so a lot will change in the coming years. Let the Big Data era begin.

What is Data?

The quantities, characters, or symbols on which operations are performed by a computer, which may be stored and transmitted in the form of electrical signals and recorded onmagnetic, optical, or mechanical recording media.

What is Big Data?

Big Data is also **data** but with a **huge size**. Big Data is a term used to describe a collection of data that is huge in size and yet growing exponentially with time. In short, such data is so large and complex that none of the traditional data management tools are able to store it or process it efficiently.

Big data is a blanket term for the non-traditional strategies and technologies needed to gather,

organize, process, and gather insights from large datasets.

An exact definition of **"big data"** is difficult to nail down because projects, vendors, practitioners, and business professionals use it quite differently. With that in mind, generally speaking, **big data** is

- large datasets
- the category of computing strategies and technologies that are used to handle largedatasets

In this context, "large dataset" means a dataset too large to reasonably process or store with traditional tooling or on a single computer. This means that the common scale of big datasets is constantly shifting and may vary significantly from organization to organization.

Examples of Big Data

Following are some the examples of Big Data-

The **New York Stock Exchange** generates about ***one terabyte*** of new trade data per day.



Social Media

The statistic shows that ***500+terabytes*** ofnew data get ingested into the databases of social media site **Facebook**, every day. This data is mainly generated in terms of photo and video uploads, message exchanges, putting comments etc.



A single **Jet engine** cangenerate ***10+terabytes*** of data in ***30minutes*** of flight time. With many thousand flights per day, generation of data reaches up to many ***Petabytes.***



2. CLASSIFICATION OF BIG DATA

Classification is essential for the study of any subject. So Big Data is widely classified intothree main types, which are-



- Structured
- Unstructured
- Semi-structured

Structured data

Any data that can be stored, accessed and processed in the form of fixed format is termed asa 'structured' data. Structured Data is used to refer to the data which is already stored in databases, in an ordered manner. It accounts for about 20% of the total existing data and is used the most in programming and computer-related activities.

There are two sources of structured data- machines and humans. All the data received from sensors, weblogs, and financial systems are classified under machine-generated data. These include medical devices, GPS data, data of usage statistics captured by servers and applications and the huge amount of data that usually move through trading platforms, to name a few.

Human-generated structured data mainly includes all the data a human input into a computer,such as his name and other personal details. When a person clicks a link on the internet, or even makes a move in a game, data is created- this can be used by companies to figure out their customer behavior and make the appropriate decisions and modifications.

Data stored in a relational database management system is one example of a '**structured**' data.

Structured data with an example.

Top 3 players who have scored most runs in international T20 matches are as follows

Player	Country	Scores	No of Matchesplayed
Brendon McCullum	New Zealand	2140	71
Rohit Sharma	India	2237	90
Virat Kohli	India	2167	65

Unstructured data

While structured data resides in the traditional row-column databases, unstructured data is the opposite- they have no clear format in storage. The rest of the data created, about 80% of the total account for unstructured big data. Most of the data a person encounters belong to this category- and until recently, there was not much to do to it except storing it or analyzing itmanually.

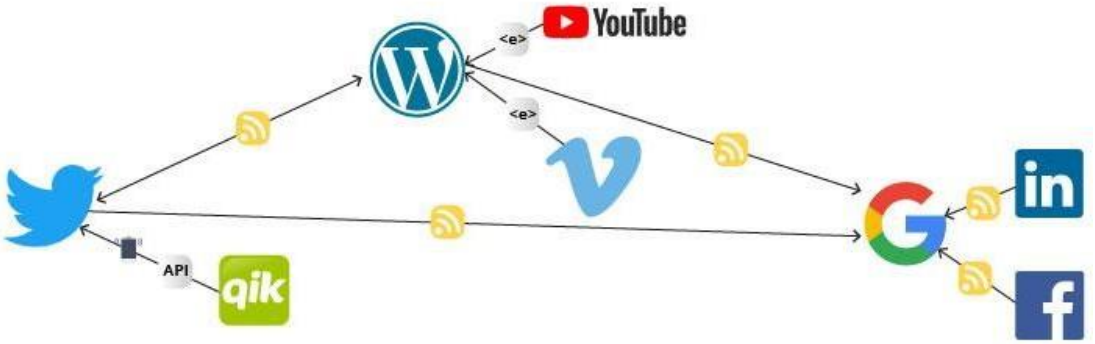
Unstructured data is also classified based on its source, into machine-generated orhuman-generated. Machine-generated data accounts for all the satellite images, the scientific data from various experiments and radar data captured by various facets of technology.

Human-generated unstructured data is found in abundance across the internet since it includessocial media data, mobile data, and website content. This means that the pictures we upload to Facebook or Instagram handle, the videos we watch on YouTube and even the text messages we

send all contribute to the gigantic heap that is unstructured data.

Examples of unstructured data include text, video, audio, mobile activity, social media activity, satellite imagery, surveillance imagery – the list goes on and on.

The following image will clearly help you to understand what exactly Unstructured data is



The Unstructured data is further divided into –

- Captured
- User-Generated data

Captured data

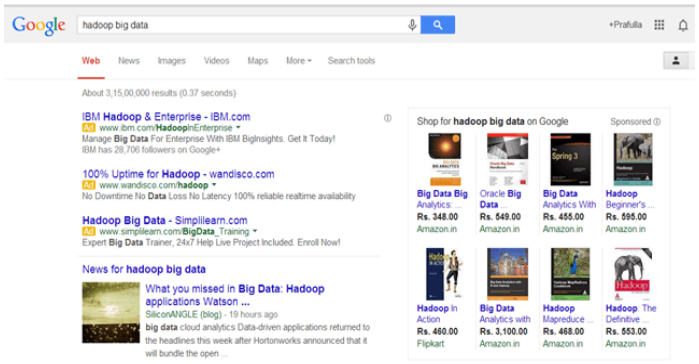
It is the data based on the user's behavior. The best example to understand it is GPS via smartphones which help the user each and every moment and provides a real-time output.

User-generated data

It is the kind of unstructured data where the user itself will put data on the internet every movement. For example, Tweets and Re-tweets, Likes, Shares, Comments, on Youtube,Facebook, etc.

Any data with unknown form or the structure is classified as unstructured data. In addition to the size being huge, un-structured data poses multiple challenges in terms of its processingfor deriving value out of it. A typical example of unstructured data is a heterogeneous data source containing a combination of simple text files, images, videos etc. Now day organizations have wealth of data available with them but unfortunately, they don't know how to derive value out of it since this data is in its raw form or unstructured format.

Examples of Un-structured Data

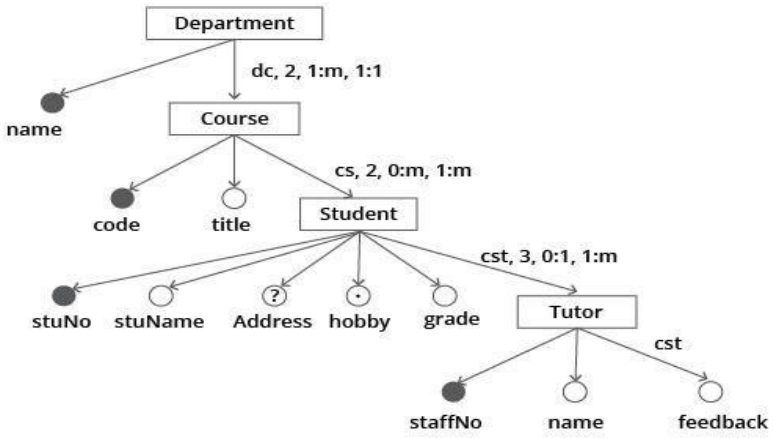


The output returned by 'Google Search'

Semi-structured data

The line between unstructured data and semi-structured data has always been unclear since most of the semi-structured data appear to be unstructured at a glance. Information that is not in the traditional database format as structured data, but contains some organizational properties which make it easier to process, are included in semi-structured data. For example, NoSQL documents are considered to be semi-structured, since they contain keywords thatcan be used to process the document easily.

Big Data analysis has been found to have definite business value, as its analysis and processing can help a company achieve cost reductions and dramatic growth. So, it is imperative that you do not wait too long to exploit the potential of this excellent business opportunity. Diagram showing Semi-structured data



Semi-structured data can contain both the forms of data. We can see semi-structured data as a structured in form but it is actually not defined with e.g., a table definition in relational DBMS. Example of semi-structured data is a data represented in an XML file.

Examples of Semi-structured Data Personal data stored in an XML file-

```
<rec><name>Prashant Rao</name><sex>Male</sex><age>35</age></rec>
<rec><name>Seema R.</name><sex>Female</sex><age>41</age></rec>
<rec><name>Satish       Mane</name><sex>Male</sex><age>29</age></rec>
<rec><name>Subrato Roy</name><sex>Male</sex><age>26</age></rec>
```

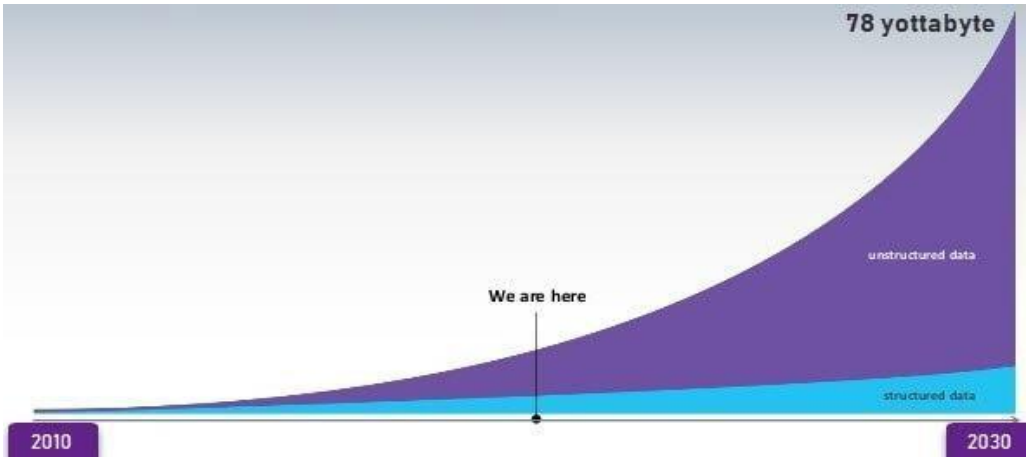
<rec><name>Jeremiah J.</name><sex>Male</sex><age>35</age></rec>

Difference between Structured, Semi-structured and Unstructured data

Factors	Structured data	Semi-structured data	Unstructured data
Flexibility	It is dependent and less flexible	It is more flexible than structured data but less than flexible than unstructured data	It is flexible in nature andthere is an absence of a schema
Transaction Management	Matured transaction and various concurrency technique	The transaction is adaptedfrom DBMS not matured	No transaction management and no concurrency
Query performance	Structured query allow complex joining	Queries over anonymous nodes are possible	An only textual query ispossible
Technology	It is based on the relational database table	It is based on RDF and XML	This is based on character and library data

Big data is indeed a revolution in the field of IT. The use of Data analytics is increasing every year. In spite of the demand, organizations are currently short of experts. To minimize this talent, gap many training institutes are offering courses on Big data analytics which helps you to upgrade skills set needed to manage and analyze big data. If you are keen to take up data analytics as a career then taking up Big data training will be an added advantage

Data Growth over the years



Please note that web application data, which is unstructured, consists of log files, transaction history files etc. OLTP systems are built to work with structured data wherein data is storedin relations (tables).

3. TYPES OF BIG DATA ANALYTICS

Prescriptive Analytics

The most valuable and most underused big data analytics technique, prescriptive analytics gives you a laser-like focus to answer a specific question. It helps to determine the best solution among a variety of choices, given the known parameters and suggests options for how to take advantage of a future opportunity or mitigate a future risk. It can also illustratethe implications of each decision to improve decision-making. Examples of prescriptive analytics for customer retention include next best action and next best offer analysis.

- Forward looking
- Focused on optimal decisions for future situations
- Simple rules to complex models that are applied on an automated or programmatic basis
- Discrete prediction of individual data set members based on similarities and differences
- Optimization and decision rules for future events

Diagnostic Analytics

Data scientists turn to this technique when trying to determine why something happened. It isuseful when researching leading churn indicators and usage trends amongst your most loyal customers. Examples of diagnostic analytics include churn reason analysis and customer health score analysis. Key points

- Backward looking
- Focused on causal relationships and sequences
- Relative ranking of dimensions/variable based on inferred explanatory power)
- Target/dependent variable with independent variables/dimensions
- Includes both frequentist and Bayesian causal inferential analyses

Descriptive Analytics

This technique is the most time-intensive and often produces the least value; however, it is useful for uncovering patterns within a certain segment of customers. Descriptive analytics provide insight into what has happened historically and will provide you with trends to dig into in more detail. Examples of descriptive analytics include summary statistics, clustering and association rules used in market basket analysis. Key points

- Backward looking
- Focused on descriptions and comparisons
- Pattern detection and descriptions

- MECE (mutually exclusive and collectively exhaustive) categorization
- Category development based on similarities and differences (segmentation)

Predictive Analytics

The most commonly used technique; predictive analytics use models to forecast what might happen in specific scenarios. Examples of predictive analytics include next best offers, churnrisk and renewal risk analysis.

- Forward looking
- Focused on non-discrete predictions of future states, relationship, and patterns
- Description of prediction result set probability distributions and likelihoods
- Model application
- Non-discrete forecasting (forecasts communicated in probability distributions)

Outcome Analytics

Also referred to as consumption analytics, this technique provides insight into customer behavior that drives specific outcomes. This analysis is meant to help you know your customers better and learn how they are interacting with your products and services.

- Backward looking, Real-time and Forward looking
- Focused on consumption patterns and associated business outcomes
- Description of usage thresholds
- Model application

The Implication

As you can see there are a lot of different approaches to harness big data and add context todata that will help you deliver customer success, while lowering your cost to serve.

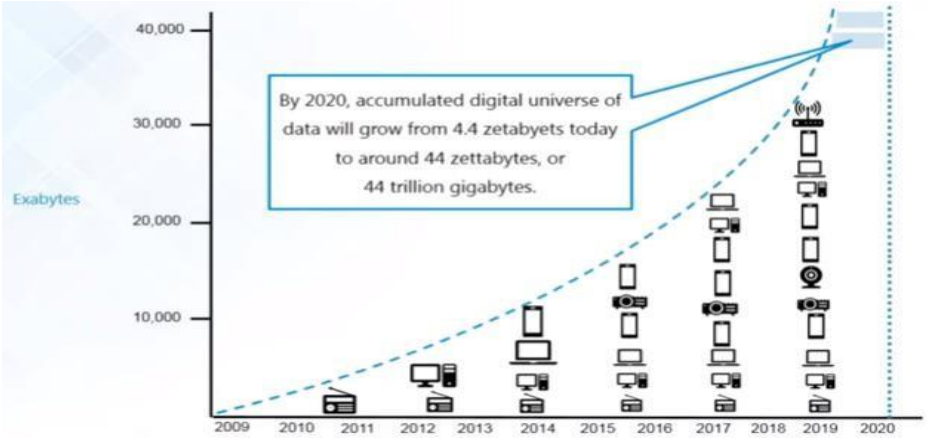
Demystify big data and you can effectively communicate with your IT department to convert complex datasets into actionable insights. It is important to approach any big data analytics project with answers to these questions

- What is the goal, business problem, who are the stakeholders and what is the value of solving the problem?
- What questions are you trying to answer?
- What are the deliverables?
- What will you do with the insights?

4. CHARACTERISTICS OF BIG DATA

i. ***Volume –*** The name Big Data itself is related to a size which is enormous. Size of data plays a

very crucial role in determining value out of data. Also, whether a particular data can actually be considered as a Big Data or not, is dependent upon the volume of data. Hence, '**Volume**' is one characteristic which needs to be considered while dealing with Big Data.



Volume refers to the incredible amounts of data generated each second from social media, cell phones, cars, credit cards, M2M sensors, photographs, video, etc. The vast amounts of data have become so large in fact that we can no longer store and analyze data using traditional database technology. We now use distributed systems, where parts of the data is stored in different locations and brought together by software. With just Facebook alone there are 10 billion messages, 4.5 billion times that the "like" button is pressed, and over 350 million new pictures are uploaded every day. Collecting and analyzing this data is clearly an engineering challenge of immensely vast proportions.

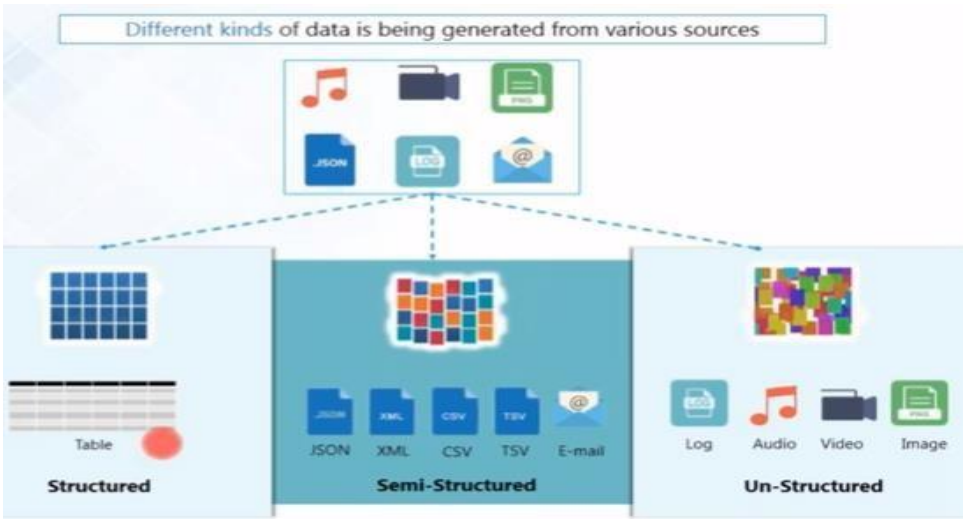
Big data implies enormous volumes of data. It used to be employees created data. Now that data is generated by machines, networks and human interaction on systems like social mediathe volume of data to be analyzed is massive. Yet, Inderpal states that the volume of data is not as much the problem as other V's like veracity.

ii. ***Variety*** **–** The next aspect of Big Data is its **variety**.

Variety refers to heterogeneous sources and the nature of data, both structured and unstructured. During earlier days, spreadsheets and databases were the only sources of data considered by most of the applications. Nowadays, data in the form of emails, photos, videos,monitoring devices, PDFs, audio, etc. are also being considered in the analysis applications. This variety of unstructured data poses certain issues for storage, mining and analyzing data.

Variety is defined as the different types of data we can now use. Data today looks very different than data from the past. We no longer just have structured data (name, phone number, address, financials, etc) that fits nice and neatly into a data table. Today's data is unstructured. In fact, 80% of all the world's data fits into this category, including photos, video sequences, social media updates, etc. New and innovative big data technology is now allowing structured and unstructured data to be harvested, stored, and used simultaneously.

Variety refers to the many sources and types of data both structured and unstructured. We used to store data from sources like spreadsheets and databases. Now data comes in the form of emails, photos, videos, monitoring devices, PDFs, audio, etc. This variety of unstructured data creates problems for storage, mining and analyzing data. Jeff Veis, VP Solutions at HP Autonomy presented how HP is helping organizations deal with big challenges including datavariety.

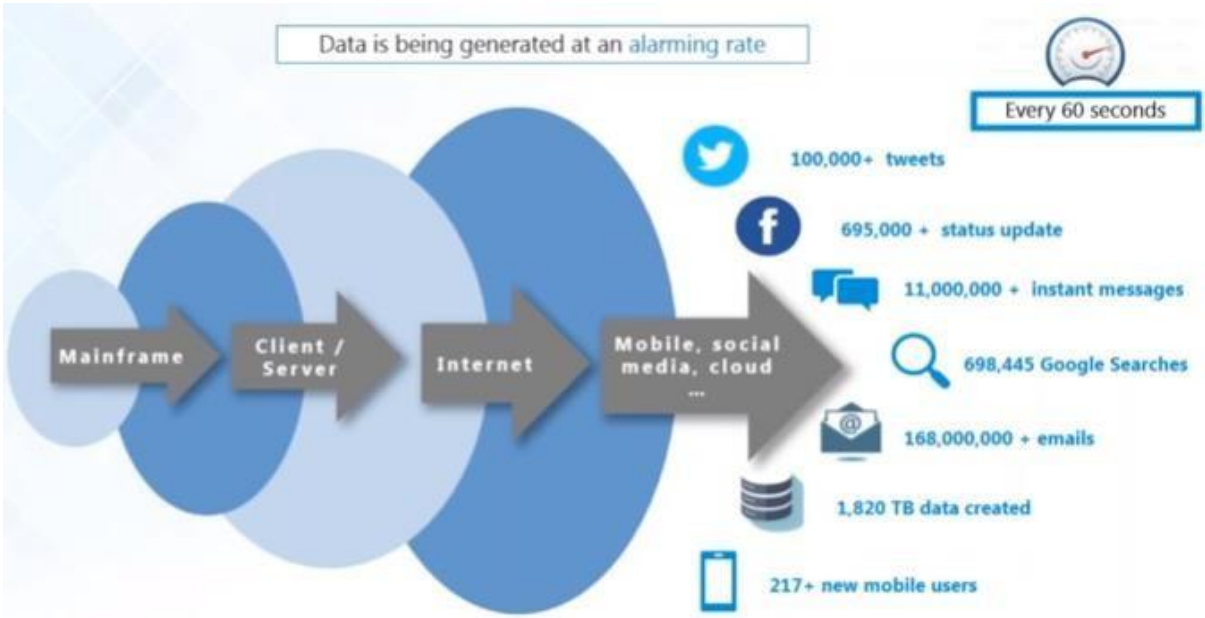


iii. ***Velocity*** **–** The term **'velocity'** refers to the speed of generation of data. How fast the datais generated and processed to meet the demands, determines real potential in the data.

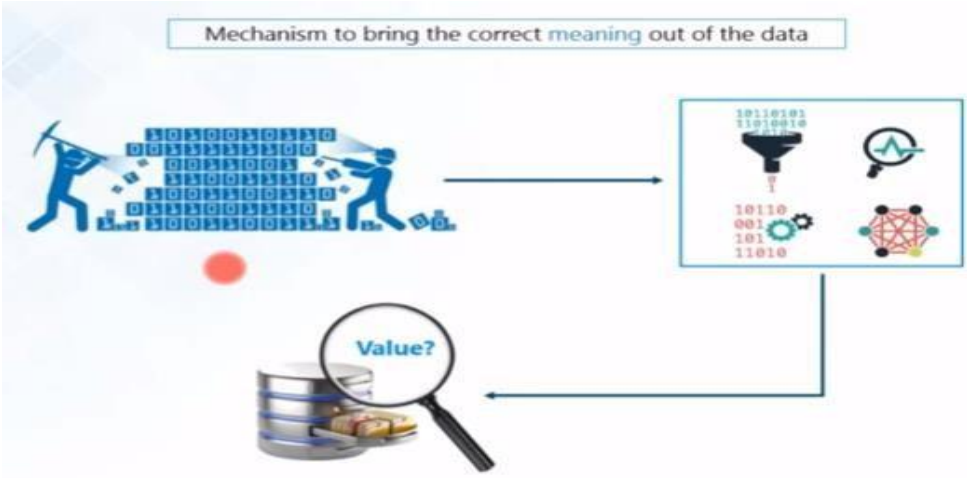
Big Data Velocity deals with the speed at which data flows in from sources like business processes, application logs, networks, and social media sites, sensors, Mobile devices, etc. The flow of data is massive and continuous.

Velocity refers to the speed at which vast amounts of data are being generated, collected and analyzed. Every day the number of emails, twitter messages, photos, video clips, etc. increases at lighting speeds around the world. Every second of every day data is increasing. Not only must it be analyzed, but the speed of transmission, and access to thedata must also remain instantaneous to allow for real-time access to website, credit card verification and instant messaging. Big data technology allows us now to analyze the data while it is being generated, without ever putting it into databases.

Big Data Velocity deals with the pace at which data flows in from sources like business processes, machines, networks and human interaction with things like social media sites, mobile devices, etc. The flow of data is massive and continuous. This real-time data can help researchers and businesses make valuable decisions that provide strategic competitive advantages and ROI if you are able to handle the velocity. Inderpal suggest that sampling data can help deal with issues like volume and velocity.



iv. ***Value*** **–** When we talk about value, we're referring to the worth of the data being extracted. Having endless amounts of data is one thing, but unless it can be turned into value it is useless. While there is a clear link between data and insights, this does not always mean there is value in Big Data. The most important part of embarking on a big data initiative is tounderstand the costs and benefits of collecting and analyzing the data to ensure that ultimately the data that is reaped can be monetized.



v. ***Veracity*** **–** This refers to the inconsistency which can be shown by the data at times, thus hampering the process of being able to handle and manage the data effectively.

Big Data Veracity refers to the biases, noise and abnormality in data. Is the data that is being stored, and mined meaningful to the problem being analyzed. Inderpal feel veracity in data analysis is the biggest challenge when compares to things like volume and velocity. In scoping out your big

data strategy you need to have your team and partners work to help keepyour data clean and processes to keep ‘dirty data’ from accumulating in your systems.

Min	Max	Mean	SD
4.3	?	5.84	0.83
2.0	4.4	3.05	50000000
15000	7.9	1.20	0.43
0.1	2.5	?	0.76

Uncertainty and inconsistencies in the data

vi. Validity

Like big data veracity is the issue of validity meaning is the data correct and accurate for the intended use. Clearly valid data is key to making the right decisions. Phil Francisco, VP of Product Management from IBM spoke about IBM’s big data strategy and tools they offer tohelp with data veracity and validity.

vii. Volatility

Big data volatility refers to how long is data valid and how long should it be stored. In this world of real time data, you need to determine at what point is data no longer relevant to the current analysis.

Big data clearly deals with issues beyond volume, variety and velocity to other concerns likeveracity, validity and volatility. To hear about other big data trends and presentation follow the Big Data Innovation Summit on twitter #BIGDBN.

Benefits of Big Data Processing

Ability to process Big Data brings in multiple benefits, such as-

- Businesses can utilize outside intelligence while taking decisions

Access to social data from search engines and sites like facebook, twitter is enabling organizations to fine tune their business strategies.

- Improved customer service

Traditional customer feedback systems are getting replaced by new systems designed with Big Data technologies. In these new systems, Big Data and natural language processing technologies are being used to read and evaluate consumer responses.

- Early identification of risk to the product/services, if any
- Better operational efficiency

Big Data technologies can be used for creating a staging area or landing zone for new data before identifying what data should be moved to the data warehouse. In addition, such integration

of Big Data technologies and data warehouse helps an organization to offload infrequently accessed data.

5. MAJOR CHALLENGES

There are 2 main challenges associated with Big Data.

- The 1st challenge is, how do we **store and manage** such a huge volume of data, efficiently?
- The 2nd challenge is, how do we **process and extract** valuable information from this huge volume of data within the given time frame?

These are the 2 main challenges associated with the Big Data, that led to thedevelopment of Hadoop framework.

Dealing with data growth

The most obvious challenge associated with big data is simply storing and analyzing all that information. In its Digital Universe report, IDC estimates that the amount of information stored in the world's IT systems is doubling about every two years. By 2020, the total amountwill be enough to fill a stack of tablets that reaches from the earth to the moon 6.6 times. Andenterprises have responsibility or liability for about 85 percent of that information.

Much of that data is unstructured, meaning that it doesn't reside in a database. Documents, photos, audio, videos and other unstructured data can be difficult to search and analyze.

It's no surprise, then, that the IDG report found, "Managing unstructured data is growing as a challenge – rising from 31 percent in 2015 to 45 percent in 2016."

In order to deal with data growth, organizations are turning to a number of different technologies. When it comes to storage, converged and hyper converged infrastructure and software-defined storage can make it easier for companies to scale their hardware. And technologies like compression deduplication and tiering can reduce the amount of space and the costs associated with big data storage.

On the management and analysis side, enterprises are using tools like NoSQL databases, Hadoop, Spark, big data analytics software, business intelligence applications, artificial intelligence and machine learning to help them comb through their big data stores to find the insights their companies need.

1. Generating insights in a timely manner

Of course, organizations don't just want to store their big data — they want to use that big data to achieve business goals. According to the New Vantage Partners survey, the most common goals

associated with big data projects included the following

- Decreasing expenses through operational cost efficiencies
- Establishing a data-driven culture
- Creating new avenues for innovation and disruption
- Accelerating the speed with which new capabilities and services are deployed
- Launching new product and service offerings

All of those goals can help organizations become more competitive — but only if they can extract insights from their big data and then act on those insights quickly. PwC's Global Data and Analytics Survey 2016 found, "Everyone wants decision-making to be faster, especially in banking, insurance, and healthcare."

To achieve that speed, some organizations are looking to a new generation of ETL and analytics tools that dramatically reduce the time it takes to generate reports. They are investing in software with real-time analytics capabilities that allows them to respond to developments in the marketplace immediately.

2. Recruiting and retaining big data talent

But in order to develop, manage and run those applications that generate insights, organizations need professionals with big data skills. That has driven up demand for big data experts — and big data salaries have increased dramatically as a result.

The 2017 Robert Half Technology Salary Guide reported that big data engineers were earning between $135,000 and $196,000 on average, while data scientist salaries ranged from $116,000 to $163, 500. Even business intelligence analysts were very well paid, making $118,000 to $138,750 per year.

In order to deal with talent shortages, organizations have a couple of options. First, many are increasing their budgets and their recruitment and retention efforts. Second, they are offering more training opportunities to their current staff members in an attempt to develop the talent they need from within. Third, many organizations are looking to technology. They are buying analytics solutions with self-service and/or machine learning capabilities. Designed to be used by professionals without a data science degree, these tools may help organizations achieve their big data goals even if they do not have a lot of big data experts on staff.

3. Integrating disparate data sources

The variety associated with big data leads to challenges in data integration. Big data comes from a lot of different places — enterprise applications, social media streams, email systems, employee-created documents, etc. Combining all that data and reconciling it so that it can be used to create reports can be incredibly difficult. Vendors offer a variety of ETL and data integration tools designed to make the process easier, but many enterprises say that they have not solved the

data integration problem yet.

In response, many enterprises are turning to new technology solutions. In the IDG report, 89 percent of those surveyed said that their companies planned to invest in new big data tools in the next 12 to 18 months. When asked which kind of tools they were planning to purchase, integration technology was second on the list, behind data analytics software.

4. Validating data

Closely related to the idea of data integration is the idea of data validation. Often organizations are getting similar pieces of data from different systems, and the data in those different systems doesn't always agree. For example, the ecommerce system may show daily sales at a certain level while the enterprise resource planning (ERP) system has a slightly different number. Or a hospital's electronic health record (EHR) system may have oneaddress for a patient, while a partner pharmacy has a different address on record.

The process of getting those records to agree, as well as making sure the records are accurate, usable and secure, is called data governance. And in the AtScale 2016 Big Data Maturity Survey, the fastest-growing area of concern cited by respondents was data governance.

Solving data governance challenges is very complex and is usually requires a combination of policy changes and technology. Organizations often set up a group of people to oversee data governance and write a set of policies and procedures. They may also invest in data management solutions designed to simplify data governance and help ensure the accuracy of big data stores — and the insights derived from them.

5. Securing big data

Security is also a big concern for organizations with big data stores. After all, some big data stores can be attractive targets for hackers or advanced persistent threats (APTs).

However, most organizations seem to believe that their existing data security methods are sufficient for their big data needs as well. In the IDG survey, less than half of those surveyed (39 percent) said that they were using additional security measure for their big data repositories or analyses. Among those who do use additional measures, the most popular include identity and access control (59 percent), data encryption (52 percent) and data segregation (42 percent).

6. Organizational resistance

It is not only the technological aspects of big data that can be challenging — people can be an issue too.

In the New Vantage Partners survey, 85.5 percent of those surveyed said that their firms werecommitted to creating a data-driven culture, but only 37.1 percent said they had been successful with those efforts. When asked about the impediments to that culture shift, respondents pointed to three big obstacles within their organizations

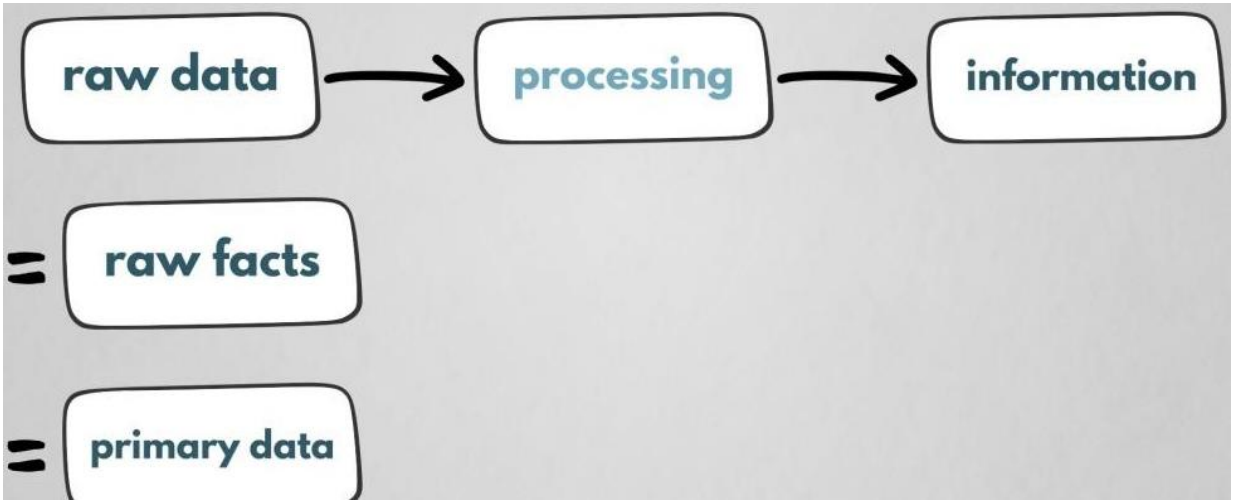
- Insufficient organizational alignment (4.6 percent)
- Lack of middle management adoption and understanding (41.0 percent)
- Business resistance or lack of understanding (41.0 percent)

In order for organizations to capitalize on the opportunities offered by big data, they aregoing to have to do some things differently. And that sort of change can be tremendously difficult for large organizations.

The PwC report recommended, "To improve decision-making capabilities at your company, you should continue to invest in strong leaders who understand data's possibilities and who will challenge the business."

One way to establish that sort of leadership is to appoint a chief data officer, a step that New Vantage Partners said 55.9 percent of Fortune 1000 companies have taken. But with or without a chief data officer, enterprises need executives, directors and managers who are going to commit to overcoming their big data challenges, if they want to remain competitive in the increasing data-driven economy.

6. TRADITIONAL APPROACH OF STORING AND PROCESSING



Raw data *(*Also called '*raw facts'* or '*primary data')* is what you have accumulated and stored on a server but not touched. This means you cannot analyze it straight away. We referto the gathering of *raw data* as 'data collection' and this is the first thing we do.

'Traditional' and 'big' raw data

We can look at data as being traditional or big data. If you are new to this idea, you could imagine traditional data in the form of tables containing categorical and numerical data. Thisdata is structured and stored in databases which can be managed from one computer. A way to collect traditional data is to survey people. Ask them to rate how much they like a productor experience on a scale of 1 to 10.

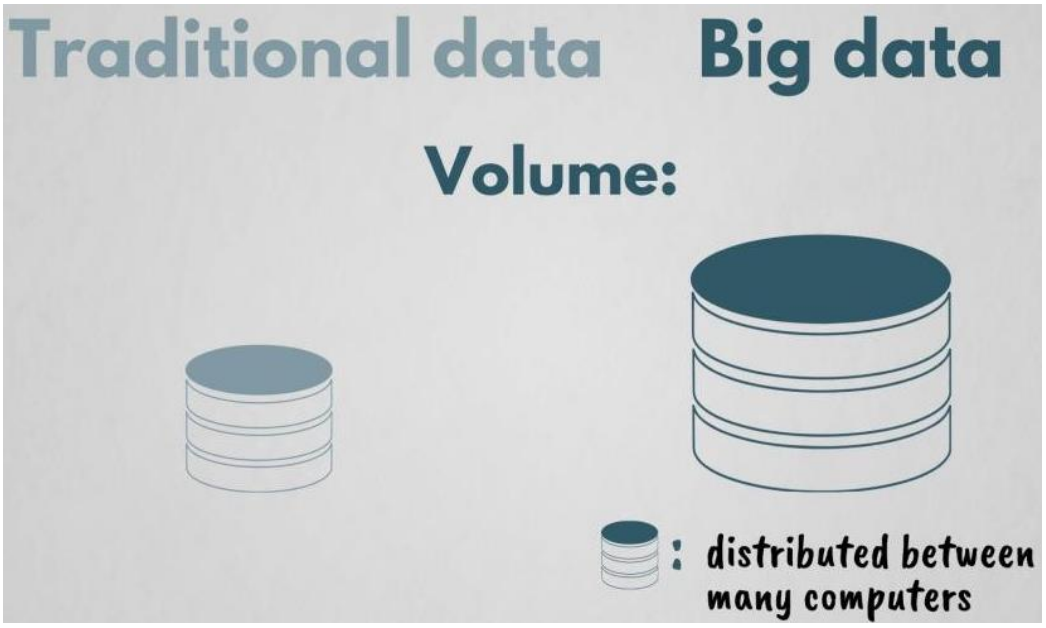
Traditional data is data most people are accustomed to. For instance, 'order management'helps you keep track of sales, purchases, e-commerce, and work orders.

Big data, however, is a whole other story. As you can guess by the name, 'Big data' is a term reserved for extremely large data.You will also often see it characterized by the letter 'V'. As in *"the 3Vs of 'big data*".
Sometimes we can have 5, 7 or even 11 'Vs of big data. They may include – the *Vision* you have about big data, the *Value* big data carries, the *Visualisation* tools you use or the

Variability in the consistency of big data. And so on…

However, the following are the most important criteria you must remember

Volume



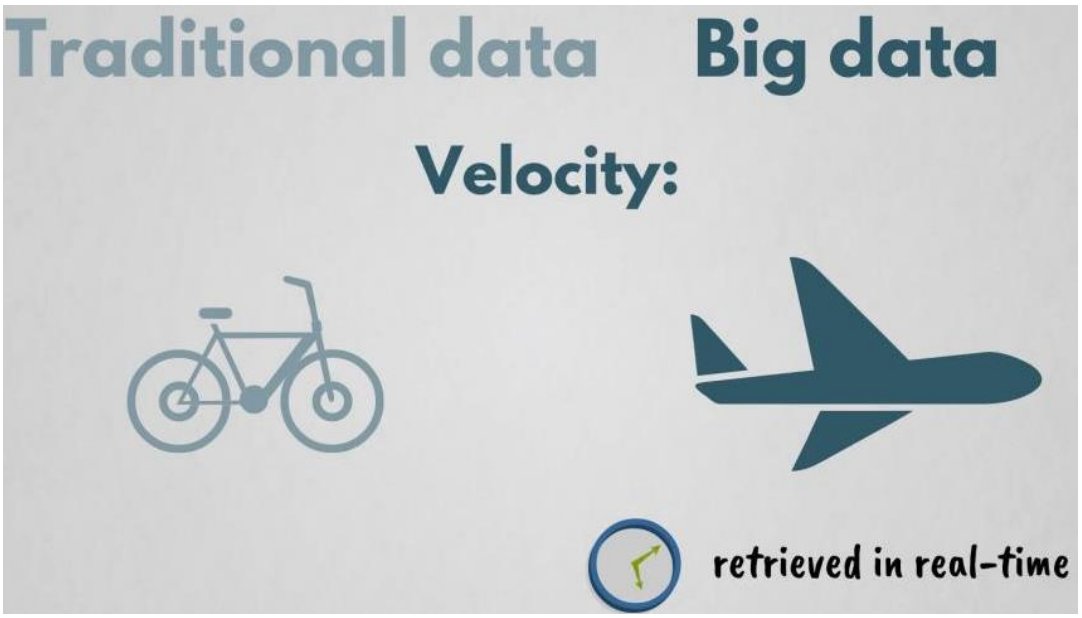
Big data needs a whopping amount of memory space, typically distributed between many computers. Its size is measured in terabytes, petabytes, and even exabytes

Variety



Here we are not talking only about numbers and text; big data often implies dealing with images, audio files, mobile data, and others.

Velocity



When working with big data, one's goal is to make extracting patterns from it as quick as possible. Where do we encounter big data?

The answer is in increasingly more industries and companies. Here are a few notable examples.

As one of the largest online communities, 'Facebook' keeps track of its users' names, personal data, photos, videos, recorded messages and so on. This means their data has a lot of *variety*. And with over *2 billion* users worldwide, the *volume* of data stored on their servers is tremendous.

Let's take 'financial trading data' for an extra example.

What happens when we record the stock price at every 5 seconds? Or every single second?We get a dataset that is *voluminous*, requiring significantly more memory, disc space and various techniques to extract meaningful information from it.

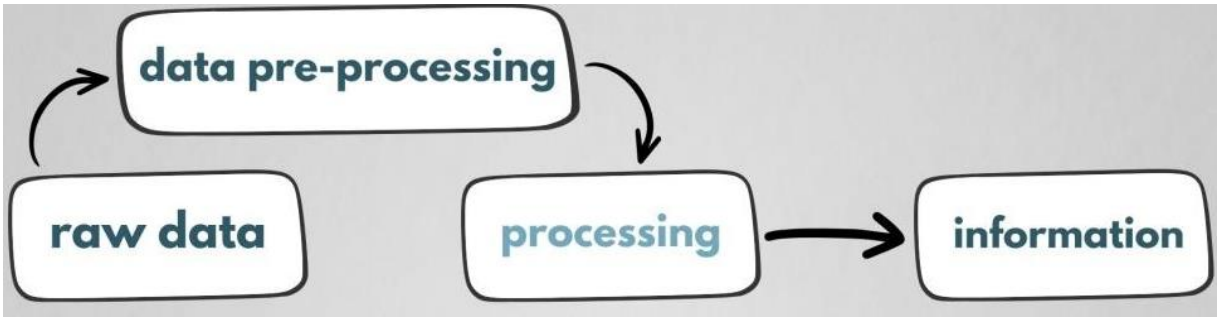
Both traditional and big data will give you a solid foundation to improve customer satisfaction. But this data will have problems, so before anything else, you must *process* it.

Processing

Let's turn that *raw data* into something beautiful!

The first thing to do, after gathering enough *raw data*, is what we call **'data preprocessing**'.This is a group of operations that will convert your raw data into a format that is more understandable and useful for further processing.

Data preprocessing



So, what does ‘data preprocessing’ aim to do?

It attempts to fix the problems that can occur with data gathering.

For example, within some customer data you collected, you may have a person registered as 932 years old or ‘United Kingdom’ as their name. Before proceeding with any analysis, youneed to mark this data as invalid or correct it. That’s what data pre-processing is all about!

Let’s delve into the techniques we apply while pre-processing both traditional and big *raw data*?

Class labelling

This involves labelling the data point to the correct data type, in other words, arranging data by category.

We divide traditional data into 2 categories

One category is ‘numerical’ – If you are storing the number of goods sold daily, then youare keeping track of numerical values. These are numbers which you can manipulate. For example, you can work out the average number of goods sold per day or month.

The other label is ‘categorical’ – Here you are dealing with information you cannot manipulate with mathematics. For example, a person’s profession. Remember that data pointscan still be numbers while not being numerical. Their date of birth is a number you can’t manipulate directly to give you any extra information.

Think of basic customer data.

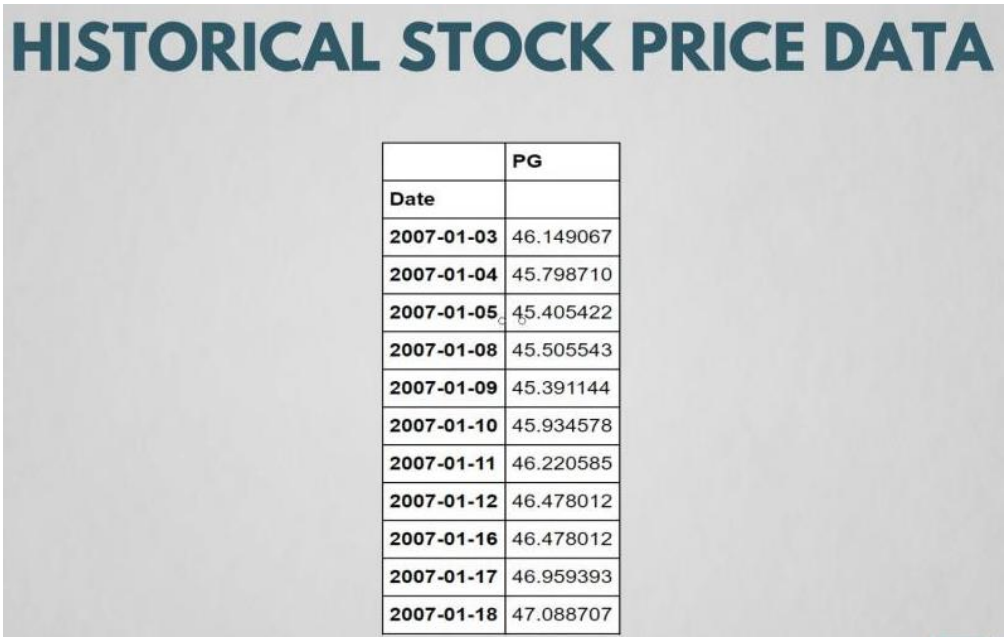
We will use this table, containing text information about customers, to give a clear exampleof the difference between a *numerical* and *categorical* variable.



Customers				
customer_id	first_name	last_name	email_address	number_of_complaints
1	John	McKinley	john.mackinley@365careers.com	0
2	Elizabeth	McFarlane	e.mcfarlane@365careers.com	2
3	Kevin	Lawrence	kevin.lawrence@365careers.com	1
4	Catherine	Winnfield	c.winnfield@365careers.com	0

Notice the first column, it shows the ID assigned to the different customers. You cannot manipulate these numbers. An ‘average’ ID is not something that would give you any useful information. This means that even though they are numbers, they hold no numerical value and are ***categorical data*.**

Now, focus on the last column. This shows how many times a customer has filed a complaint. You can manipulate these numbers. Adding them all together to give a total number of complaints is useful information, therefore, they are ***numerical data.***



	PG
Date	
2007-01-03	46.149067
2007-01-04	45.798710
2007-01-05	45.405422
2007-01-08	45.505543
2007-01-09	45.391144
2007-01-10	45.934578
2007-01-11	46.220585
2007-01-12	46.478012
2007-01-16	46.478012
2007-01-17	46.959393
2007-01-18	47.088707

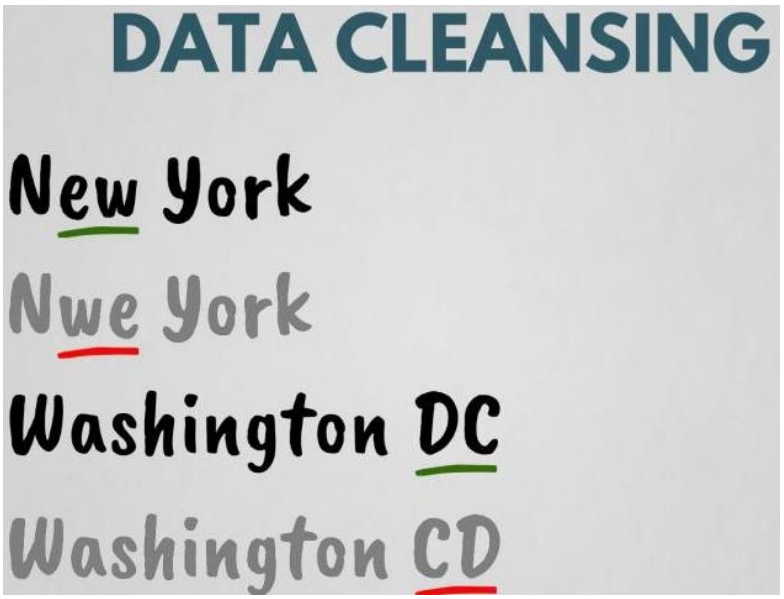
In the data set you see here, there's a column containing the dates of the observations, which is considered ***categorical data.*** And a column containing the stock prices, which is ***numericaldata***. When you work with big data things get a little more complex. You have much more variety,beyond 'numerical' and 'categorical' data, for example

- Text data
- Digital image data
- Digital video data
- And digital audio data

Data Cleansing

Also known as, ***'data cleaning'*** or ***'data scrubbing'***.

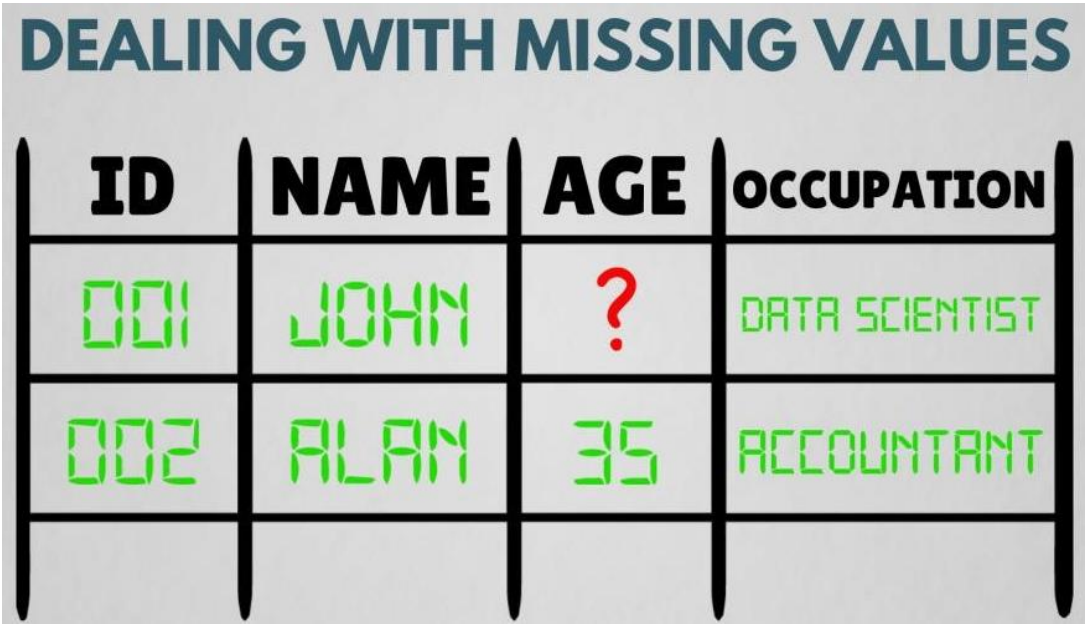
The goal of data cleansing is to deal with inconsistent data. This can come in various forms. Say, you gather a data set containing the US states and a quarter of the names are misspelled.In this situation, you must perform certain techniques to correct these mistakes. You must clean the data; the clue is in the name!



Big data has more data types and they come with a wider range of data cleansing methods. There are techniques that verify if a *digital image* is ready for processing. And specific approaches exist that ensure the *audio quality* of your file is adequate to proceed.

Missing values

'*Missing values'* are something else you must deal with. Not every customer will give you all the data you are asking for. What can often happen is that a customer will give you his name and occupation but not his age. What can you do in that case?



Should you disregard the customer's entire record? Or could you enter the average age of the remaining customers?

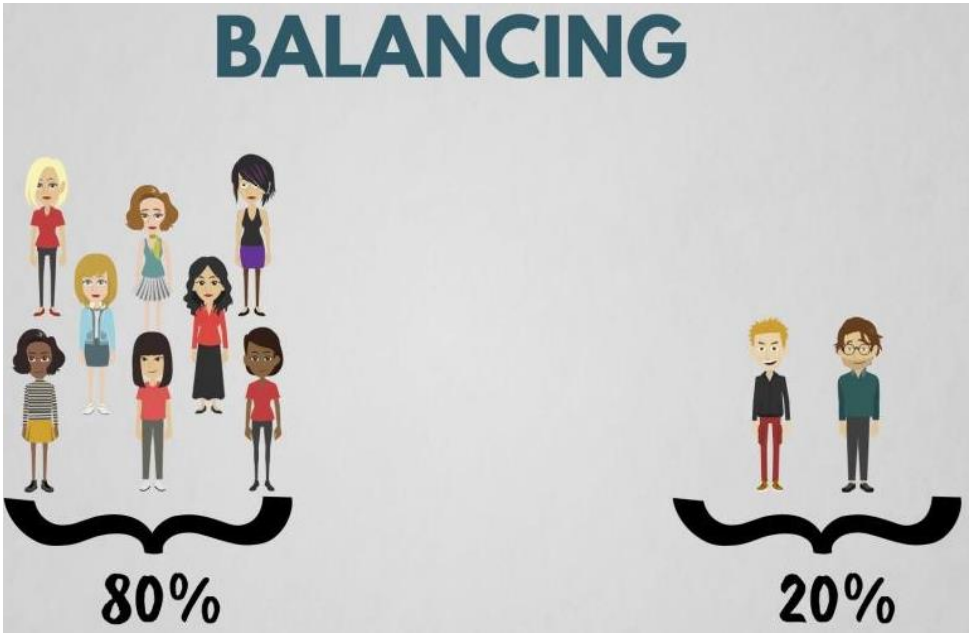
Whatever the best solution is, it is essential you clean the data and deal with missing values before you can process the data further.

Techniques to process traditional data

Let's move onto two common techniques for processing traditional data.

Balancing

Imagine you have compiled a survey to gather data on the shopping habits of men and women. Say, you want to ascertain who spends more money during the weekend. However, when you finish gathering your data you become aware that 80% of respondents were femaleand only 20% male.

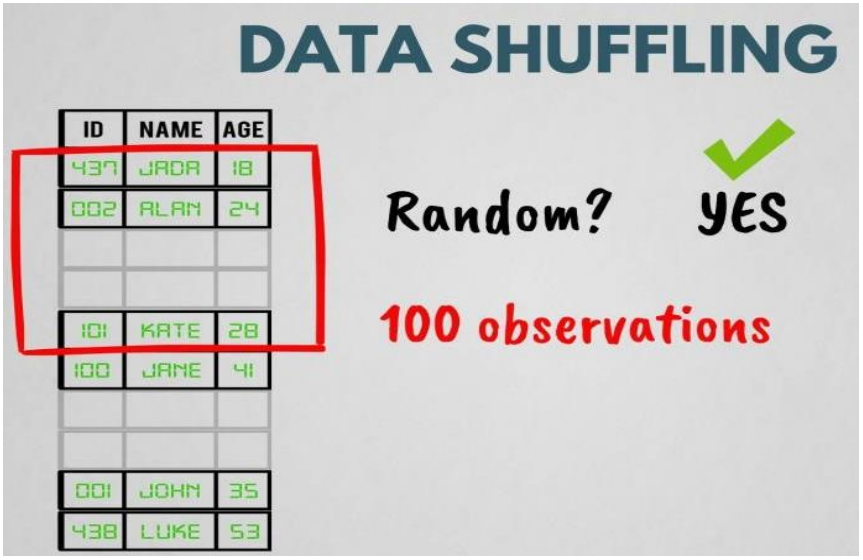


Under these circumstances, the trends you discover will be more towards women. The bestway to counteract this problem is to apply balancing techniques. Such as taking an equal number of respondents from each group, so the ratio is 50/50.



Data shuffling

Shuffling the observations from your data set is just like shuffling a deck of cards. It will ensure that your dataset is free from unwanted patterns caused by problematic data collection.Data shuffling is a technique which improves predictive performance and helps avoid misleading results. But how does it avoid delusive results?



Well, it is a detailed process but, in a nutshell, shuffling is a way to randomize data. If I take the first 100 observations from the dataset that's not a random sample. The top observationswould be extracted first. If I shuffle the data, I am sure that when I take 100 consecutive entries, they'll be random (and most likely representative).

Techniques to process big data

Let's look at some case-specific techniques for dealing with big data.

Text data mining

Think of the huge amount of text that is stored in digital format. Well, there are many scientific projects in progress which aim to extract specific text information from digital sources. For instance, you may have a database which has stored information from academicpapers about 'marketing expenditure', the main topic of your research. You could find the information you need without much of a problem if the number of sources and the volume oftext stored in your database was low enough. Often, though the data is huge. It may contain information from academic papers, blog articles, online platforms, private excel files and more.



This means you will need to extract ‘marketing expenditure’ information from many sources.In other words, **‘big data’.**

Not an easy task, which has led to academics and practitioners developing methods to perform ‘text data mining’.

Data Masking

If you want to maintain a credible business or governmental activity, you must preserve confidential information. When personal details are shared online, you must apply some ‘data masking’ techniques to the information so you can analyze it without compromising the participant’s privacy.



Like data shuffling, ’data masking’ can be complex. It conceals the original data with randomand false data and allows you to conduct analysis and keep all confidential information in a secure place. An example of applying data masking to big data is through ‘confidentiality preserving data mining’ techniques.

Once you finish with data processing, you obtain the valuable and meaningful informationyou need. In a traditional approach, usually the data that is being generated out of the organizations, the financial institutions such as banks or stock markets and the hospitals is given as an input to the ETL System. An ETL System, would then Extract this data, and transform this data, that is, it would convert this data into proper format and finally load this data onto the database. Now the end users can generate reports and perform analytics, by querying this data. But as this data grows, it becomes a very challenging task to manage and process this data, using the traditional approach, this is one of the fundamental drawbacks of using the Traditional Approach.

- Now let us try to understand some of the **major drawbacks** of using the Traditional Approach.
- The 1st drawback is, it an **expensive system**, as it requires a lot of investment for implementing, or upgrading the system, therefore it is, out of the reach of small and mid-sized companies.
- The 2nd drawback is, **scalability**. As the data grows, expanding the system is a challenging task.

- And the 3rd drawback is, it is **time consuming**, it takes lot of time to process and extract, valuable information from the data.

CHAPTER 2

HADOOP

CONTENTS

- Introduction
- Important Features
- How Hadoop Works
- Hadoop Eco Systems

1. INTRODUCTION

Hadoop is an open-source framework, developed by Doug Cutting in 2006, and it is managed by the Apache Software Foundation. The project was named as "Hadoop" after the name of a yellow stuffed toy elephant, which the Doug Cutting's son had. Hadoop is designed to store and process, a huge volume of data, efficiently.

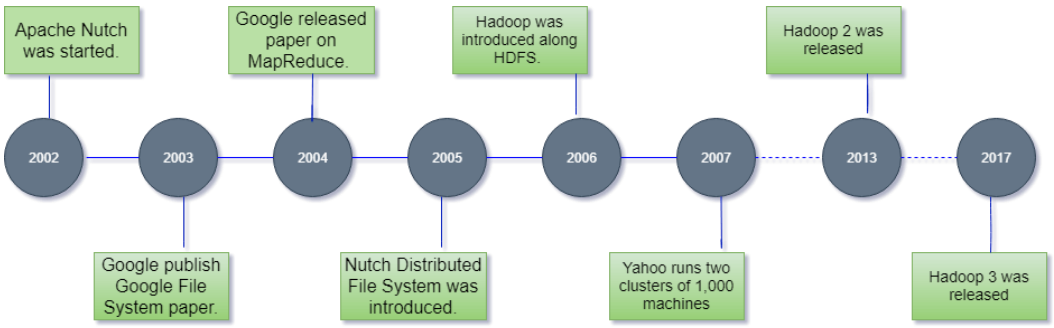


Hadoop is an open-source framework. It is provided by Apache to process and analyze very huge volume of data. It is written in Java and currently used by Google, Facebook, LinkedIn, Yahoo, Twitter etc.

Hadoop is an open-source framework from Apache and is used to store process and analyze data which are very huge in volume. Hadoop is written in Java and is not OLAP (online analytical processing). It is used for batch/offline processing. It is being used by Facebook, Yahoo, Google, Twitter, LinkedIn and many more. Moreover, it can be scaled up just by adding nodes in the cluster.

History of Hadoop

The Hadoop was started by Doug Cutting and Mike Cafarella in 2002. Its origin was theGoogle File System paper, published by Google.



Let's focus on the history of Hadoop in the following steps -

- In 2002, Doug Cutting and Mike Cafarella started to work on a project, **ApacheNutch.** It is an open-source web crawler software project.
- While working on Apache Nutch, they were dealing with big data. To store that data they have to spend a lot of costs which becomes the consequence of that project. This problem becomes one of the important reason for the emergence of Hadoop.
- In 2003, Google introduced a file system known as GFS (Google file system). It is a proprietary distributed file system developed to provide efficient access to data.
- In 2004, Google released a white paper on Map Reduce. This technique simplifies the data processing on large clusters.
- In 2005, Doug Cutting and Mike Cafarella introduced a new file system known as NDFS (Nutch Distributed File System). This file system also includes Map reduce.
- In 2006, Doug Cutting quit Google and joined Yahoo. On the basis of the Nutch project, Dough Cutting introduces a new project Hadoop with a file system known asHDFS (Hadoop Distributed File System). Hadoop first version 0.1.0 released in this year.
- Doug Cutting gave named his project Hadoop after his son's toy elephant.
- In 2007, Yahoo runs two clusters of 1000 machines.
- In 2008, Hadoop became the fastest system to sort 1 terabyte of data on a 900 node cluster within 209 seconds.
- In 2013, Hadoop 2.2 was released.
- In 2017, Hadoop 3.0 was released.

Year	Event
2003	Google released the paper, Google File System (GFS).
2004	Google released a white paper on Map Reduce.
2006	• Hadoop introduced. • Hadoop 0.1.0 released. • Yahoo deploys 300 machines and within this year reaches 600 machines.
2007	• Yahoo runs 2 clusters of 1000 machines. • Hadoop includes HBase.
2008	• YARN JIRA opened • Hadoop becomes the fastest system to sort 1 terabyte of data on a 900-nodecluster within 209 seconds. • Yahoo clusters loaded with 10 terabytes per day. • Cloudera was founded as a Hadoop distributor.
2009	• Yahoo runs 17 clusters of 24,000 machines. • Hadoop becomes capable enough to sort a petabyte. • MapReduce and HDFS become separate subproject.
2010	• Hadoop added the support for Kerberos. • Hadoop operates 4,000 nodes with 40 petabytes. • Apache Hive and Pig released.
2011	• Apache Zookeeper released. • Yahoo has 42,000 Hadoop nodes and hundreds of petabytes of storage.
2012	• Apache Hadoop 1.0 version released.
2013	Apache Hadoop 2.2 version released.
2014	Apache Hadoop 2.6 version released.
2015	Apache Hadoop 2.7 version released.
2017	Apache Hadoop 3.0 version released.
2018	Apache Hadoop 3.1 version released.

2. IMPORTANT FEATURES

1. **Cost Effective System.**

   Hadoop does not require any expensive or specialized hardware, in order tobe implemented. In other words, it can be implemented on a simple hardware, these hardware components are technically referred to as Commodity Hardware.

2. **Large Cluster of Nodes**.

   Therefore, a Hadoop Cluster can be made up of 100's and 1000's of Nodes. One ofthe main advantages of having a large cluster is, offering More Computing Power and a Huge Storage system to the clients.

3. **Parallel Processing of Data**, therefore the data can be processed simultaneously across all the nodes within the cluster, and thus saving a lot of time.

4. **Distributed Data**. The Hadoop Framework takes care of splitting and distributing the data across all the nodes within a cluster. It also replicates the data, over the entire cluster.

5. **Automatic Failover Management**. In case if any of the node, within the cluster fails. The Hadoop Framework would replace that particular machine, with another machine, and it replicates all the configuration settings and the data, from the failed machine onto this newly replicated machine. Admins may need not have to worry about this, once the Automatic Failover Management has been properly configured on a cluster.

6. **Data Locality Optimization.**

   Most important feature. In a traditional approach, whenever a software program is executed, the data is transferred from the datacentre onto the machine, where the program is getting executed.

   For example, let us say, the data required by our program is located at some data centre in USA, and the program that requires this data is located at Singapore. Let usassume the data required by our program is around 1 Petta byte in size. Transferring such a huge volume of data from USA to Singapore, would consume a lot of bandwidth and time.

   Hadoop eliminates this problem, by transferring the code, which is of few megabytes in size, located at Singapore to the datacentre located in USA, and then it, compiles and executes the code locally on the data. Since this code is of few megabytes in size as compared to the input data which is of 1 Petta byte is size, this saves a lot of time and bandwidth.

7. **Heterogeneous Cluster**. Even this can be classified as one of the most important features offered

by Hadoop Framework.

We know that a Hadoop Cluster is made up of several nodes. A HeterogeneousCluster basically refers to a cluster, within which each node can be from a different vendor, and each node can be running a different version and flavor of operating system. Let us say our cluster is made up of 4 nodes.

From Instance, the 1st node is an IBM machine running on Red Hat Enterprise Linux, the 2nd node is an Intel machine running on Ubuntu, the 3rd node is an AMD machine running on Fedora, and the last node is an HP machine running on Cent OS.

8. **Scalability** Scalability refers to the ability of adding or removing the nodes or the hardware components to the cluster. We can easily add or remove a node to or from a Hadoop Cluster without bringing down or affecting the cluster operation. Even we the individual hardware components such as RAM and Hard Drive can be added or removed from a cluster on a fly. HADOOP clusters can easily be scaled to any extent by adding additional cluster nodes and thus allows for the growth of Big Data. Also, scaling does not require modifications to application logic.
9. **Suitable for Big Data Analysis** As Big Data tends to be distributed and unstructured in nature, HADOOP clusters are best suited for analysis of Big Data. Since it is processing logic(not the actual data) that flows to the computing nodes, less network bandwidth is consumed. This concept is called as data locality concept which helps increase the efficiency of Hadoopbased applications.
10. **Fault Tolerance** HADOOP ecosystem has a provision to replicate the input data on to other cluster nodes. That way, in the event of a cluster node failure, data processing can stillproceed by using data stored on another cluster node.

Modules of Hadoop

1. **HDFS** Hadoop Distributed File System. Google published its paper GFS and on thebasis of that HDFS was developed. It states that the files will be broken into blocks and stored in nodes over the distributed architecture.
2. **Yarn** Yet another Resource Negotiator is used for job scheduling and manage thecluster.
3. **Map Reduce** This is a framework which helps Java programs to do the parallel computation on data using key value pair. The Map task takes input data and convertsit into a data set which can be computed in Key value pair. The output of Map task is consumed by reduce task and then the out of reducer gives the desired result.
4. **Hadoop Common** These Java libraries are used to start Hadoop and are used byother Hadoop

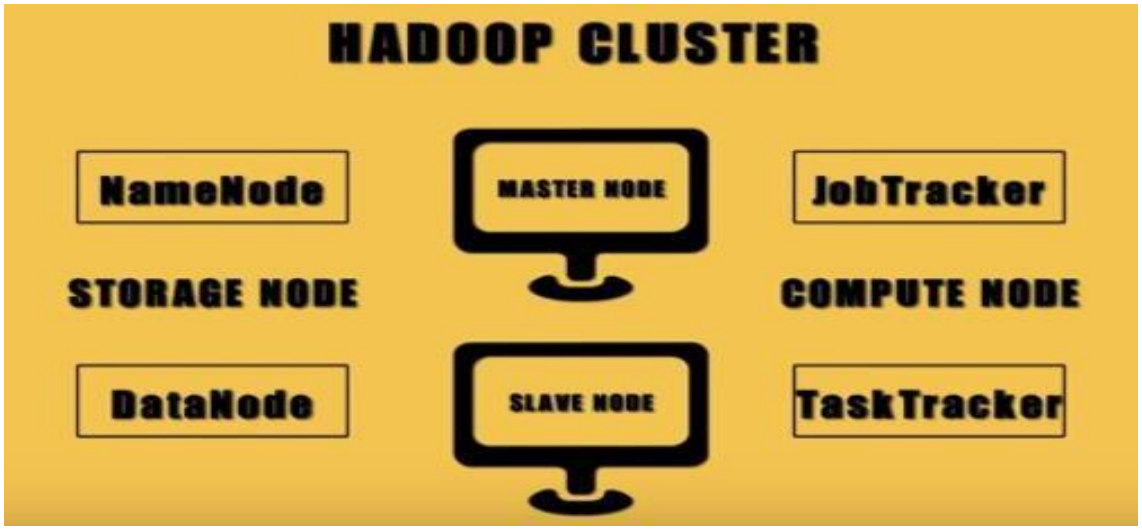
modules.

The Hadoop framework comprises of 2 main components.

The 1st component is the HDFS, HDFS stands for **Hadoop Distributed File System.** The HDFS takes care of storing and managing the data within the Hadoop Cluster.

The 2nd component is the **MapReduce.** Whereas the MapReduce takes careof processing and computing the data, that is present within the HDFS.

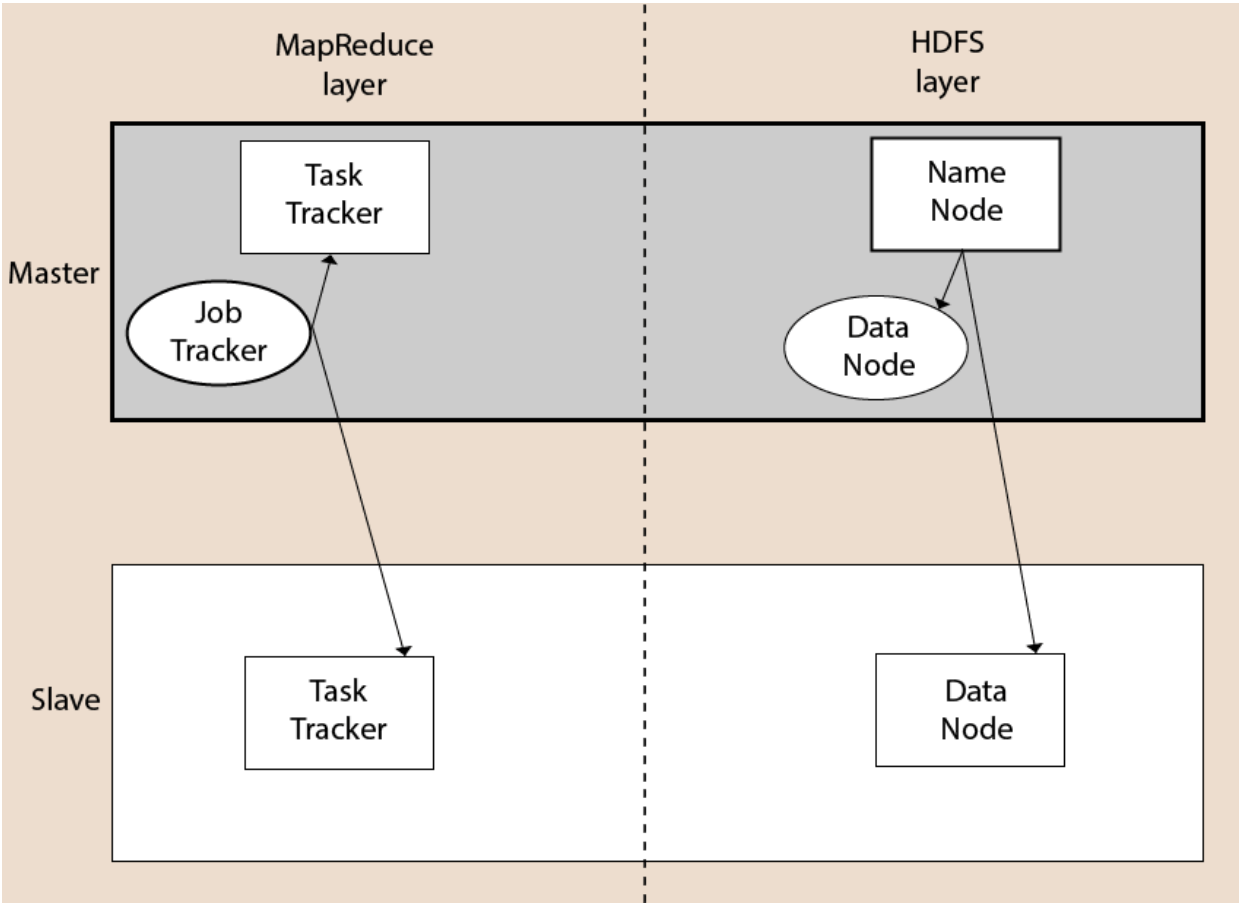
Now let us try to understand what actually makes up a Hadoop Cluster.The 1st one is the Master Node and the 2nd one is the Slave Node. The Master Node, is responsible for running the NameNode and JobTracker daemons. Node is a technical term used to describe a machine or a computer that is present within a cluster. Daemon is atechnical term used to describe a background process running on a Linux machine.The Slave Node, on the other hand is responsible for running the DataNode and TaskTracker daemons. The NameNode and DataNode are responsible for storing and managing the data, and they are commonly referred as Storage Node. Whereasthe JobTracker and TaskTracker are responsible for processing and computing the data, and they are commonly referred to as Compute Node. Usually, the NameNode and JobTracker are configured and running on a single machine. Whereas the DataNode and TaskTracker are configured on multiple machines, but can have instances running on more than one machine at the same time.



3. HOW HADOOP WORKS?

Hadoop Architecture

The Hadoop architecture is a package of the file system, MapReduce engine and the HDFS(Hadoop Distributed File System). The MapReduce engine can be MapReduce/MR1 or YARN/MR2.
A Hadoop cluster consists of a single master and multiple slave nodes. The master nodeincludes Job Tracker, Task Tracker, NameNode, and DataNode whereas the slave nodeincludes DataNode and TaskTracker.



Hadoop Distributed File System

The Hadoop Distributed File System (HDFS) is a distributed file system for Hadoop. It contains a master/slave architecture. This architecture consist of a single NameNode performsthe role of master, and multiple DataNodes performs the role of a slave.

Both NameNode and DataNode are capable enough to run on commodity machines. The Javalanguage is used to develop HDFS. So any machine that supports Java language can easily run the NameNode and DataNode software.

NameNode

- It is a single master server exist in the HDFS cluster.
- As it is a single node, it may become the reason of single point failure.
- It manages the file system namespace by executing an operation like the opening, renaming and closing the files.
- It simplifies the architecture of the system.

DataNode

- The HDFS cluster contains multiple DataNodes.
- Each DataNode contains multiple data blocks.
- These data blocks are used to store data.
- It is the responsibility of DataNode to read and write requests from the file system's clients.
- It performs block creation, deletion, and replication upon instruction from the NameNode.

Job Tracker

- The role of Job Tracker is to accept the MapReduce jobs from client and process the data by using NameNode.
- In response, NameNode provides metadata to Job Tracker.

Task Tracker

- It works as a slave node for Job Tracker.
- It receives task and code from Job Tracker and applies that code on the file. Thisprocess can also be called as a Mapper.

MapReduce Layer

The MapReduce comes into existence when the client application submits the MapReduce job to Job Tracker. In response, the Job Tracker sends the request to the appropriate Task Trackers. Sometimes, the TaskTracker fails or time out. In such a case, that part of the job isrescheduled.

Advantages of Hadoop

- **Fast** In HDFS the data distributed over the cluster and are mapped which helps in faster retrieval. Even the tools to process the data are often on the same servers, thusreducing the processing time. It is able to process terabytes of data in minutes and Peta bytes in hours.
- **Scalable** Hadoop cluster can be extended by just adding nodes in the cluster.
- **Cost Effective** Hadoop is open source and uses commodity hardware to store data soit really cost effective as compared to traditional relational database management system.

Resilient to failure HDFS has the property with which it can replicate data over thenetwork, so if one node is down or some other network failure happens, then Hadoop takes the other copy of data

and use it. Normally, data are replicated thrice but the replication factoris configurable.

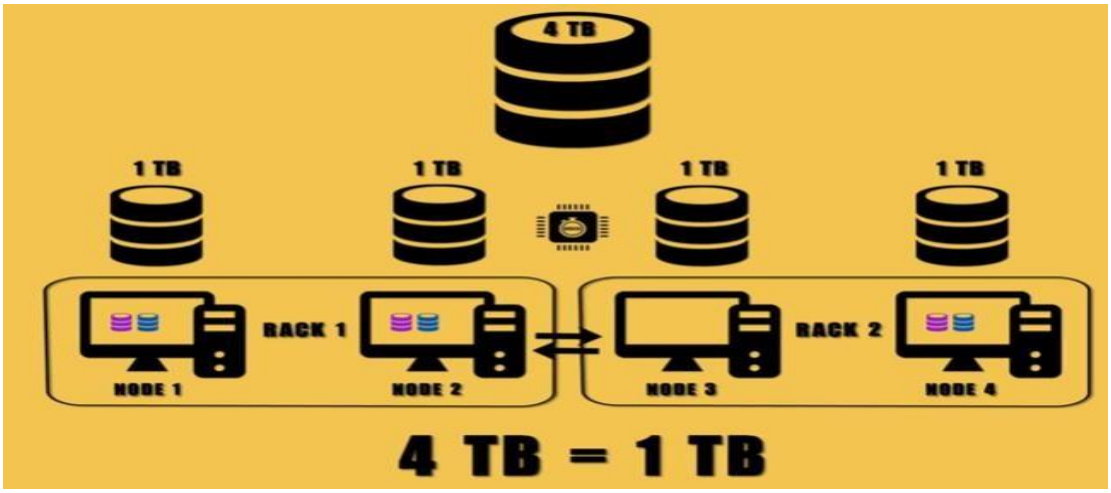
The Hadoop framework comprises of Hadoop Distributed File System and MapReduce framework. Let us try to understand, how the data is managed and processed by the Hadoop framework? The Hadoop framework, divides the data into smaller chunks and stores each part of the data on a separate node within the cluster. Let us say we have around4 terabytes of data and a 4 node Hadoop cluster. The HDFS would divide this data into 4 parts of 1 terabyte each. By doing this, the time taken to store this data onto the disk is significantly reduced. The total time taken to store this entire data onto the disk is equal to storing 1 part of the data, as it will store all the parts of the data simultaneously on different machines.

In order to provide high availability what Hadoop does is, it would replicate each part of the data onto other machines that are present within the cluster. The number of copies it will replicate depends on the "Replication Factor". By default, the replication factor is set to 3.

If we consider, the default replication factor is set, then there will be 3 copies for each part of the data on 3 different machines.

In order to reduce the bandwidth and latency time, it would store 2 copies of the same part of the data, on the nodes that are present within the same rack, and the last copy would be stored on a node, that is present on a different rack.

Let's say Node 1 and Node 2 are on Rack 1 and Node 3 & Node 4 are on Rack 2. Then the 1st 2 copies of part 1 will be stored, on Node 1 and Node 2, and the 3rd copyof part 1, will be stored, either on Node 3 or Node 4. The similar process is followed, for storing remaining parts of the data. Since this data is distributed across the cluster, the HDFS takes care of networking required by these nodes to communicate. Another advantage of distributing this data across the cluster is that, while processing this data,it reduces lot of time, as this data can be processed simultaneously.

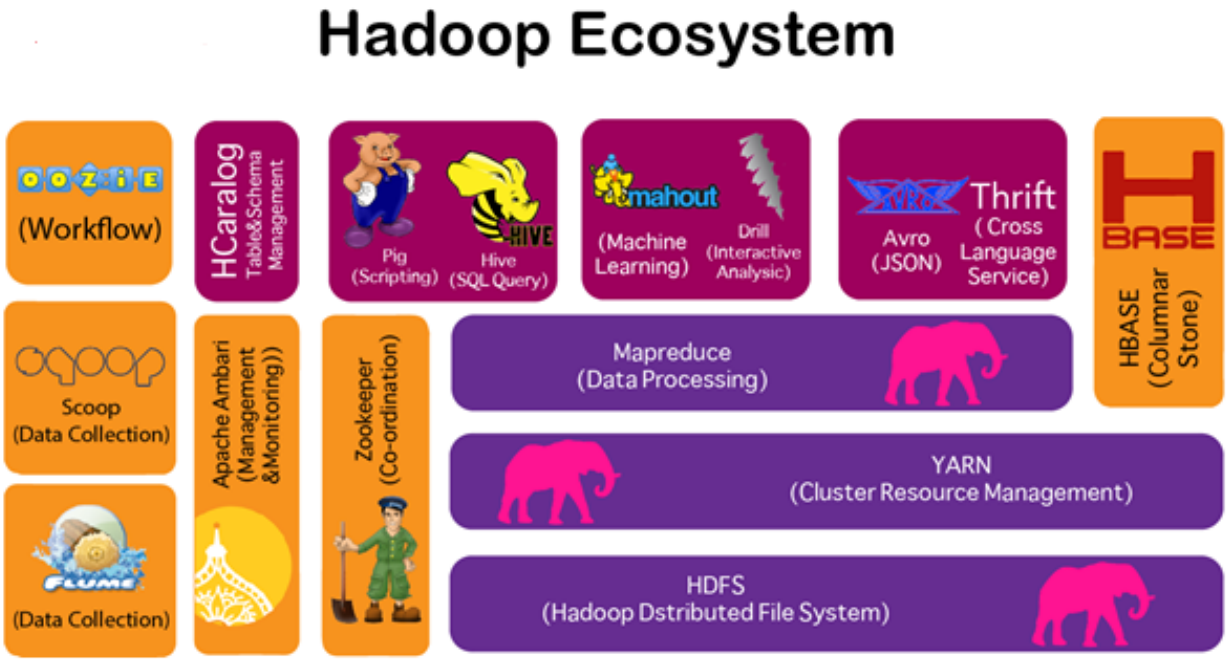


4. HADOOP ECOSYSTEM AND COMPONENTS

Hadoop Ecosystem Overview

Hadoop ecosystem is a platform or framework which helps in solving the big data problems. It comprises of different components and services (ingesting, storing, analyzing, and maintaining) inside of it. Most of the services available in the Hadoop ecosystem are to supplement the main four core components of Hadoop which include HDFS, YARN, MapReduce and Common.

Hadoop ecosystem includes both Apache Open-Source projects and other wide variety of commercial tools and solutions. Some of the well-known open-source examples include Spark, Hive, Pig, Sqoop and Oozie.



HDFS(Hadoop distributed file system)

The Hadoop distributed file system is a storage system which runs on Java programming language and used as a primary storage device in Hadoop applications. HDFS consists of two components, which are Namenode and Datanode; these applications are used to store large data across multiple nodes on the Hadoop cluster. First, let's discuss about the NameNode.

NameNode

- NameNode is a daemon which maintains and operates all DATA nodes (slave nodes).
- It acts as the recorder of metadata for all blocks in it, and it contains information like size, location, source, and hierarchy, etc.

- It records all changes that happen to metadata.
- If any file gets deleted in the HDFS, the NameNode will automatically record it in EditLog.
- NameNode frequently receives heartbeat and block report from the data nodes in the cluster to ensure they are working and live.

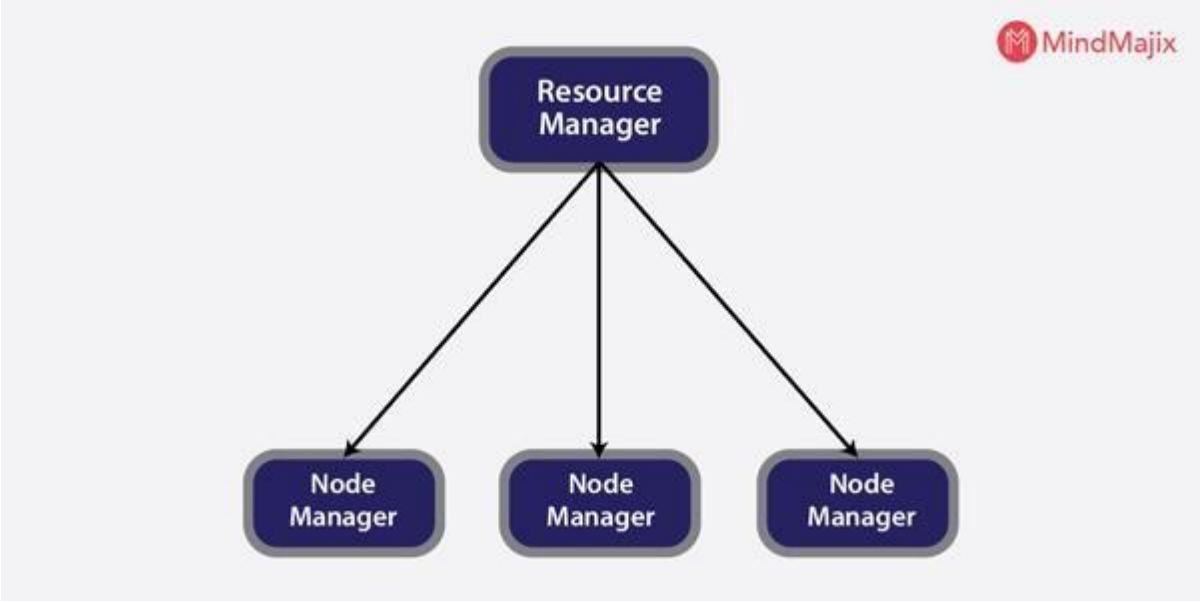
DataNode

- It acts as a slave node daemon which runs on each slave machine.
- The data nodes act as a storage device.
- It takes responsibility to serve read and write request from the user.
- It takes the responsibility to act according to the instructions of NameNode, which includes deleting blocks, adding blocks, and replacing blocks.
- It sends heartbeat reports to the NameNode regularly and the actual time is once in every 3 seconds.

YARN

YARN (Yet Another Resource Negotiator) acts as a brain of the Hadoop ecosystem. It takes responsibility in providing the computational resources needed for the applicationexecutions YARN consists of two essential components. They are Resource Manager and Node Manager

Resource Manager

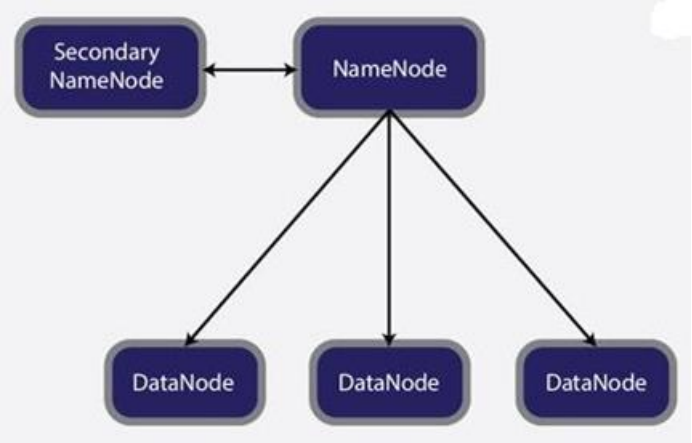


- It works at the cluster level and takes responsibility of or running the master machine.
- It stores the track of heartbeats from the Node manager.
- It takes the job submissions and negotiates the first container for executing an

application.

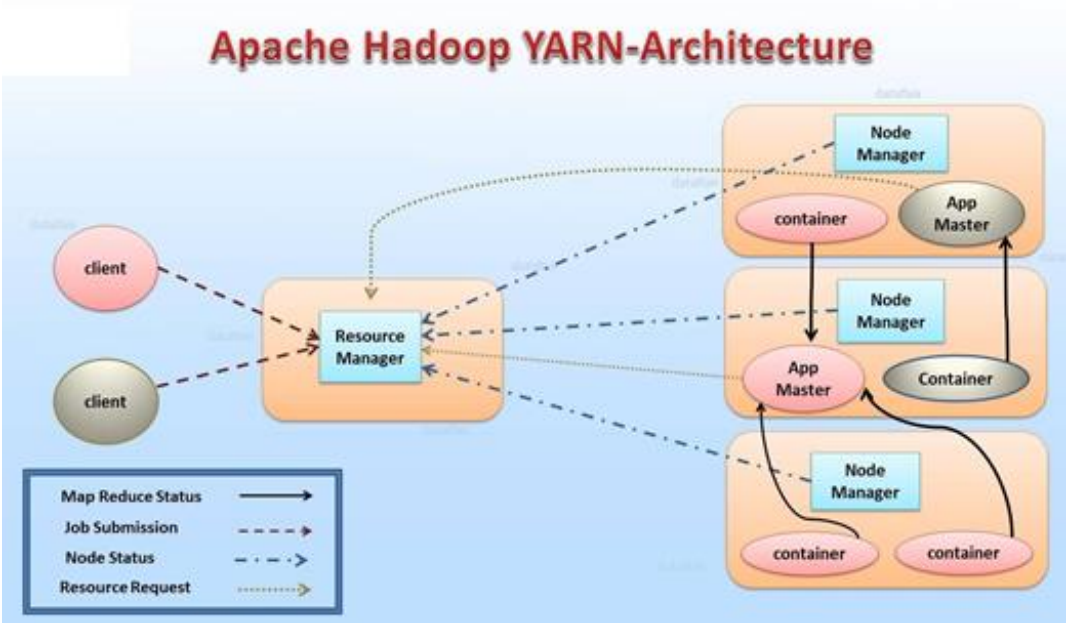
- It consists of two components Application manager and Scheduler.

Node manager



- It works on node level component and runs on every slave machine.
- It is responsible for monitoring resource utilization in each container and managing containers.
- It also keeps track of log management and node health.
- It maintains continuous communication with a resource manager to give updates.

Hadoop YARN (**Y**et **A**nother **R**esource **N**egotiator) is a Hadoop ecosystem component that provides the resource management. Yarn is also one the most important component of Hadoop Ecosystem. YARN is called as the operating system of Hadoop as it is responsible for managing and monitoring workloads. It allows multiple data processing engines such asreal-time streaming and batch processing to handle data stored on a single platform.



Hadoop Yarn Diagram

YARN has been projected as a data operating system for **Hadoop2**. Main features of YARNare

- **Flexibility –** Enables other purpose-built data processing models beyond MapReduce (batch), such as interactive and streaming. Due to this feature of YARN, other applications can also be run along with Map Reduce programs in Hadoop2.
- **Efficiency –** As many applications run on the same cluster, Hence, efficiency of Hadoop increases without much effect on quality of service.
- **Shared –** Provides a stable, reliable, secure foundation and shared operational services across multiple workloads. Additional programming models such as graph processing and iterative modelling are now possible for data processing.

MapReduce

MapReduce acts as a core component in Hadoop Ecosystem as it facilitates the logic of processing. To make it simple, MapReduce is a software framework which enables us in writing applications that process large data sets using distributed and parallel algorithms in a Hadoop environment.

Parallel processing feature of MapReduce plays a crucial role in Hadoop ecosystem. It helps in performing Big data analysis using multiple machines in the same cluster.

How does MapReduce work

In the MapReduce program, we have two Functions; one is Map, and the other is Reduce.

Map function It converts one set of data into another, where individual elements are broken down into tuples. (key /value pairs).

Reduce function It takes data from the Map function as an input. Reduce function aggregates & summarizes the results produced by Map function.

Apache Spark

Apache Spark is an essential product from the Apache software foundation, and it is considered as a powerful data processing engine. Spark is empowering the big data applications around the world. It all started with the increasing needs of enterprises and where MapReduce is unable to handle them.

The growth of large unstructured amounts of data increased need for speed and to fulfill the real-time analytics led to the invention of Apache Spark.



Spark Features

- It is a framework for real-time analytics in a distributed computing environment.
- It acts as an executor of in-memory computations which results in increased speed of data processing compared to MapReduce.
- It is 100X faster than Hadoop while processing data with its exceptional in-memory execution ability and other optimization features.

Spark is equipped with high-level libraries, which support R, Python, Scala, Java etc. These standard libraries make the data processing seamless and highly reliable. Spark can process the enormous amounts of data with ease and Hadoop was designed to store the unstructured data which must be processed. When we combine these two, we get the desired results.

Hive

Apache Hive is a data warehouse open-source software built on Apache Hadoop for performing data query and analysis. Hive mainly does three functions; data summarization, query, and analysis. Hive uses a language called HiveQL (HQL), which is similar to SQL. Hive QL works as a translator which translates the SQL queries into MapReduce Jobs, which will be executed on Hadoop.

Main components of Hive are

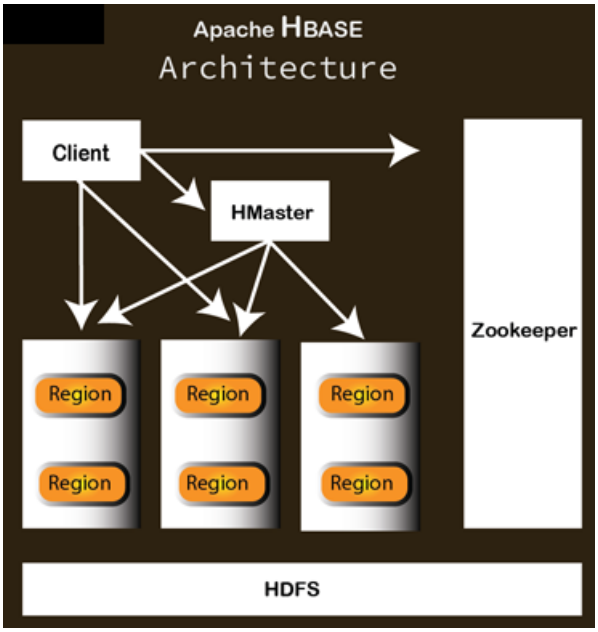
Metastore- It serves as a storage device for the metadata. This metadata holds the information of each table such as location and schema. Metadata keeps track of data and replicates it, and acts as a backup store in case of data loss.

Driver- Driver receives the HiveQL instructions and acts as a Controller. It observes the progress and life cycle of various executions by creating sessions. Whenever HiveQL executes a statement, driver stores the metadata generated out of that action.

Compiler- The compiler is allocated with the task of converting the HiveQL query into MapReduce input. A compiler is designed with the process to execute the steps and functionsneeded to enable the HiveQL output, as required by the MapReduce.

H Base

Hbase is considered as a Hadoop database, because it is scalable, distributed, and because NoSQL database that runs on top of Hadoop. Apache HBase is designed to store the structured data on table format which has millions of columns and billions of rows. HBase gives access to get the real-time data to read or write on HDFS.



HBase features

- HBase is an open source, NoSQL database.
- It is featured after Google's big table, which is considered as a distributed storage system designed to handle big data sets.
- It has a unique feature to support all types of data. With this feature, it plays a crucial role in handling various types of data in Hadoop.
- The HBase is originally written in Java, and its applications can be written in Avro, REST, and Thrift APIs.

Components of HBase

There are majorly two components in HBase. They are HBase master and regional server.

a) HBase master It is not part of the actual data storage, but it manages load balancing activities across all RegionServers.

- It controls the failovers.
- Performs administration activities which provide an interface for creating, updating and deleting tables.
- Handles DDL operations.
- It maintains and monitors the Hadoop cluster.

b) Regional server It is a worker node. It reads, writes, and deletes request from Clients. Region server runs on every node of Hadoop cluster. Its server runs on HDFS data nodes.

H Catalogue

H Catalogue is a table and storage management tool for Hadoop. It exposes the tabular metadata stored in the hive to all other applications of Hadoop. H Catalogue accepts all kinds of components available in Hadoop such as Hive, Pig, and MapReduce to quickly read and write data from the cluster. H Catalogue is a crucial feature of Hive which allows users to store their data in any format and structure.

H Catalogue defaulted supports CSV, JSON, RCFile, ORC file from and sequence File formats.

Benefits of H Catalogue

- It assists the integration with the other Hadoop tools and provides read data from a Hadoop cluster or write data into a Hadoop cluster. It allows notifications of data availability.
- It enables APIs and web servers to access the metadata from hive meta store.
- It gives visibility for data archiving and data cleaning tools.

Apache Pig

Apache Pig is a high-level language platform for analyzing and querying large data sets that are stored in HDFS. Pig works as an alternative language to Java programming for MapReduce and generates MapReduce functions automatically. Pig included with Pig Latin, which is a scripting language. Pig can translate the Pig Latin scripts into MapReduce which can run on YARN and process data in HDFS cluster.

Pig is best suitable for solving complex use cases that require multiple data operations. It is more like a processing language than a query language (exJava, SQL). Pig is considered as a highly customized one because the users have a choice to write their functions by using their preferred scripting language.

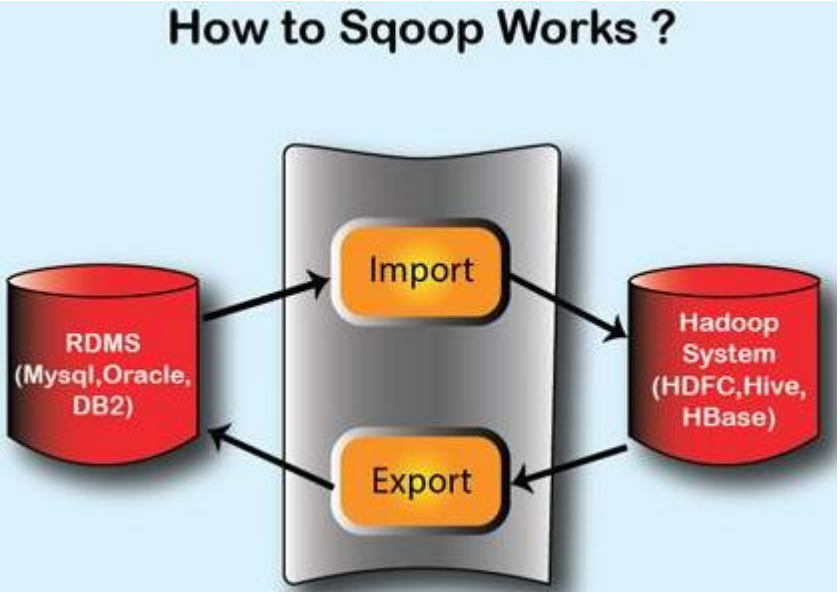
How does Pig work?

We use ‘load’ command to load the data in the pig. Then, we can perform various functions such as grouping data, filtering, joining, sorting etc. At last, you can dump the data on a screen, or you can store the result back in HDFS according to your requirement.

Apache Sqoop

Sqoop works as a front-end loader of Big data. Sqoop is a front-end interface that enables in moving bulk data from Hadoop to relational databases and into variously structured data marts.

Sqoop replaces the function called ‘developing scripts’ to import and export data. It mainly helps in moving data from an enterprise database to Hadoop cluster to performing the ETL process.



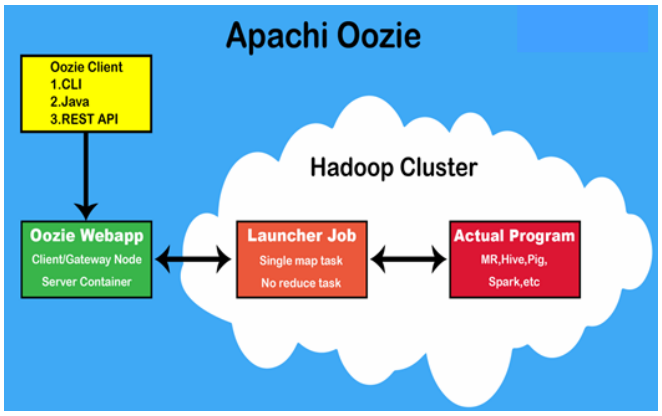
What Sqoop does

Apache Sqoop undertakes the following tasks to integrate bulk data movement between Hadoop and structured databases.

- Sqoop fulfils the growing need to transfer data from the mainframe to HDFS.
- Sqoop helps in achieving improved compression and light-weight indexing for advanced query performance.
- It facilitates feature to transfer data parallelly for effective performance and optimal system utilization.
- Sqoop creates fast data copies from an external source into Hadoop.
- It acts as a load balancer by mitigating extra storage and processing loads to other devices.

Oozie

Apache Ooze is a tool in which all sort of programs can be pipelined in a required manner to work in Hadoop's distributed environment. Oozie works as a scheduler system to run and manage Hadoop jobs.



Oozie allows combining multiple complex jobs to be run in a sequential order to achieve the desired output. It is strongly integrated with Hadoop stack supporting various jobs like Pig, Hive, Sqoop, and system-specific jobs like Java, and Shell. Oozie is an open-source Java web application.

Oozie consists of two jobs

1. **Oozie workflow** It is a collection of actions arranged to perform the jobs one after another. It is just like a relay race where one has to start right after one finish, to complete therace.

2. **Oozie Coordinator** It runs workflow jobs based on the availability of data and predefined schedules.

Avro

Apache Avro is a part of the Hadoop ecosystem, and it works as a data serialization system. Itis an open-source project which helps Hadoop in data serialization and data exchange. Avro enables big data in exchanging programs written in different languages. It serializes data into files or messages.

Avro Schema Schema helps Avaro in serialization and deserialization process without code generation. Avro needs a schema for data to read and write. Whenever we store data in a file it's schema also stored along with it, with this the files may be processed later by any program.

Dynamic typing it means serializing and deserializing data without generating any code. It replaces the code generation process with its statistically typed language as an optional optimization.

Avro features

- Avro makes Fast, compact, dynamic data formats.
- It has Container file to store continuous data format.
- It helps in creating efficient data structures.

Apache Drill

The primary purpose of Hadoop ecosystem is to process the large sets of data either it is structured or unstructured. Apache Drill is the low latency distributed query engine which is designed to measure several thousands of nodes and query petabytes of data. The drill has a specialized skill to eliminate cache data and releases space.

Features of Drill

- It gives an extensible architecture at all layers.
- Drill provides data in a hierarchical format which is easy to process and understandable.
- The drill does not require centralized metadata, and the user doesn't need to create and manage tables in metadata to query data.

Apache Zookeeper

Apache Zookeeper is an open-source project designed to coordinate multiple services in the Hadoop ecosystem. Organizing and maintaining a service in a distributed environment is a complicated task. Zookeeper solves this problem with its simple APIs and Architecture. Zookeeper allows developers to focus on core application instead of concentrating on a distributed environment of the application.

Features of Zookeeper

- Zookeeper acts fast enough with workloads where reads to data are more common than writes.
- Zookeeper acts as a disciplined one because it maintains a record of all transactions.

Apache Flume

Flume collects, aggregates and moves large sets of data from its origin and send it back to HDFS. It works as a fault tolerant mechanism. It helps in transmitting data from a source into a Hadoop environment. Flume enables its users in getting the data from multiple servers immediately into Hadoop.

Apache Ambari

Ambari is an open-source software of Apache software foundation. It makes Hadoop manageable. It consists of software which is capable of provisioning, managing, and monitoring of Apache Hadoop clusters. Let's discuss each concept.

Hadoop cluster provisioning It guides us with a step-by-step procedure on how to install Hadoop services across many hosts. Ambari handles configuration of Hadoop services acrossall clusters.

Hadoop Cluster management It acts as a central management system for starting, stopping and reconfiguring of Hadoop services across all clusters.

Hadoop cluster monitoring Ambari provides us with a dashboard for monitoring health andstatus.

CHAPTER 3

HADOOP DISTRIBUTED FILE SYSTEMS

CONTENTS

- Introduction to HDFS
- HDFS Daemons
- Core Components of HADOOP
- HADOOP Architecture.
    - Name Node
    - Data Node
    - Secondary Name Node
    - Job Tracker
    - Task Tracker
- Reading Data from HDFS
- Writing Data to HDFS.
    - Setting up Development Environment
- Exploring HADOOP Commands
- Rack Awareness.

1. INTRODUCTION TO HDFS (HADOOP DISTRIBUTED FILE SYSTEM)

HDFS stands for **Hadoop Distributed File System**. It is the file system of the Hadoop framework. It was designed to store and manage huge volumes of data in an efficient manner.HDFS has been developed based on the paper published by Google about its file system, known as the **Google File System (GFS).**

HDFS is a **User space File System**. Traditionally file systems are embedded in the operating system kernel and runs as an operating system process. But HDFS is not embedded in the operating system kernel. It runs as a User process within the process space allocated for user processes, on the operating system process table. On a traditional process system, the block size is of **4-8KB** whereas in HDFS the default block size is of 64MB.

- HDFS <- GFS
- Userspace File System

- Default Block Size = 64 MB

With growing data velocity, the data size easily outgrows the storage limit of a machine. A solution would be to store the data across a network of machines. Such filesystems are called *distributed filesystems*. Since data is stored across a network all the complications of anetwork come in.

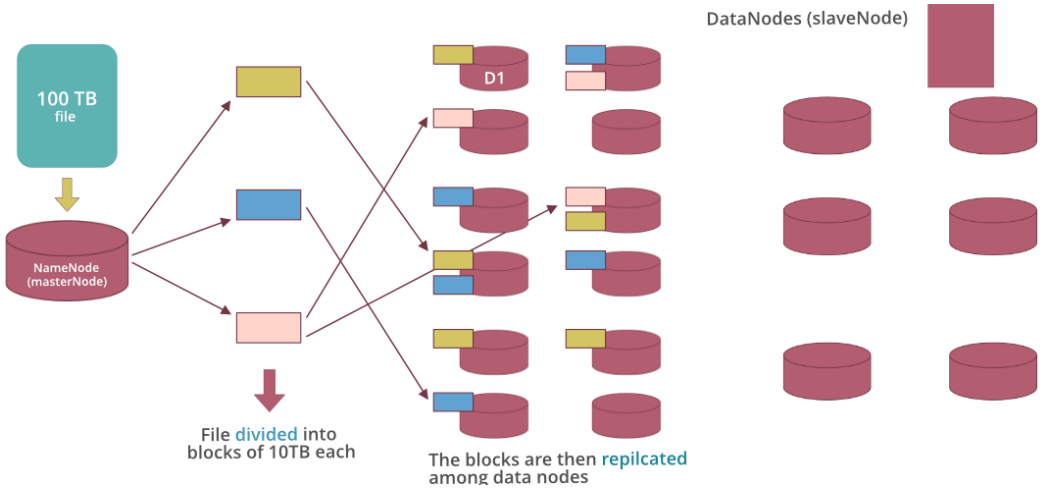
This is where Hadoop comes in. It provides one of the most reliable filesystems. HDFS (Hadoop Distributed File System) is a unique design that provides storage for *extremely largefiles* with streaming data access pattern and it runs on *commodity hardware*. Let's elaborate the terms

- ***Extremely large files:*** Here we are talking about the data in range of petabytes (1000TB).
- ***Streaming Data Access Pattern:*** HDFS is designed on principle of *write-once and read-many-times*. Once data is written large portions of dataset can be processed anynumber times.
- ***Commodity hardware:*** Hardware that is inexpensive and easily available in the market.This is one of feature which specially distinguishes HDFS from other file system.

Nodes Master-slave nodes typically forms the HDFS cluster.

1. **MasterNode**
    - Manages all the slave nodes and assign work to them.
    - It executes filesystem namespace operations like opening, closing, renaming files and directories.
    - It should be deployed on reliable hardware which has the high config. not on commodity hardware.
2. **NameNode**
    - Actual worker nodes, who do the actual work like reading, writing, processing etc.
    - They also perform creation, deletion, and replication upon instruction from the master.
    - They can be deployed on commodity hardware.

Data storage in HDFS Now we see how the data is stored in a distributed manner.



Let's assume that 100TB file is inserted, then master node (name node) will first *divide* the fileinto blocks of 10TB (default size is *128 MB* in Hadoop 2.x and above). Then these blocks are stored across different data nodes (slave node). Data nodes (slave node) *replicate* the blocks among themselves and the information of what blocks they contain is sent to the master.

Default replication factor is *3* means for each block 3 replicas are created (including itself).In hdfs.site.xml we can increase or decrease the replication factor i.e we can edit its configuration here.

Note Master Node has the record of everything, it knows the location and info of each and every single data node and the blocks they contain, i.e., nothing is done without the permission of master node.

Why divide the file into blocks?

Answer Let's assume that we don't divide, now it's very difficult to store a 100 TB file on a single machine. Even if we store, then each read and write operation on that *whole file* is going to take very high seek time. But if we have multiple blocks of size 128MB then it's become easy to perform various read and write operations on it compared to doing it on a whole file at once. So, we divide the file to have faster data access i.e. reduce seek time.

Why replicate the blocks in data nodes while storing

Answer Let's assume we don't replicate and only one yellow block is present on datanode D1. Now if the data node D1 crashes we will lose the block and which will make the overalldata inconsistent and faulty. So we replicate the blocks to achieve *fault-tolerence.*

Terms related to HDFS

- ***HeartBeat:*** It is the signal that datanode continuously sends to namenode. If namenode doesn't receive heartbeat from a datanode then it will consider it dead.
- ***Balancing:*** If a datanode is crashed the blocks present on it will be gone too and the blocks will be *under-replicated* compared to the remaining blocks. Here master node(namenode) will give a signal to datanodes containing replicas of those lost blocks

to replicate so that overall distribution of blocks is balanced.

- ***Replication:*** It is done by datanode.

Note No two replicas of the same block are present on the same datanode.

Features

- Distributed data storage.
- Blocks reduce seek time.
- The data is highly available as the same block is present at multiple datanodes.
- Even if multiple datanodes are down we can still do our work, thus making it highly reliable.
- High fault tolerance.

Limitations Though HDFS provide many features there are some areas where it doesn'twork well.

- **Low latency data access** Applications that require low-latency access to data i.e inthe range of milliseconds will not work well with HDFS, because HDFS is designedkeeping in mind that we need high-throughput of data even at the cost of latency.
- **Small file problem** Having lots of small files will result in lots of seeks and lots of movement from one datanode to another datanode to retrieve each small file, this whole process is a very inefficient data access pattern.

Advantages of HDFS

- It can be implemented on commodity hardware.
- It is designed for large files of size up to GB/TB.
- It is suitable for streaming data access, that is, data is written once but read multiple times. For example, Log files where the data is written once but read multiple times.
- It performs automatic recovery of file system upon when a fault is detected.

Disadvantages of HDFS

- It is not suitable for files that are small in size.
- It is not suitable for reading data from a random position in a file. It is best suitable for reading data either from beginning or end of the file.
- It does not support writing of data into the files using multiple writers.

The reasons why HDFS works so well with Big Data

- **HDFS** uses the method of MapReduce for access to data which is **very fast**
- It follows a **data coherency model** that is simple yet **highly robust** and **scalable**
- **Compatible** with **any** commodity **hardware** and **operating system**
- Achieves economy by distributing data and processing on clusters with parallel nodes

- **Data is always safe** as it is automatically saved in multiple locations in a foolproof way
- It provides a **JAVA API** and even a **C** language wrapper on top
- It is **easily accessible** using a web browser making it **highly utilitarian.**

2. HADOOP DAEMONS

Deamons are the processes running in background.

Namenodes

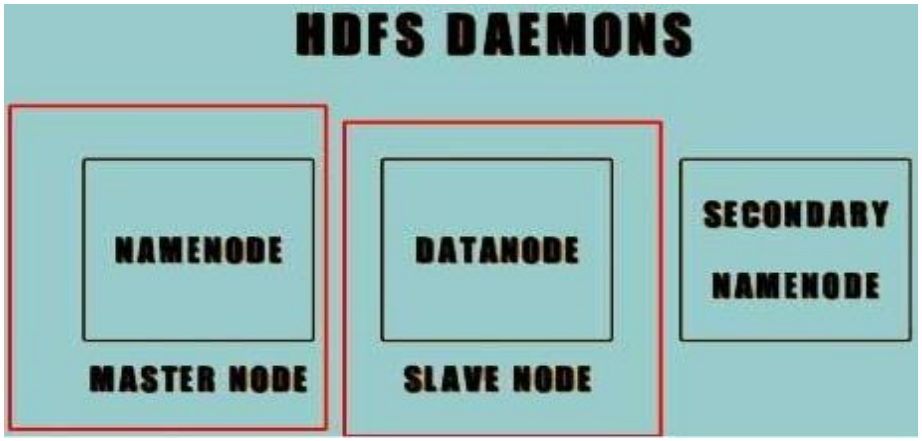
- Run on the master node.
- Store metadata (data about data) like file path, the number of blocks, block Ids. etc.
- Require high amount of RAM.
- Store meta-data in RAM for fast retrieval i.e to reduce seek time. Though a persistent copy of it is kept on disk.

DataNodes

- Run on slave nodes.
- Require high memory as data is actually stored here.

The HDFS consists of three Daemons which are-

- Namenode
- Datanode
- Secondary Namenode.

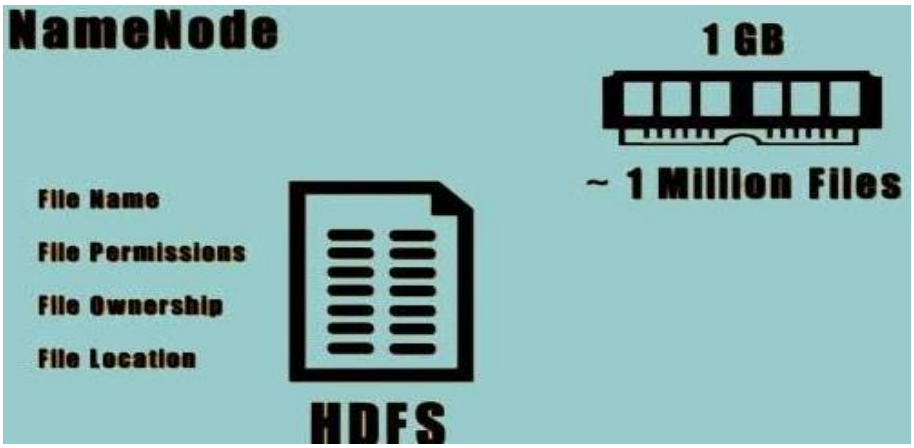


The Namenode is the master node while the data node is the slave node. Within the HDFS, there is only a single Namenode and multiple Datanodes.

Functionality of Nodes

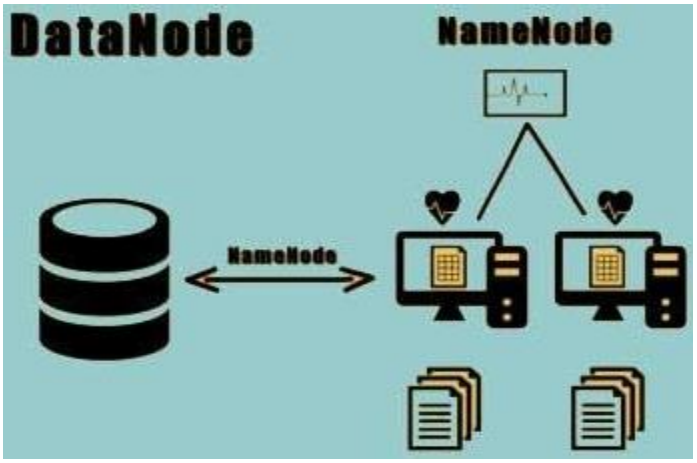
The Namenode is used for storing the **metadata of the HDFS**. This metadata keeps track and stores information about all the files in the HDFS. All the information is stored in the RAM.

Typically, the Namenode occupies around 1 GB of space to store around 1 million files.



The information stored in the RAM is known as file system metadata. This metadata is stored in a file system on a disc.

The Datanodes are responsible for retrieving and storing information as instructed by the Namenode. They periodically report back to the Namenodes about their status and the files they are storing through a heartbeat. The Datanodes stores multiple copies for each file that ispresent within the Hadoop distributed file system.



Secondary NameNode

The role played by the Secondary NameNode in managing the file system metadata.

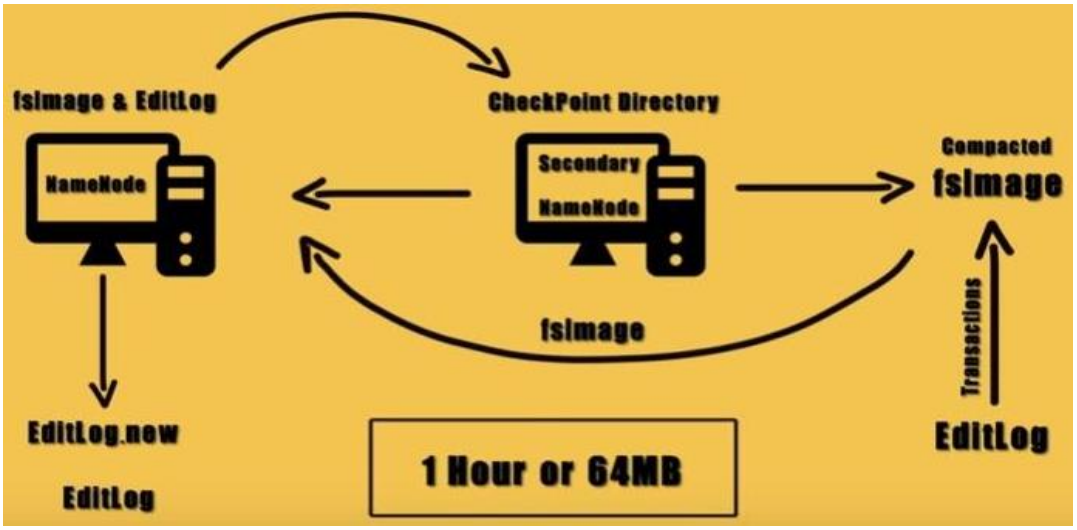
We all know that each and every transaction records on the file system, recorded within theEditLog file.

At some point of time this file becomes very large. At this point of time if the NameNode fails due to corrupted metadata or any other reason, then it has to retrieve the fsImage from the disc and apply all the transactions to it doesn't look into EditLog file.

In order to apply all these transactions, the system resources should be available. It also takes lot of time to apply all these transactions. Until as these transactions are not applied the contents of fsImage are inconsistent. Hence the cluster cannot be operational.

Now let us see how the Secondary NameNode can be used to prevent this situation from occurring.

Secondary NameNode instructs the NameNode to record the transactions in new Edit file. Now the Secondary NameNode copies the fsImage and EditLog file to its CheckPoint Directory. Once this files accommodate, the Secondary NameNode loads the fsImage and applies all the transactions from the EditLog file and stores this information in new compacted fsImage file. Secondary NameNode transfers this compacted fsImage file to the NameNode. The NameNode adopts this new fsImage file and also renames the new EditFile. This process occurs every hour or whenever the size of the edit log file is 64MB.



Core Components of HADOOP

Hadoop is an open-source software framework for *distributed Storage* and *processing* of large datasets. Apache Hadoop main components are

- HDFS
- MapReduce
- YARN

HDFS- Hadoop distributed file system (HDFS) is the primary storage system of Hadoop. HDFS store very large files running on a cluster of *commodity hardware.* It works on principle of storage of less number of large files rather than the huge number of small files. HDFS stores data reliably even in the case of hardware failure. It provides highest throughput access to application by accessing in parallel.

MapReduce- MapReduce is the data processing layer of Hadoop. It writes application that process large structured and unstructured data stored in HDFS. It processes huge amount ofdata in parallel. It does this by dividing the job (submitted job) into a set of independent tasks (sub-job). In Hadoop, MapReduce works by breaking the processing into phases *Map and Reduce*. The **Map** is the first phase of processing, where we specify all the complex logic code. **Reduce** is the second

phase of processing. Here we specify light-weight processing likeaggregation/summation.

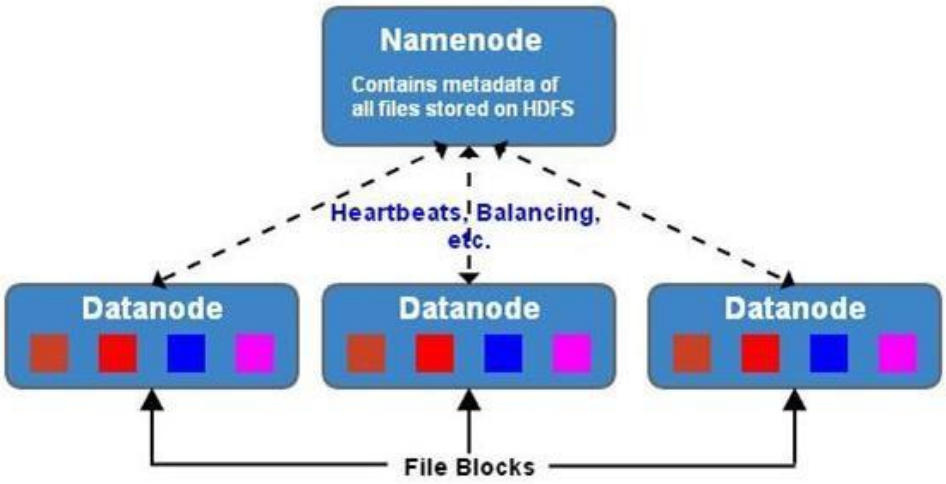
YARN- YARN is the processing framework in Hadoop. It provides ***Resource management***, and allows multiple data processing engines, for example real-time streaming, data science and batch processing.

Hadoop is designed for parallel processing into a distributed environment, so Hadoop requires such a mechanism which helps users to answer these questions. In 2003 Google has published two white papers Google File System (GFS) and MapReduce framework. Dug Cutting had read these papers and designed file system for hadoop which is known as Hadoop Distributed File System (HDFS) and implemented a MapReduce framework on this file system to process data. This has become the core components of Hadoop.

Hadoop Distributed File System

HDFS is a virtual file system which is scalable, runs on commodity hardware and provides high throughput access to application data. It is a data storage component of Hadoop. It stores its data blocks on top of the native file system. It presents a single view of multiple physical disks or file systems. Data is distributed across the nodes; node is an individual machine in a cluster and cluster is a group of nodes. It is designed for applications which need a write-once-read-many access. It does not allow modification of data once it is written. Hadoop has a master/slave architecture. The Master of HDFS is known as **Namenode** and Slave is known as **Datanode**.

Architecture



Architecture

Namenode

It is a deamon which runs on master node of hadoop cluster. There is only one namenode in a cluster. It contains metadata of all the files stored on HDFS which is known as namespace of HDFS. It maintain two files EditLog, record every change that occurs to file system metadata (transaction history) and FsImage, which stores entire namespace, mapping of blocks to files and

file system properties. The FsImage and the EditLog are central data structures of HDFS.

Datanode

It is a deamon which runs on slave machines of Hadoop cluster. There are number of datanodes in a cluster. It is responsible for serving read/write request from the clients. Italso performs block creation, deletion, and replication upon instruction from the Namenode. It also sends a Heartbeat message to the namenode periodically about the blocks it holds. Namenode and Datanode machines typically run a GNU/Linux operating system (OS).

Following are some of the characteristics of HDFS,

1) **DataIntegrity**

When a file is created in HDFS, it computes a checksum of each block of the file and stores this checksum in a separate hidden file. When a client retrieves file contents, it verifies that the data it received matches the checksum stored in the associated checksumfile.

2) **Robustness**

The primary objective of HDFS is to store data reliably even in the presence of failures. The three common types of failures are NameNode failures, DataNode failures and network partitions.

3) **ClusterRebalancing**

The HDFS is compatible with data re balancing that means it will automatically move the data from one datanode to another, if free space on datanode falls below a certain threshold.

4) **Accessibility**

It can be accessed from applications in many different ways. Hadoop provides a Java API for applications to use. An HTTP browser can also be used to browse the files of an HDFS instance using default web interface of hadoop.

5) **Re-replication**

When a datanode send heartbeats to namenode and if any block is missing then namenode mark that block as dead. This dead block is re-replicated from the other datanode. Re- replication arise when a datanode become unavailable, a replica is corrupted, a hard disk may fail, or the replication factor value is increased.

MapReduce Framework

In general MapReduce is a programming model which allows to process large data sets with a parallel, distributed algorithm on a cluster. Hadoop uses this model to process data which is stored on HDFS. It splits a task across the processes. Generally, we send data tothe process but in MapReduce we send process to the data which decreases network overhead.

MapReduce job is an analysis work that we want to run on data, which is broken down into multiple task because the data is stored on different nodes which can run paralleled. AMapReduce

program processes data by manipulating (key/value) pairs in the general form

map (K1,V1) ? list(K2,V2)

reduce (K2,list(V2)) ? list(K3,V3)

Following are the phases of MapReduce job,

1) Map

In this phase we simultaneously ask our machines to run a computation on their local block of data. As this phase completes, each node stores the result of its computation in temporary local storage, this is called the "intermediate data". Please note that the outputof this phase is written to the local disk, not to the HDFS.

2) Combine

Sometime we want to perform a local reduce before we transfer result to reduce task. In such scenarios we add combiner to perform local reduce task. It is a reduce task whichruns on local data. For example, if the job processes a document containing the word "the" 574 times, it is much more efficient to store and shuffle the pair ("the", 574) once instead of the pair ("the", 1) multiple times. This processing step is known as combining.

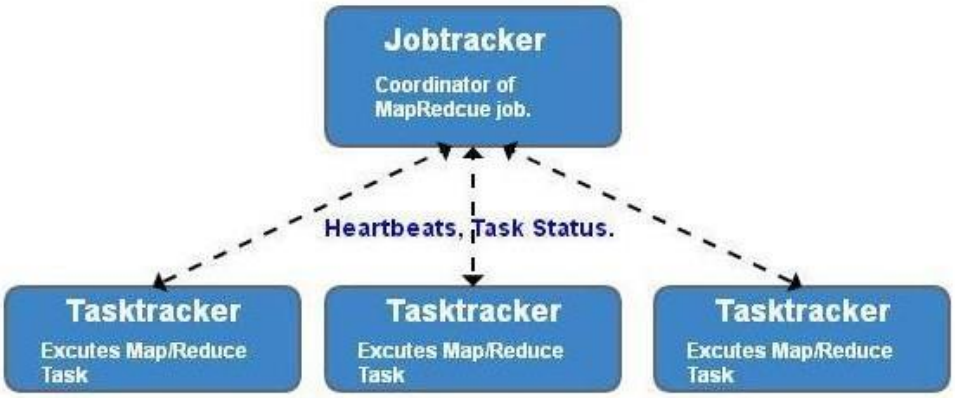
3) Partition

In this phase partitioner will redirect the result of mappers to different reducers. When there are multiple reducers, we need some ways to determine the appropriate one to send a (key/value) pair outputted by a mapper.

4) Reduce

The Map task on the machines have completed and generated their intermediate data. Now we need to gather all of this intermediate data to combine it for further processing such that we have one final result. Reduce task run on any of the slave nodes. When the reduce task receives the output from the various mappers, it sorts the incoming dataon the key of the (key/value) pair and groups together all values of the same key.

The Master of MapReduce engine is known as Jobtracker and Slave is known as Tasktracker.



Jobtracker

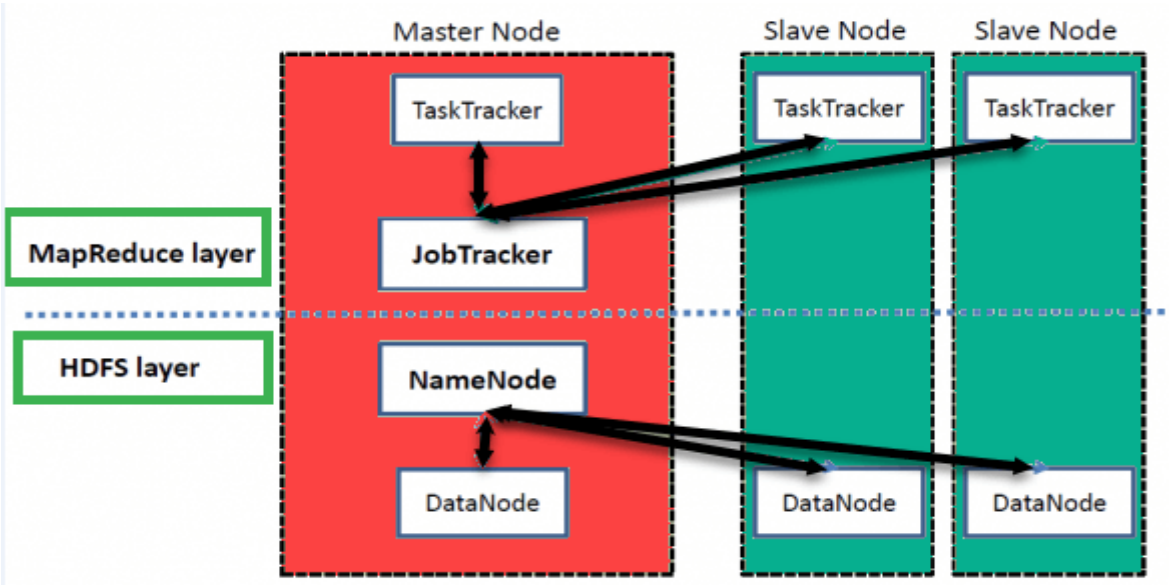
Jobtracker is a coordinator of the MapReduce job which runs on master node. When the client machine submits the job then it first consults Namenode to know about which datanode have blocks of file which is input for the submitted job. The Job Tracker then provides the Task Tracker running on those nodes with the Java code required to execute job.

Tasktracker

Tasktracker runs actual code of job on the data blocks of input file. It also sends heartbeats and task status back to the jobtracker.

If the node running the map task fails before the map output has been consumed by the reduce task, then Jobtracker will automatically rerun the map task on another node to re- create the map output that is why it is known as self-hexaling system.

3. HADOOP ARCHITECTURE



High Level Hadoop Architecture

Hadoop has a Master-Slave Architecture for data storage and distributed data processing using MapReduce and HDFS methods.

NameNode NameNode represented every files and directory which is used in the namespace

DataNode DataNode helps you to manage the state of an HDFS node and allows you to interacts with the blocks

MasterNode The master node allows you to conduct parallel processing of data using Hadoop MapReduce.

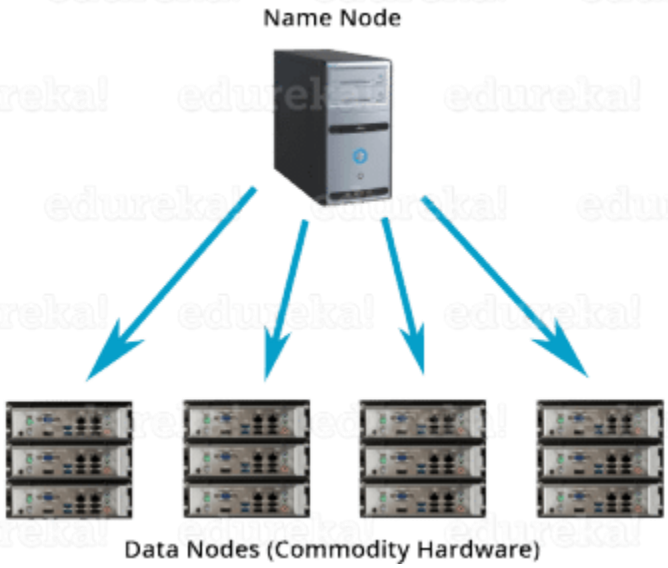
Slave node The slave nodes are the additional machines in the Hadoop cluster which allows you to store data to conduct complex calculations. Moreover, all the slave node comes with Task Tracker and a DataNode. This allows you to synchronize the processes with the NameNode and Job

Tracker respectively.

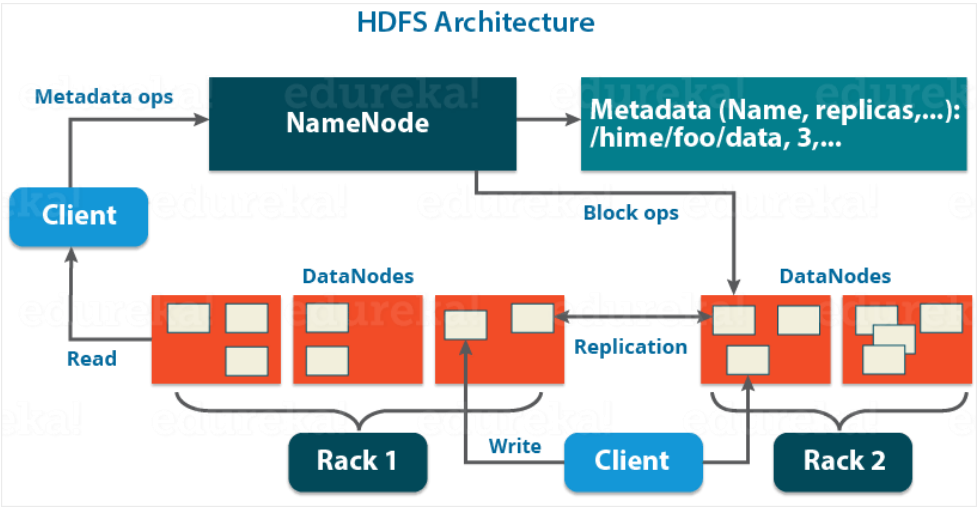
JobTracker is a master which creates and runs the job. JobTracker which can run on the NameNode allocates the job to tasktrackers. It is tracking resource availability and task life cycle management, tracking its progress, fault tolerance etc.

TaskTracker run the tasks and report the status of task to JobTracker. TaskTracker run on DataNodes. It has function of following the orders of the job tracker and updating the job tracker with its progress status periodically.

HDFS Architecture



Apache HDFS or **Hadoop Distributed File System** is a block-structured file system where each file is divided into blocks of a pre-determined size. These blocks are stored across a cluster of one or several machines. Apache Hadoop HDFS Architecture followsa *Master/Slave Architecture*, where a cluster comprises of a single NameNode (Master node) and all the other nodes are DataNodes (Slave nodes). HDFS can be deployed on a broad spectrum of machines that support Java. Though one can run several DataNodes on a single machine, but in the practical world, these DataNodes are spread across various machines.



NameNode

NameNode is the master node in the Apache Hadoop HDFS Architecture that maintains and manages the blocks present on the DataNodes (slave nodes). NameNode is a very highly available server that manages the File System Namespace and controls access to files by clients. I will be discussing this High Availability feature of Apache Hadoop HDFS in my next blog. The HDFS architecture is built in such a way that the user data never resides on theNameNode. The data resides on DataNodes only.

Functions of NameNode

- It is the master daemon that maintains and manages the DataNodes (slave nodes)
- It records the metadata of all the files stored in the cluster, e.g. The location of blocks stored, the size of the files, permissions, hierarchy, etc. There are two files associated with the metadata
    - **FsImage** It contains the complete state of the file system namespace since the start of the NameNode.
    - **EditLogs** It contains all the recent modifications made to the file system with respect to the most recent FsImage.
- It records each change that takes place to the file system metadata. For example, if a file is deleted in HDFS, the NameNode will immediately record this in the EditLog.
- It regularly receives a Heartbeat and a block report from all the DataNodes in the cluster to ensure that the DataNodes are live.
- It keeps a record of all the blocks in HDFS and in which nodes these blocks are located.
- The NameNode is also responsible to take care of the **replication factor** of all the blocks which we will discuss in detail later in this HDFS tutorial blog.
- In **case of the DataNode failure**, the NameNode chooses new DataNodes for new replicas, balance disk usage and manages the communication traffic to the DataNodes.

DataNode

DataNodes are the slave nodes in HDFS. Unlike NameNode, DataNode is a commodity hardware, that is, a non-expensive system which is not of high quality or high-availability. The DataNode is a block server that stores the data in the local file ext3 or ext4.

Functions of DataNode

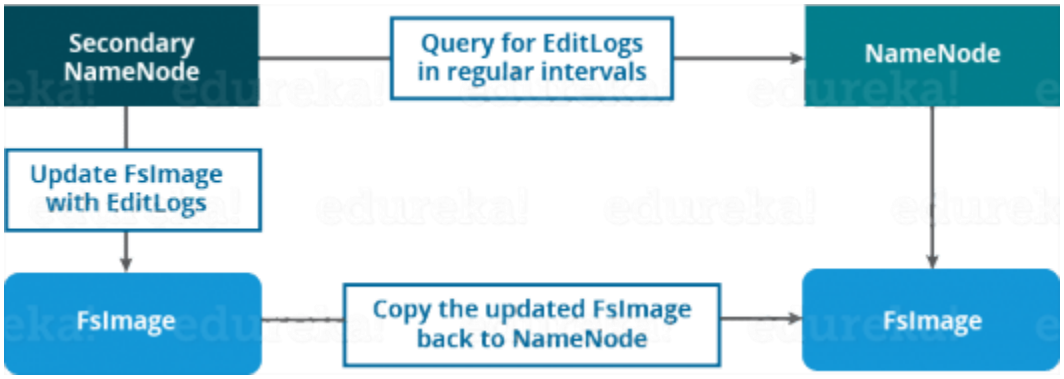
- These are slave daemons or process which runs on each slave machine.
- The actual data is stored on DataNodes.
- The DataNodes perform the low-level read and write requests from the file system's clients.

- They send heartbeats to the NameNode periodically to report the overall health of HDFS, by default, this frequency is set to 3 seconds.

Till now, you must have realized that the NameNode is pretty much important to us. If it fails, we are doomed. But don't worry, we will be talking about how Hadoop solved this single point of failure problem in the next Apache Hadoop HDFS Architecture blog. So, just relax for now and let's take one step at a time.

Secondary NameNode

Apart from these two daemons, there is a third daemon or a process called Secondary NameNode. The Secondary NameNode works concurrently with the primary NameNode asa **helper daemon.** And don't be confused about the Secondary NameNode being a **backup NameNode because it is not.**



Functions of Secondary NameNode

- The Secondary NameNode is one which constantly reads all the file systems and metadata from the RAM of the NameNode and writes it into the hard disk or the file system.
- It is responsible for combining the EditLogs with FsImage from the NameNode.
- It downloads the EditLogs from the NameNode at regular intervals and applies to FsImage. The new FsImage is copied back to the NameNode, which is used whenever the NameNode is started the next time.

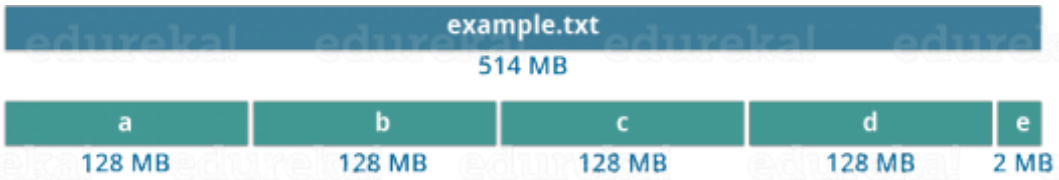
Hence, Secondary NameNode performs regular checkpoints in HDFS. Therefore, it is alsocalled CheckpointNode.

Blocks

Now, as we know that the data in HDFS is scattered across the DataNodes as blocks. Let's have a look at what is a block and how is it formed?

Blocks are the nothing but the smallest continuous location on your hard drive where data is stored. In general, in any of the File System, you store the data as a collection of blocks. Similarly, HDFS stores each file as blocks which are scattered throughout the Apache Hadoop cluster. The

default size of each block is 128 MB in Apache Hadoop 2.x (64 MB in Apache Hadoop 1.x) which you can configure as per your requirement.



It is not necessary that in HDFS, each file is stored in exact multiple of the configured block size (128 MB, 256 MB etc.). Let's take an example where I have a file "example.txt" of size 514 MB as shown in above figure. Suppose that we are using the default configuration of block size, which is 128 MB. Then, how many blocks will be created? 5, Right. The first fourblocks will be of 128 MB. But, the last block will be of 2 MB size only.

NameNode, DataNode And Secondary NameNode in HDFS

HDFS has a **master/slave** architecture. Within an HDFS cluster there is a single NameNode and a number of DataNodes, usually one per node in the cluster.

In this post we'll see in detail what NameNode and DataNode do in Hadoop framework. Apart from that we'll also talk about Secondary NameNode in Hadoop which can take some of the work load of the NameNode.

NameNode in HDFS

The NameNode is the centerpiece of an HDFS file system. NameNode manages the file system namespace by storing information about the file system tree which contains the metadata about all the files and directories in the file system tree.

Metadata stored about the file consists of file name, file path, number of blocks, block Ids, replication level.

This metadata information is stored on the local disk. Namenode uses two files for storing this metadata information.

- FsImage
- EditLog

We'll discuss these two files, FsImage and EditLog in more detail in the Secondary NameNode section.

NameNode in Hadoop also keeps, location of the DataNodes that store the blocks for any given file, in it's memory. Using that information Namenode can reconstruct the whole file by getting the location of all the blocks of a given file.

Client application has to talk to NameNode to add/copy/move/delete a file. Since block information is also stored in NameNode so any client application that wishes to use a file has to get

BlockReport from NameNode. The NameNode returns list of DataNodes where the data blocks are stored for the given file.

DataNode in HDFS

Data blocks of the files are stored in a set of DataNodes in Hadoop cluster.

Client application gets the list of DataNodes where data blocks of a particular file are stored from NameNode. After that DataNodes are responsible for serving read and write requests from the file system's clients. Actual user data never flows through NameNode.

The DataNodes store blocks, delete blocks and replicate those blocks upon instructions from the NameNode.

DataNodes in a Hadoop cluster periodically send a blockreport to the NameNode too. A blockreport contains a list of all blocks on a DataNode.

Secondary NameNode in HDFS

Secondary NameNode in Hadoop is more of a helper to NameNode, it is **not a backup NameNode server** which can quickly take over in case of NameNode failure. Before going into details about Secondary NameNode in HDFS let's go back to the two fileswhich were mentioned while discussing NameNode in Hadoop– FsImage and EditLog.

- **EditLog**– All the file write operations done by client applications are first recorded inthe EditLog.
- **FsImage**– This file has the complete information about the file system metadata when the NameNode starts. All the operations after that are recorded in EditLog.

When the NameNode is restarted it first takes metadata information from the FsImage and then apply all the transactions recorded in EditLog. NameNode restart doesn't happen that frequently so EditLog grows quite large. That means merging of EditLog to FsImage at the time of startup takes a lot of time keeping the whole file system offline during that process.

Now you may be thinking only if there is some entity which could take over this job of merging FsImage and EditLog and keep the FsImage current that will save a lot of time. That's exactly what Secondary NameNode does in Hadoop. Its main function is to checkpoint the file system metadata stored on NameNode.

The process followed by Secondary NameNode to periodically merge the fsimage and the edits log files is as follows-

- Secondary NameNode gets the latest FsImage and EditLog files from theprimary NameNode.
- Secondary NameNode applies each transaction from EditLog file to FsImageto create a new merged FsImage file.

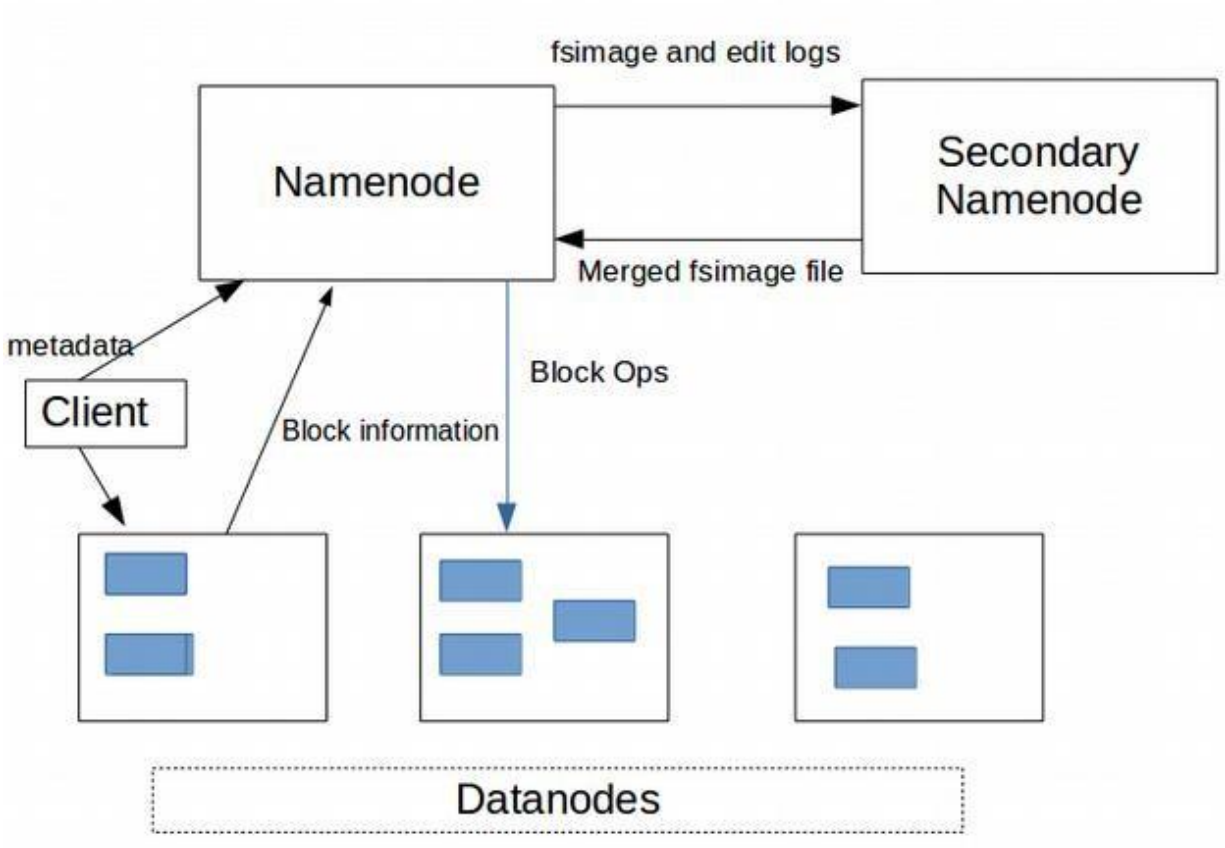
- Merged FsImage file is transferred back to primary NameNode.

The start of the checkpoint process on the secondary NameNode is controlled by two configuration parameters which are to be configured in **hdfs-site.xml**.

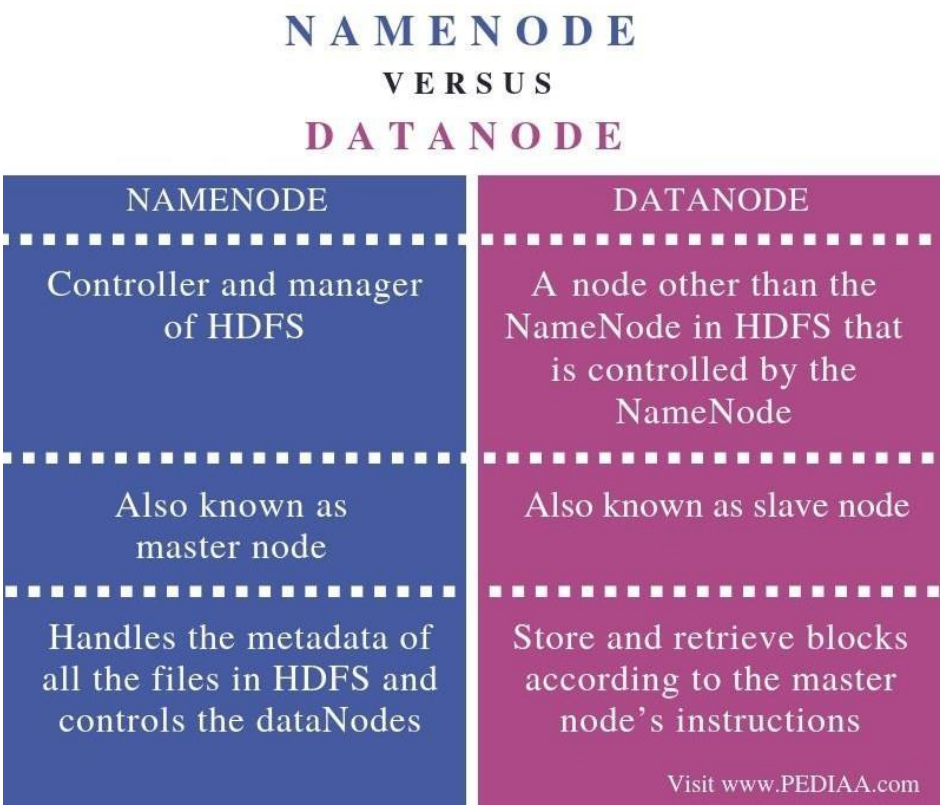
dfs.namenode.checkpoint.period - This property specifies the maximum delay between two consecutive checkpoints. Set to 1 hour by default.

dfs.namenode.checkpoint.txns - This property defines the number of uncheckpointed transactions on the NameNode which will force an urgent checkpoint, even ifthe checkpoint period has not been reached. Set to 1 million by default.

Following image shows the HDFS architecture with communication among NameNode,Secondary NameNode, DataNode and client application.



DataNode, Hadoop, HDFS, NameNode

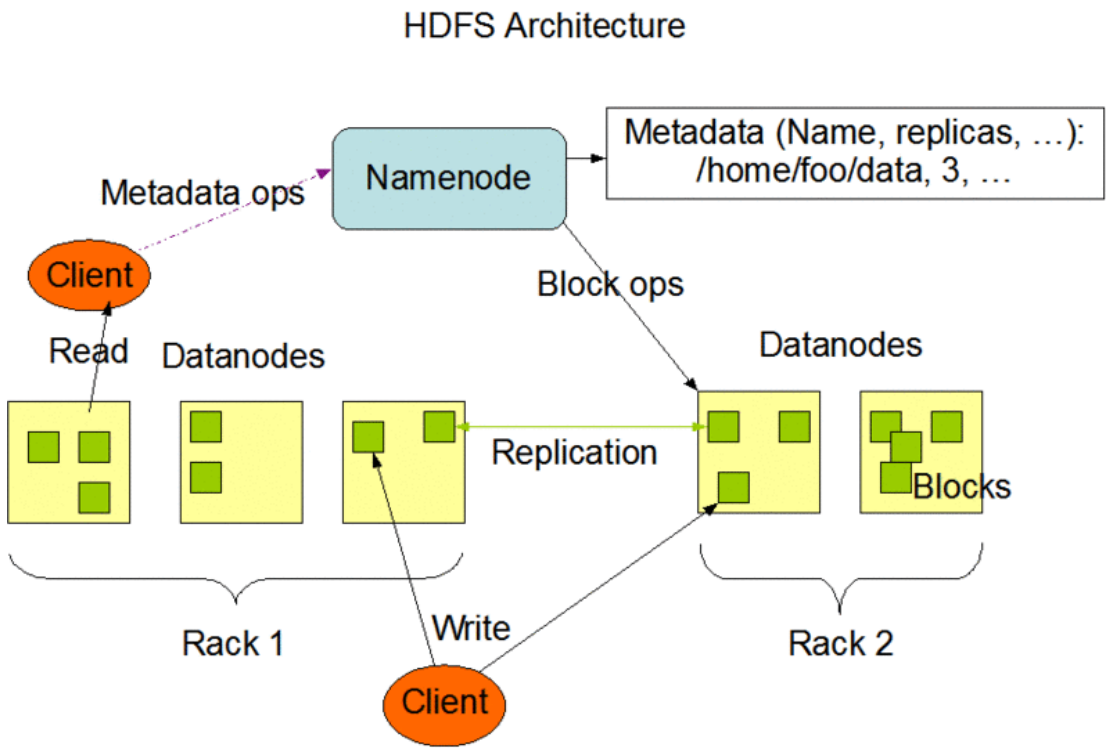


What is NameNode

Metadata refers to a small amount of data, and it requires a minimum amount of memory to store. Namenode stores this metadata of all the files in HDFS. Metadata includes file permission, names, and location of each block. A block is a minimum amount of data that canbe read or write. Moreover, NameNode maps these blocks to dataNodes. Furthermore, nameNode manages all other dataNodes. **Master node** is an alternative name for nameNode.

What is DataNode

The nodes other than the nameNode are called dataNodes. **Slave node** is another name for dataNode. The data nodes store and retrieve blocks as instructed by the nameNode



All dataNodes continuously communicate with the name node. They also inform the nameNode about the blocks they are storing. Furthermore, the dataNodes also perform block creation, deletion, and replication as instructed by the nameNode.

Relationship Between NameNode and DataNode

- Namenode and Datanode operate according to master-slave architecture in Hadoop Distributed File System (HDFS).

Difference Between NameNode and DataNode

Definition

NameNode is the controller and manager of HDFS whereas DataNode is a node other than the NameNode in HDFS that is controlled by the NameNode. Thus, this is the main difference between NameNode and DataNode in Hadoop.

Synonyms

Moreover, Master node is another name for NameNode while Slave node is another name for DataNode.

Main Functionality

While nameNode handles the metadata of all the files in HDFS and controls the dataNodes, Datanode store and retrieve blocks according to the master node's instructions. Hence, this is another difference between NameNode and DataNode in Hadoop.

What is JobTracker and TaskTracker in hadoop?

The main work of JobTracker and TaskTracker in hadoop is given below.

JobTracker is a master which creates and runs the job. JobTracker which can run on the NameNode allocates the job to tasktrackers. It is tracking resource availability and task lifecycle management, tracking its progress, fault tolerance etc.

JobTracker is a daemon which runs on Apache Hadoop's MapReduce engine. JobTracker is an essential service which farms out all MapReduce tasks to the different nodes in the cluster,ideally to those nodes which already contain the data, or at the very least are located in the same rack as nodes containing the data.

JobTracker is the service within Hadoop that is responsible for taking client requests. It assigns them to TaskTrackers on DataNodes where the data required is locally present. If thatis not possible, JobTracker tries to assign the tasks to TaskTrackers within the same rack where the data is locally present. If for some reason this also fails, JobTracker assigns the task to a TaskTracker where a replica of the data exists. In Hadoop, data blocks are replicated across DataNodes to ensure redundancy, so that if one node in the cluster fails, the job does not fail as well.

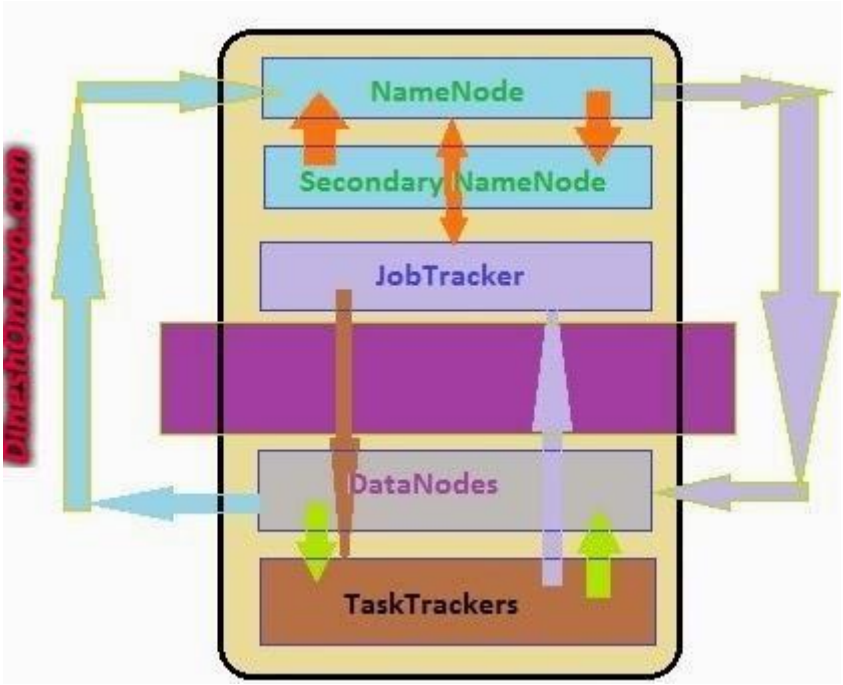
The JobTracker is the service within Hadoop that farms out MapReduce tasks to specificnodes in the cluster, ideally the nodes that have the data, or at least are in the same rack.

- Client applications submit jobs to the Job tracker.
- The JobTracker talks to the NameNode to determine the location of the data
- The JobTracker locates TaskTracker nodes with available slots at or near the data
- The JobTracker submits the work to the chosen TaskTracker nodes.
- The TaskTracker nodes are monitored. If they do not submit heartbeat signals oftenenough, they are deemed to have failed and the work is scheduled on a
    - different TaskTracker.
- A TaskTracker will notify the JobTracker when a task fails. The JobTrackerdecides what to do then it may resubmit the job elsewhere, it may mark thatspecific record as something to avoid, and it may may even blacklist
    - the TaskTracker as unreliable.
- When the work is completed, the JobTracker updates its status.
- Client applications can poll the JobTracker for information.

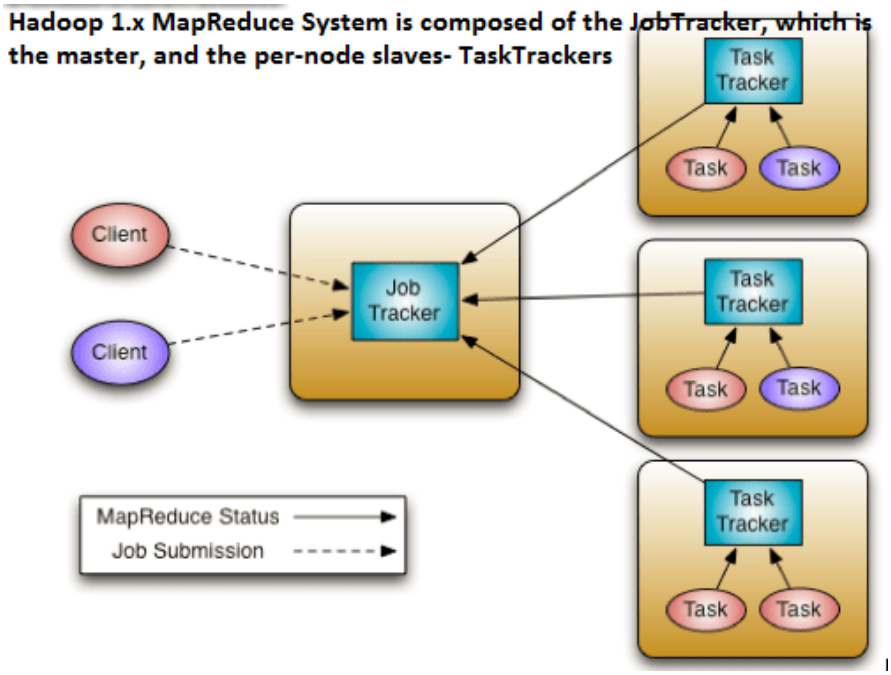
TaskTracker run the tasks and report the status of task to JobTracker. TaskTracker run on DataNodes. It has function of following the orders of the job tracker and updating the job tracker with its progress status periodically.

Daemon Services of Hadoop

- Namenodes
- Secondary Namenodes
- Jobtracker
- Datanodes
- Tasktracker



Above three services 1, 2, 3 can talk to each other and other two services 4,5 can also talk to each other. Namenode and datanodes are also talking to each other as well as Jobtracker and Tasktracker are also.

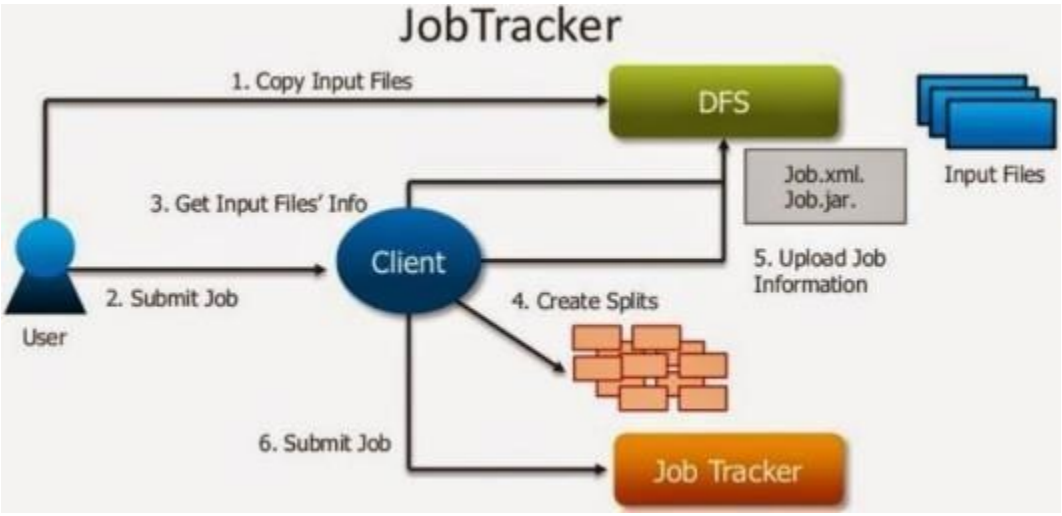


Above the file systems comes the MapReduce engine, which consists of one JobTracker, to which client applications submit MapReduce jobs. The JobTracker pushes work out to available TaskTracker nodes in the cluster, striving to keep the work as close to the data as possible. With a rack-aware file system, the JobTracker knows which node contains the data, and which other machines are nearby.

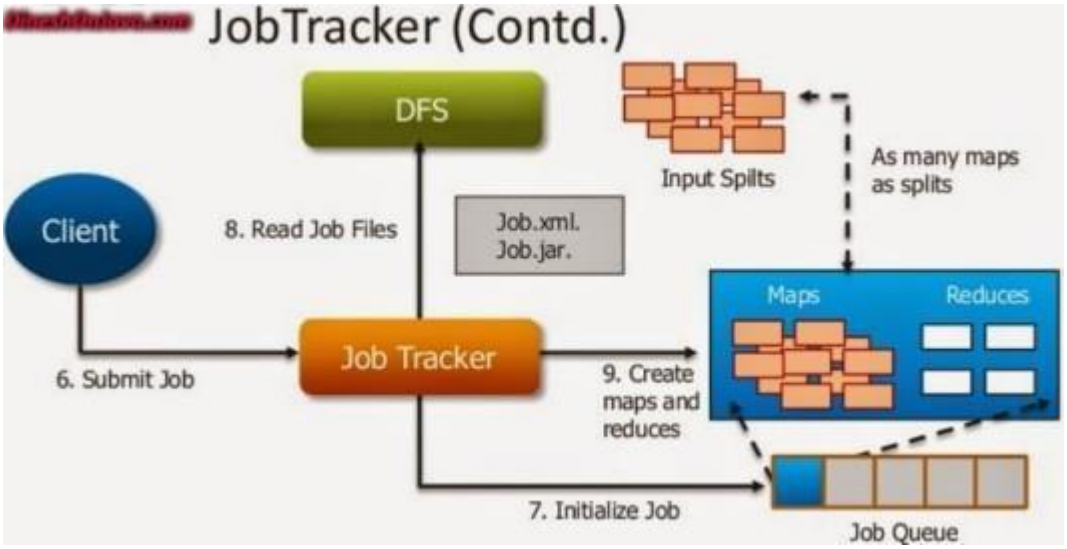
If the work cannot be hosted on the actual node where the data resides, priority is given to nodes in the same rack. This reduces network traffic on the main backbone network. If a TaskTracker fails or times out, that part of the job is rescheduled. The TaskTracker on each node spawns off a separate Java Virtual Machine process to prevent the TaskTracker itself from failing if the running job crashes the JVM. A heartbeat is sent from the TaskTracker to the JobTracker every few minutes to check its status. The Job Tracker and TaskTracker statusand information is exposed by Jetty and can be viewed from a web browser.

If the JobTracker failed on Hadoop 0.20 or earlier, all ongoing work was lost. Hadoopversion 0.21 added some checkpointing to this process; the JobTracker records what it is upto in the file system. When a JobTracker starts up, it looks for any such data, so that it can restart work from where it left off.

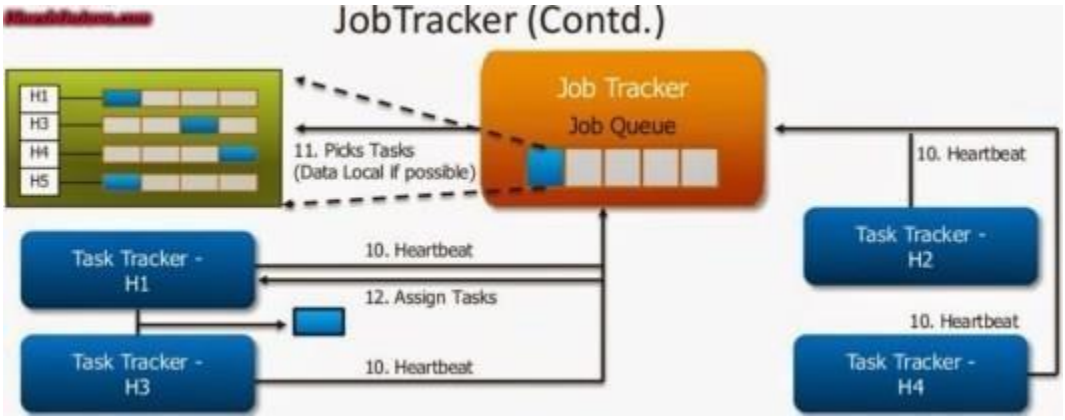
JobTracker and TaskTrackers Work Flow



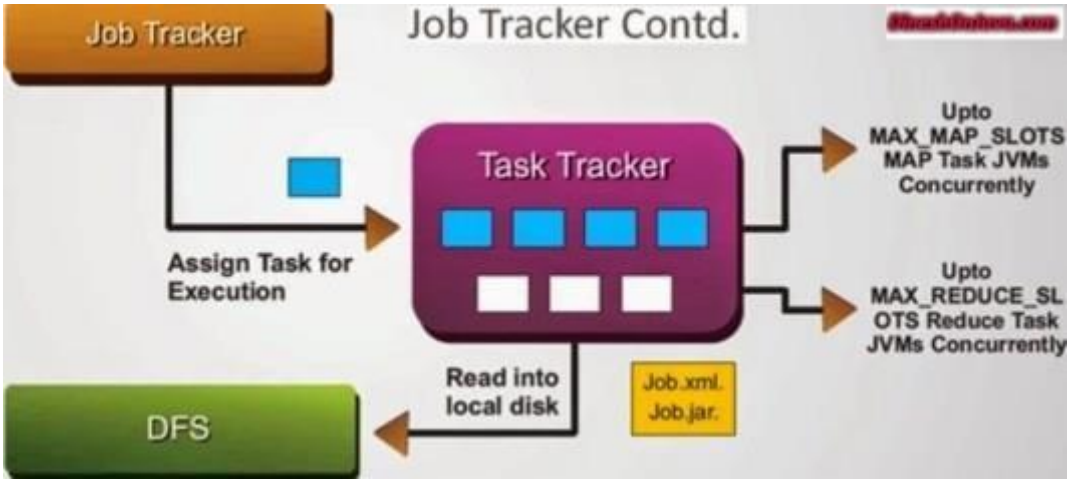
- User copy all input files to distributed file system using namenode meta data.
- Submit jobs to client which applied to input files fetched stored in datanodes.
- Client gets information about input files from namenodes to be process.
- Client creates splits of all files for the jobs
- After splitting files client stored meta data about this job to DFS.
- Now client submit this job-to-job tracker.



- Now jobtracker come into picture and initialize job with job queue.
- Jobtracker read job files from DFS submitted by client.
- Now jobtracker create maps and reduces for jobs and input splits applied to mappers. Same number of mappers are there as many input splits are there. Every map work on individual split and creates output.



1. Now tasktrackers come into picture and jobs submitted to every tasktrackers by jobtracker and **receiving heartbeat from every TaskTracker for confirming tasktracker working properly or not**. This heartbeat frequently sent to JobTracker in 3 second by every TaskTrackers. If suppose any task tracker is not sending heartbeat to jobtracker in 3 second then JobTracker wait for 30 second more after that jobtracker consider those tasktracker as a dead state and upate metadata about those task trackers.
2. Picks tasks from splits.
3. Assign to TaskTracker.



Finally all tasktrackers create outputs and n**umber of reduces generate as number of outputs created by task trackers**. After all reducer give us final output.

Features Of 'Hadoop'

- **Suitable for Big Data Analysis**

As Big Data tends to be distributed and unstructured in nature, HADOOP clusters are best suited for analysis of Big Data. Since it is processing logic (not the actual data) that flows to the computing nodes, less network bandwidth is consumed. This concept is called as **data locality concept** which helps increase the efficiency of Hadoop based applications.

Scalability

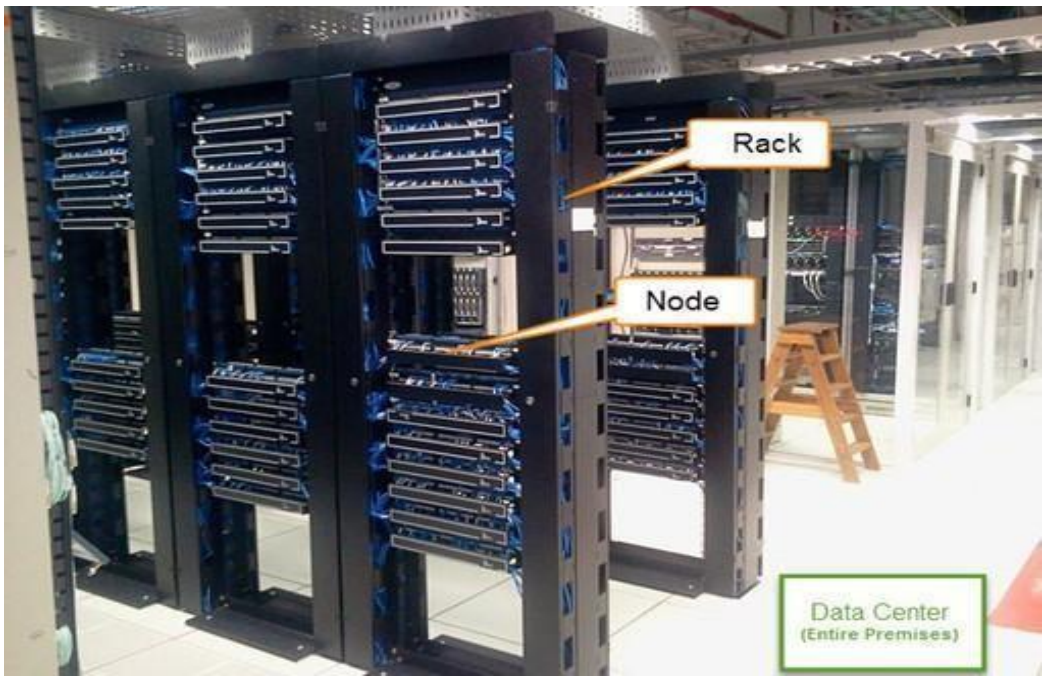
HADOOP clusters can easily be scaled to any extent by adding additional cluster nodes and thus allows for the growth of Big Data. Also, scaling does not require modifications to application logic.

- **Fault Tolerance**

HADOOP ecosystem has a provision to replicate the input data on to other cluster nodes.That way, in the event of a cluster node failure, data processing can still proceed by using data stored on another cluster node.

Network Topology in Hadoop

Topology (Arrangment) of the network, affects the performance of the Hadoop cluster when the size of the Hadoop cluster grows. In addition to the performance, one also needs to care about the high availability and handling of failures. In order to achieve this Hadoop, cluster formation makes use of network topology.



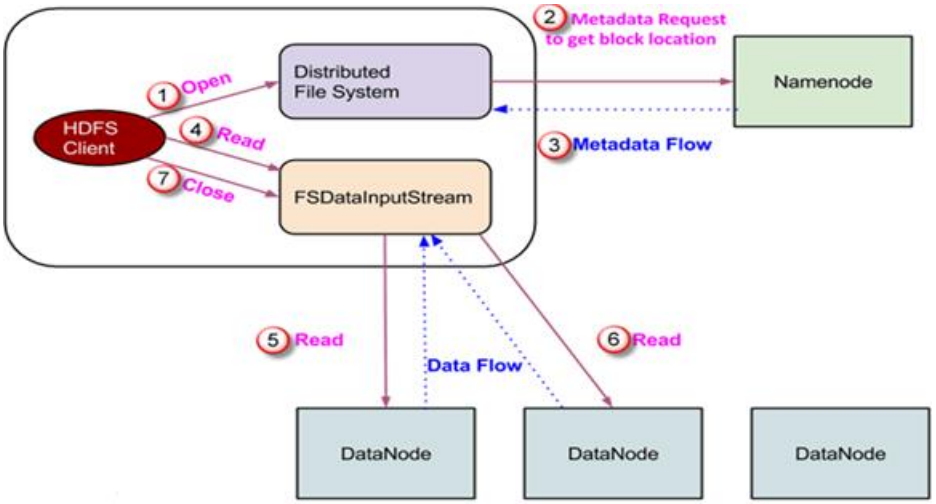
Typically, network bandwidth is an important factor to consider while forming any network. However, as measuring bandwidth could be difficult, in Hadoop, a network is represented as a tree and distance between nodes of this tree (number of hops) is considered as an important factor in the formation of Hadoop cluster. Here, the distance between two nodes is equal to sum of their distance to their closest common ancestor.

Hadoop cluster consists of a data center, the rack and the node which actually executes jobs. Here, data center consists of racks and rack consists of nodes. Network bandwidth available to processes varies depending upon the location of the processes. That is, the bandwidth available becomes lesser as we go away from-

- Processes on the same node
- Different nodes on the same rack
- Nodes on different racks of the same data center
- Nodes in different data centers

4. READ OPERATION IN HDFS

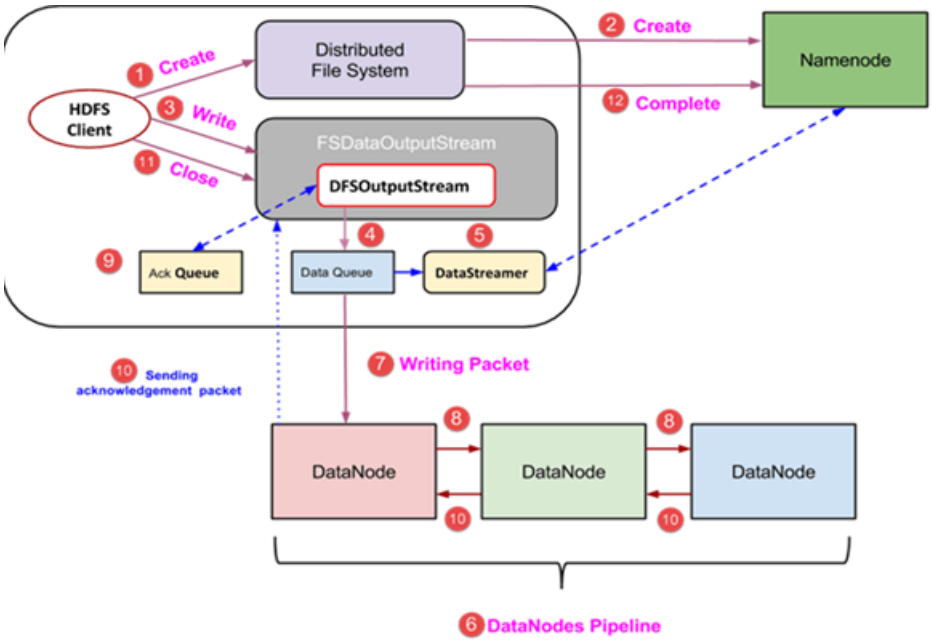
Data read request is served by HDFS, NameNode, and DataNode. Let's call the reader as a 'client'. Below diagram depicts file read operation in Hadoop.



1. A client initiates read request by calling **'open()'** method of FileSystem object; it is an object of type **DistributedFileSystem**.
2. This object connects to namenode using RPC and gets metadata information such as the locations of the blocks of the file. Please note that these addresses are of first few blocks of a file.
3. In response to this metadata request, addresses of the DataNodes having a copy of that block is returned back.
4. Once addresses of DataNodes are received, an object of type **FSDataInputStream** is returned to the client. **FSDataInputStream** contains **DFSInputStream** which takes care of interactions with DataNode and NameNode. In step 4 shown in the above diagram, a client invokes **'read()'** method which causes **DFSInputStream** to establish a connection with the first DataNode with the first block of a file.
5. Data is read in the form of streams wherein client invokes **'read()'** method repeatedly. This process of **read()** operation continues till it reaches the end of block.
6. Once the end of a block is reached, DFSInputStream closes the connection and moves on to locate the next DataNode for the next block
7. Once a client has done with the reading, it calls **a close()** method

5. WRITE OPERATION IN HDFS

In this section, we will understand how data is written into HDFS through files.



1. A client initiates write operation by calling 'create()' method of DistributedFileSystem object which creates a new file - Step no. 1 in the above diagram.
2. DistributedFileSystem object connects to the NameNode using RPC call and initiates new file creation. However, this file creates operation does not associate any blocks with the file. It is the responsibility of NameNode to verify that the file (which is being created) does not exist already and a client has correct permissions to create a new file. If a file already exists or client does not have sufficient permission to create a new file, then **IOException** is thrown to the client. Otherwise, the operation succeeds and a new record for the file is created by the NameNode.
3. Once a new record in NameNode is created, an object of type FSDataOutputStream is returned to the client. A client uses it to write data into the HDFS. Data write method is invoked (step 3 in the diagram).
4. FSDataOutputStream contains DFSOutputStream object which looks after communication with DataNodes and NameNode. While the client continues writing data, **DFSOutputStream** continues creating packets with this data. These packets are enqueued into a queue which is called as **DataQueue**.
5. There is one more component called **DataStreamer** which consumes this **DataQueue**. DataStreamer also asks NameNode for allocation of new blocks thereby

picking desirable DataNodes to be used for replication.

6. Now, the process of replication starts by creating a pipeline using DataNodes. In our case, we have chosen a replication level of 3 and hence there are 3 DataNodes in the pipeline.
7. The DataStreamer pours packets into the first DataNode in the pipeline.
8. Every DataNode in a pipeline stores packet received by it and forwards the same to the second DataNode in a pipeline.
9. Another queue, 'Ack Queue' is maintained by DFSOutputStream to store packets which are waiting for acknowledgment from DataNodes.
10. Once acknowledgment for a packet in the queue is received from all DataNodes in the pipeline, it is removed from the 'Ack Queue'. In the event of any DataNode failure, packets from this queue are used to reinitiate the operation.
11. After a client is done with the writing data, it calls a close() method (Step 9 in the diagram) Call to close(), results into flushing remaining data packets to the pipeline followed by waiting for acknowledgment.
12. Once a final acknowledgment is received, NameNode is contacted to tell it that the file write operation is complete.

Java Program to Write to HDFS

```
FileSystem fileSystem = FileSystem.get(conf);
// Check if the file already exists
Path path = new Path("/path/to/file.ext");
if (fileSystem.exists(path)) {
System.out.println("File " + dest + " already exists");
return;
}
// Create a new file and write data to it.
FSDataOutputStream out = fileSystem.create(path);
InputStream in = new BufferedInputStream(new FileInputStream(
new File(source)));
byte[] b = new byte[1024];
int numBytes = 0;
while ((numBytes = in.read(b)) > 0) {
out.write(b, 0, numBytes);
}
// Close all the file descripters
in.close();
```

How to Read a file from HDFS

```
1. FileSystem fileSystem = FileSystem.get(conf);
2. Path path = new Path("/path/to/file.ext");
3. if (!fileSystem.exists(path)) {
4. System.out.println("File does not exists");
5. return;6.  }
7. FSDataInputStream in = fileSystem.open(path);
8. int numBytes = 0;
9. while ((numBytes = in.read(b))> 0) {
10. System.out.prinln((char)numBytes));// code to manipulate the data which is read
11. }
12. in.close();
13. out.close();
14. fileSystem.close();
```

1. HDFS Write Architecture

Let us assume a situation where an HDFS client, wants to write a file named “ashok.txt”of size 250 MB. Assume that the system block size is configured for 128 MB (default). So,the client will be dividing the file “ashok.txt” into 2 blocks – one of 128 MB (Block A) andthe other of 122 MB (block B). Now, the following protocol will be followed whenever thedata is written into HDFS

1. At first, the HDFS client will reach out to the NameNode for a Write Request against the two blocks, say, Block A and Block B.
2. The NameNode will then grant the client the write permission and will provide the IP addresses of the DataNodes where the file blocks will be copied eventually.
3. The selection of IP addresses of DataNodes is purely randomized based on availability, replication factor and rack awareness.
4. Let’s say the replication factor is set to default i.e. 3. Therefore, for each block the NameNode will be providing the client a list of (3) IP addresses of DataNodes. The list willbe unique for each block.
5. Suppose, the NameNode provided following lists of IP addresses to the client

- For Block A, list A = {IP of DataNode 1, IP of DataNode 4, IP of DataNode 6}
- For Block B, list B = {IP of DataNode 3, IP of DataNode 7, IP of DataNode 9}

6. Each block will be copied in three different DataNodes to maintain the replication factor consistent throughout the cluster.

7. Now the whole data copy process will happen in three stages.

- Set up of Pipeline
- Data streaming and replication
- Shutdown of Pipeline (Acknowledgement stage)

1. **Set up of Pipeline**

Before writing the blocks, the client confirms whether the DataNodes, present in each of the list of IPs, are ready to receive the data or not. In doing so, the client creates a pipeline for each of the blocks by connecting the individual DataNodes in the respective list for that block. Let us consider Block A. The list of DataNodes provided by the NameNode is For Block A, list A = {IP of DataNode 1, IP of DataNode 4, IP of DataNode 6}. So, for block A, the client will be performing the following steps to create a pipeline

2. The client will choose the first DataNode in the list (DataNode IPs for Block A) which is DataNode 1 and will establish a TCP/IP connection.

3. The client will inform DataNode 1 to be ready to receive the block. It will also provide the IPs of next two DataNodes (4 and 6) to the DataNode 1 where the block is supposed to be replicated.

4. The DataNode 1 will connect to DataNode 4. The DataNode 1 will inform DataNode 4 to be ready to receive the block and will give it the IP of DataNode 6. Then, DataNode 4 willtell DataNode 6 to be ready for receiving the data.

5. Next, the acknowledgement of readiness will follow the reverse sequence, i.e. From the DataNode 6 to 4 and then to 1.

6. At last DataNode 1 will inform the client that all the DataNodes are ready and a pipeline will be formed between the client, DataNode 1, 4 and 6.

Now pipeline set up is complete and the client will finally begin the data copy or streaming process.

2. Data Streaming After the pipeline has been created, now the client will push the data into the pipeline. Now, don't forget that in HDFS, data is replicated based on replication

factor. So, here BlockA will be stored to three DataNodes as the assumed replication factor is 3. Moving ahead, theclient will copy the block (A) to DataNode 1 only. The replication is always done byDataNodes sequentially. So, the following steps will take place during replication

1. Once the block has been written to DataNode 1 by the client, DataNode 1 will connect to DataNode
2. Then, DataNode 1 will push the block in the pipeline and data will be copied to DataNode
3. Again, DataNode 4 will connect to DataNode 6 and will copy the last replica of the block.

3. Shutdown of Pipeline or Acknowledgement stage

Once the block has been copied into all the 3 DataNodes, a series of acknowledgements will take place to ensure the client and NameNode that the data has been written successfully. Then, the client will finally close the pipeline to end the TCP session.

The acknowledgement happens in the reverse sequence i.e. from DataNode 6 to 4 and then to 1. Finally, the DataNode 1 will push three acknowledgements (including its own) into the pipeline and send it to the client. The client will inform NameNode that data has been written successfully. The NameNode will update its metadata and the client will shut down the pipeline.

Similarly, Block B will also be copied into the DataNodes in parallel with Block A. So, the following things are to be noticed here

1. The client will copy Block A and Block B to the first DataNode simultaneously.
2. Therefore, in our case, two pipelines will be formed for each of the block and all the process discussed above will happen in parallel in these two pipelines.
3. The client writes the block into the first DataNode and then the DataNodes will be replicating the block sequentially.

2. HDFS Read ArchitectureHDFS Read architecture is comparatively easy to understand. Let's take the above example again where the HDFS client wants to read the file "ashok.txt" now. Now, followingsteps will be taking place while reading the file

1. First client will reach out to NameNode asking for the block metadata for the file "ashok.txt".
2. The NameNode will return the list of DataNodes where each block (Block A and B) are stored.
3. After that client, will connect to the DataNodes where the blocks are stored.

4. The client starts reading data parallel from the DataNodes (Block A from DataNode 1 and Block B from DataNode
5. Once the client gets all the required file blocks, it will combine these blocks to form a file.

While serving read request of the client, HDFS selects the replica which is closest to the client. This reduces the read latency and the bandwidth consumption. Therefore, that replica is selected which resides on the same rack as the reader node, if possible.

Hadoop HDFS (In Depth) Data Read and Write Operations

HDFS – Hadoop Distributed File System is the storage layer of Hadoop. It is most reliable storage system on the planet. HDFS works in master-slave fashion, NameNode is the master daemon which runs on the master node, DataNode is the slave daemon which runs on the slave node.

Hadoop HDFS Data Write Operation

To write a file in HDFS, a client needs to interact with master i.e. NameNode (master). Now NameNode provides the address of the DataNodes (slaves) on which client will start writing the data. Client directly writes data on the DataNodes, now DataNode will create datawrite pipeline.

The first datanode will copy the block to another DataNode, which intern copy it to the third DataNode. Once it creates the replicas of blocks, it sends back the acknowledgment.

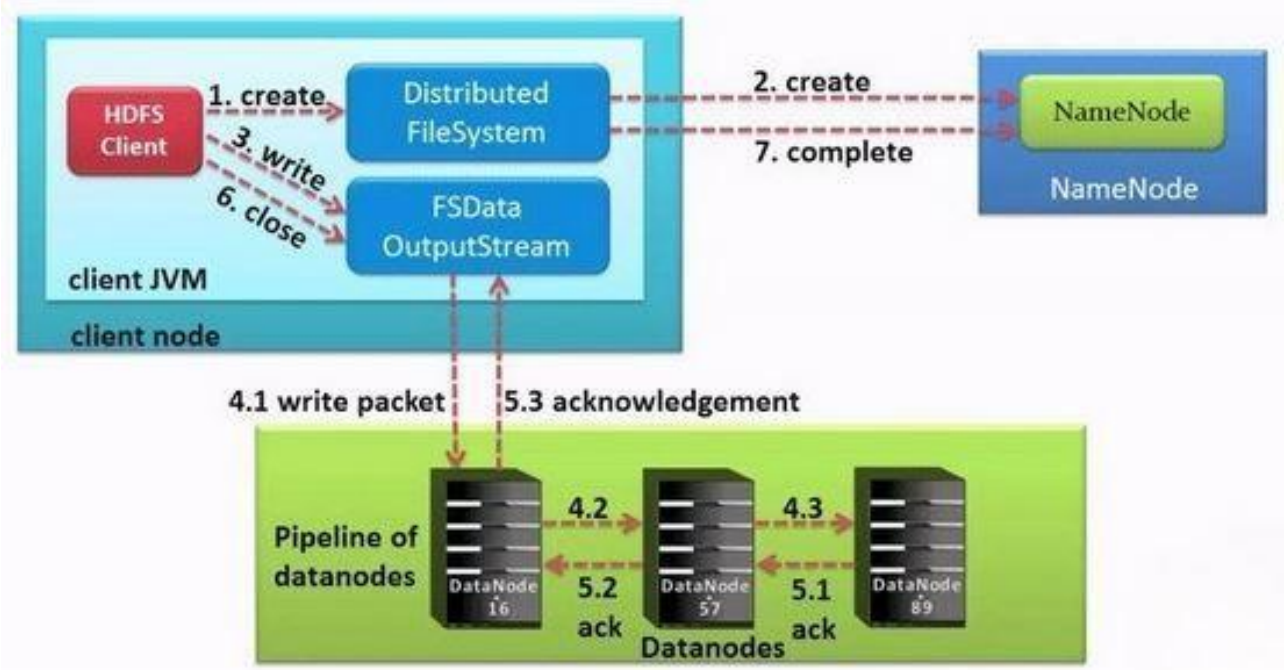
HDFS Data Write Pipeline Workflow

Step 1 The HDFS client sends a create request on DistributedFileSystem APIs. **Step 2** DistributedFileSystem makes an RPC call to the namenode to create a new file in the file system's namespace. The namenode performs various checks to make sure that the file doesn't already exist and that the client has the permissions to create the file. When these checks pass, then only the namenode makes a record of the new file; otherwise, file creation fails and the client is thrown an IOException.
Step 3 The DistributedFileSystem returns a FSDataOutputStream for the client to startwriting data to. As the client writes data, DFSOutputStream splits it into packets, which it writes to an internal queue, called the data queue. The data queue is consumed by the DataStreamer, which

is responsible for asking the namenode to allocate new blocks bypicking a list of suitable DataNodes to store the replicas. **Step 4** The list of DataNodes form a pipeline, and here we'll assume the replication level is three, so there are three nodes in the pipeline. The DataStreamer streams the packets to the

first DataNode in the pipeline, which stores the packet and forwards it to the second DataNode in the pipeline. Similarly, the second DataNode stores the packet and forwards it to the third (and last) DataNode in the pipeline. **Step 5** DFSOutputStream also maintains an internal queue of packets that are waiting to be acknowledged by DataNodes, called the Ack Queue. A packet is removed from the ack queue only when it has been acknowledged by the DataNodes in the pipeline. DataNodes sends the acknowledgment once required replicas are created (3 by default). Similarly, all the blocks are stored and replicated on the different DataNodes, the data blocks are copied in parallel. **Step 6** When the client has finished writing data, it calls close () on the stream.**Step 7** This action flushes all the remaining packets to the DataNode pipeline and waits for acknowledgments before contacting the NameNode to signal that the file is complete. The NameNode already knows which blocks the file is made up of, so it only has to wait for blocks to be minimally replicated before returning successfully.

We can summarize the HDFS data write operation from the following diagram



Hadoop HDFS Data Read Operation

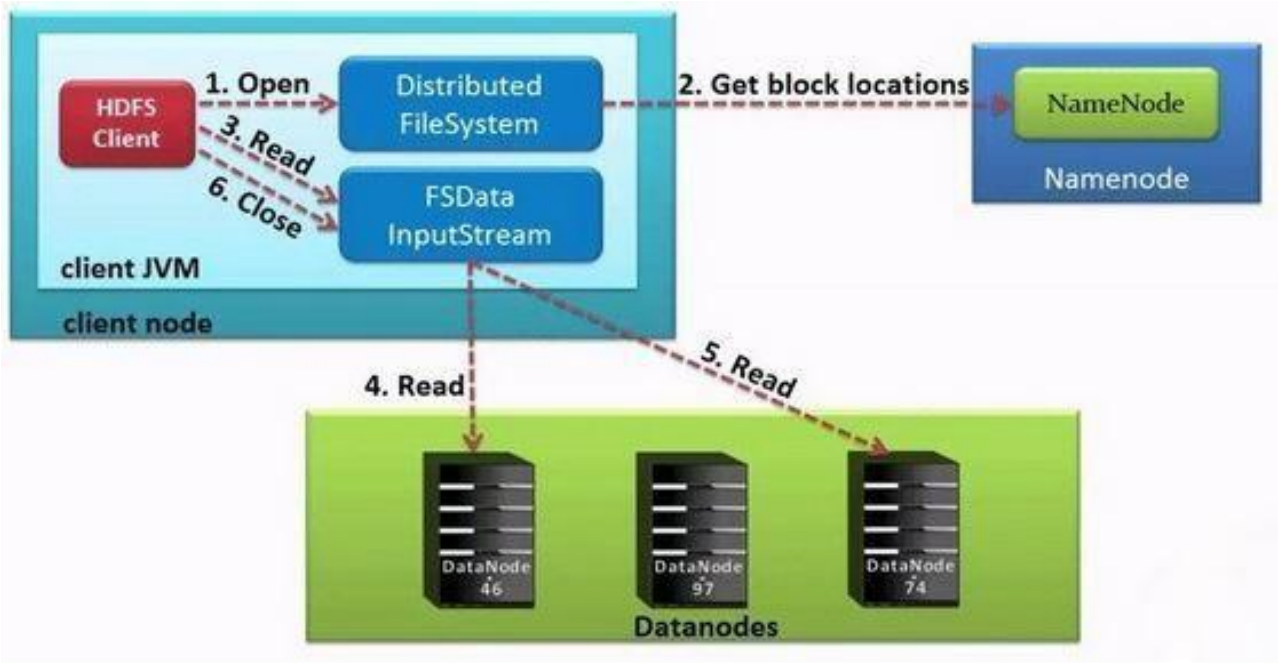
To read a file from HDFS, a client needs to interact with NameNode (master) as

NameNode is the centerpiece of Hadoop cluster (it stores all the metadata i.e. data about the data). Now NameNode checks for required privileges, if the client has sufficient privileges then NameNode provides the address of the slaves where a file is stored. Now client will interact directly with the respective DataNodes to read the data blocks.

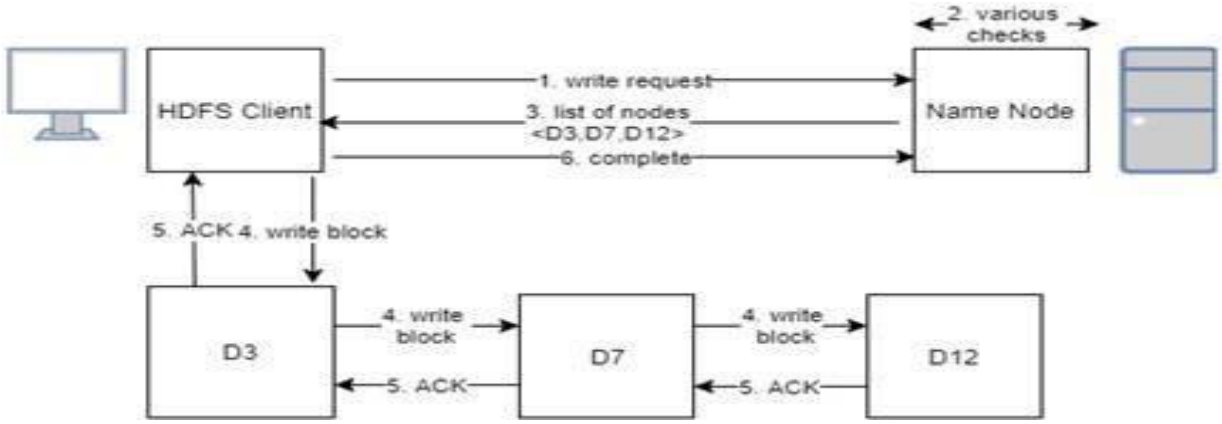
HDFS File Read Workflow

Step 1 Client opens the file it wishes to read by calling open() on the FileSystem object, which for HDFS is an instance of DistributedFileSystem. **Step 2** DistributedFileSystem calls the NameNode using RPC to determine the locations of the blocks for the first few blocks in the file. For each block, the NameNode returns the addresses of the DataNodes that have a copy of that block and DataNode are sorted according to their proximity to the client.
Step 3 DistributedFileSystem returns a FSDataInputStream to the client for it to read data from. FSDataInputStream, thus, wraps the DFSInputStream which manages the DataNode and NameNode I/O. Client calls read () on the stream. DFSInputStream which has stored the DataNode addresses then connects to the closest DataNode for the first block in the file.
Step 4 Data is streamed from the DataNode back to the client, as a result client can call read () repeatedly on the stream. When the block ends, DFSInputStream will close the connection to the DataNode and then finds the best DataNode for the next block. **Step 5** If the DFSInputStream encounters an error while communicating with a DataNode, it will try the next closest one for that block. It will also remember DataNodes that have failed so that it doesn't needlessly retry them for later blocks. The DFSInputStream also verifies checksums for the data transferred to it from the DataNode. If it finds a corrupt block, it reports this to the NameNode before the DFSInputStream attempts to read a replica of the block from another DataNode.
Step 6 When the client has finished reading the data, it calls close () on the stream.

We can summarize the HDFS data read operation from the following diagram



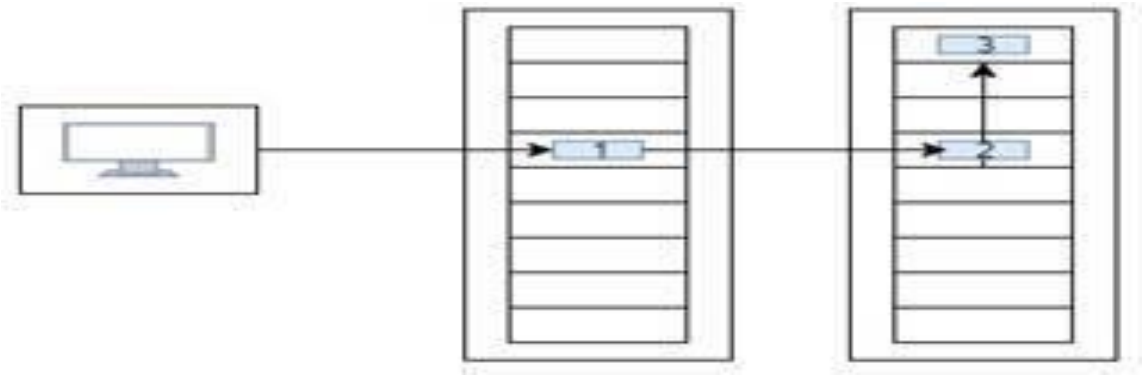
How Read and Write operations are performed in HDFS **HDFS Write**



- By default the replication factor(multiple copiesof blocks) for a block is 3.
- As Name Node receives write request from HDFS client (JVM), Name Node checks whether file is available or not as well as whether client is authorized or not (performs various checks) and returns multiple nodes.
- Step 3,4 and 5 will get repeated until the whole file gets written on HDFS.
- In case of Data Node failure-
- The data is written on the remaining two nodes.
- Name node notices under replication and arranges for replication.
- Same is the case with multiple node failure.

HDFS Write- Selection of the Data Nodes

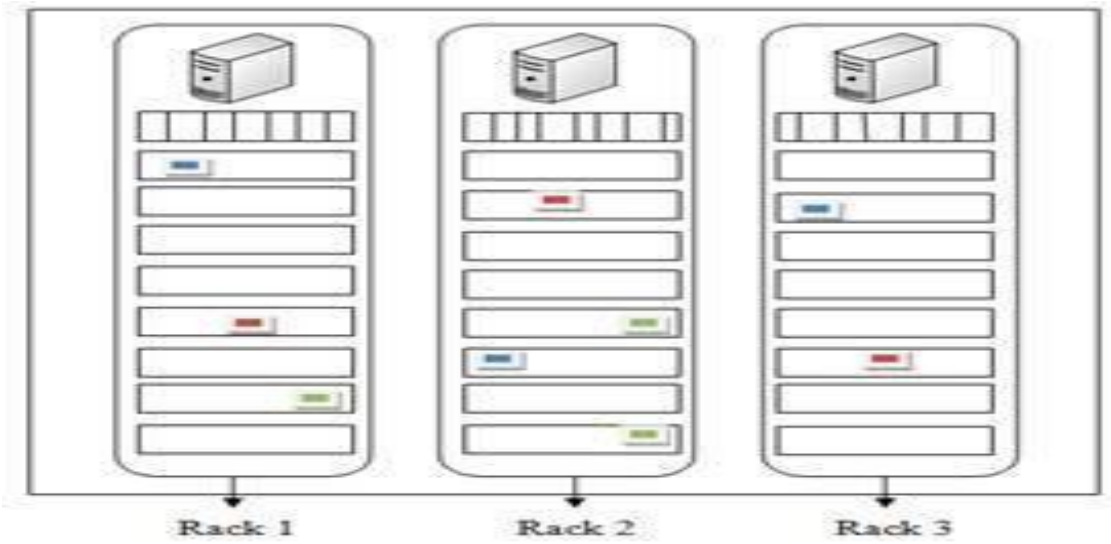
How Data Nodes are selected by Name Nodes



- Any node within the cluster is chosen as the first node but it should not be too busy or overloaded.
- Second node is chosen as the first node is chosen.
- Third node is chosen to be on the same rack as the second one.This forms thepipeline.

Simulation on block distribution

- File is broken into blocks (64 mb) and then replicated and distributed across the file system.

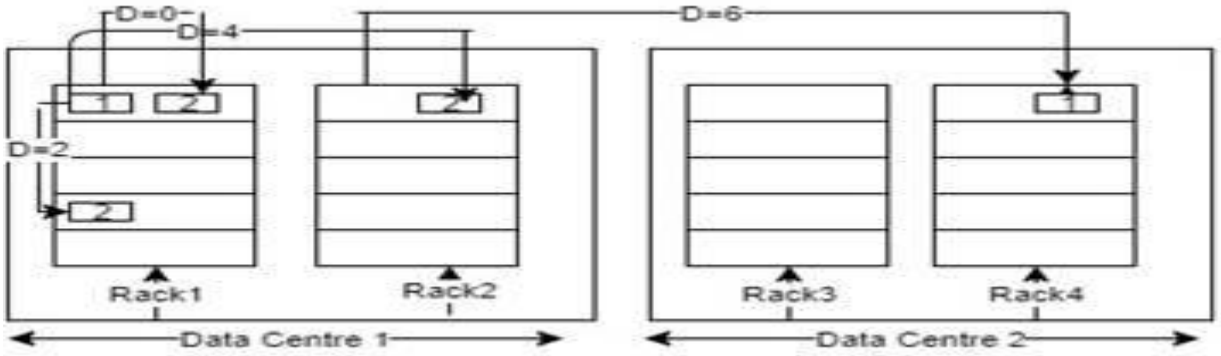


- If one of the node/rack fails then also the replication of ( ) that block is available on other racks .
- Failure of multiple racks are more serious but less probable.
- Also, the whole procedure of selection and replication happens behind a curtain, no developer or client is able to see all this or has to worry about what happens in the background.

Node Distance

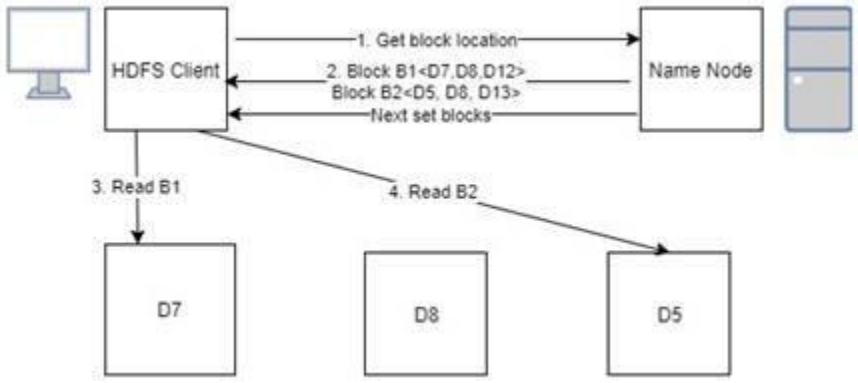
How is distance calculated in HDFS? Idea of distance is based on bandwidth.

The only possible cases for calculating distance are-



- D=0, blocks on same node, same rack.
- D=2, blocks on different node,same rack.
- D=4, blocks on a node having different rack.
- D=6, blocks on a node having different data centre.

HDFS Read



If D7 fails, next D8 is pickedFailure cases,

- Data block is corrupted,
    - Next node in the list is picked up.
- Data Node fails,
    - Next node in the list is picked up.
    - That node is not tried for the later blocks.

Setting Up Development Environment

Hadoop is supported by GNU/Linux platform and its flavors. Therefore, we have to install a Linux operating system for setting up Hadoop environment. In case you have an OS other than Linux, you can install a Virtualbox software in it and have Linux inside the Virtualbox.

6. HADOOP INSTALLATION PROCESS

Pre-installation Setup

Before installing Hadoop into the Linux environment, we need to set up Linux using **ssh** (Secure Shell). Follow the steps given below for setting up the Linux environment.

Creating a User

At the beginning, it is recommended to create a separate user for Hadoop to isolate Hadoop file system from Unix file system. Follow the steps given below to create a user −

- Open the root using the command “su”.
- Create a user from the root account using the command “useradd username”.
- Now you can open an existing user account using the command “su username”.

Open the Linux terminal and type the following commands to create a user.

$ su

password

useradd hadoop# passwd hadoopNew passwd

Retype new passwd

SSH Setup and Key Generation

SSH setup is required to do different operations on a cluster such as starting, stopping, distributed daemon shell operations. To authenticate different users of Hadoop, it is required to provide public/private key pair for a Hadoop user and share it with different users.

The following commands are used for generating a key value pair using SSH. Copy the public keys form id_rsa.pub to authorized_keys, and provide the owner with read and write permissions to authorized_keys file respectively.

```
$ ssh-keygen -t rsa
$ cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
$ chmod 0600 ~/.ssh/authorized_keys
```

Installing Java

Java is the main prerequisite for Hadoop. First of all, you should verify the existence of java in your system using the command “java -version”. The syntax of java version command is given below.

```
$ java -version
```

If everything is in order, it will give you the following output.java version "1.7.0_71" Java(TM) SE Runtime Environment (build 1.7.0_71-b13) Java HotSpot(TM) Client VM (build 25.0-b02, mixed mode)

If java is not installed in your system, then follow the steps given below for installing java.

Step 1

Download java (JDK <latest version> - X64.tar.gz) by visiting the following link www.oracle.com

Then **jdk-7u71-linux-x64.tar.gz** will be downloaded into your system.

Step 2

Generally you will find the downloaded java file in Downloads folder. Verify it and extract the **jdk-7u71-linux-x64.gz** file using the following commands.

```
$ cd Downloads/
$ ls
jdk-7u71-linux-x64.gz

$ tar zxf jdk-7u71-linux-x64.gz
$ ls
jdk1.7.0_71  jdk-7u71-linux-x64.gz
```

Step 3

To make java available to all the users, you have to move it to the location "/usr/local/". Open root, and type the following commands.

```
$ su password
# mv jdk1.7.0_71 /usr/local/# exit
```

Step 4

For setting up **PATH** and **JAVA_HOME** variables, add the following commands to **~/.bashrc** file.

```
export JAVA_HOME=/usr/local/jdk1.7.0_71export PATH=$PATH$JAVA_HOME/bin
```

Now apply all the changes into the current running system.

```
$ source ~/.bashrc
```

Step 5

Use the following commands to configure java alternatives −

```
# alternatives --install /usr/bin/java java usr/local/java/bin/java 2
```

alternatives --install /usr/bin/javac javac usr/local/java/bin/javac 2# alternatives --install /usr/bin/jar jar usr/local/java/bin/jar 2

alternatives --set java usr/local/java/bin/java
alternatives --set javac usr/local/java/bin/javac# alternatives --set jar usr/local/java/bin/jar

Now verify the java -version command from the terminal as explained above.

Downloading Hadoop

Download and extract Hadoop 2.4.1 from Apache software foundation using the following commands.

$ su password
cd /usr/local
wget http//apache.claz.org/hadoop/common/hadoop-2.4.1/hadoop-2.4.1.tar.gz
tar xzf hadoop-2.4.1.tar.gz
mv hadoop-2.4.1/* to hadoop/# exit

Hadoop Operation Modes

Once you have downloaded Hadoop, you can operate your Hadoop cluster in one of the three supported modes −

- **Local/Standalone Mode** − After downloading Hadoop in your system, by default, itis configured in a standalone mode and can be run as a single java process.
- **Pseudo Distributed Mode** − It is a distributed simulation on single machine. Each Hadoop daemon such as hdfs, yarn, MapReduce etc., will run as a separate java process. This mode is useful for development.
- **Fully Distributed Mode** − This mode is fully distributed with minimum two or more machines as a cluster. We will come across this mode in detail in the coming chapters.

Installing Hadoop in Standalone Mode

Here we will discuss the installation of **Hadoop 2.4.1** in standalone mode.

There are no daemons running and everything runs in a single JVM. Standalone mode is suitable for running MapReduce programs during development, since it is easy to test and debug them.

Setting Up Hadoop

You can set Hadoop environment variables by appending the following commands to **~/.bashrc** file.

export HADOOP_HOME=/usr/local/hadoop

Before proceeding further, you need to make sure that Hadoop is working fine. Just issue the following command −

$ hadoop version
If everything is fine with your setup, then you should see the following result −Hadoop 2.4.1
Subversion https//svn.apache.org/repos/asf/hadoop/common -r 1529768Compiled by hortonmu on 2013-10-07T0628Z
Compiled with protoc 2.5.0
From source with checksum 79e53ce7994d1628b240f09af91e1af4

It means your Hadoop's standalone mode setup is working fine. By default, Hadoop is configured to run in a non-distributed mode on a single machine.

Example

Let's check a simple example of Hadoop. Hadoop installation delivers the following example MapReduce jar file, which provides basic functionality of MapReduce and can be used for calculating, like Pi value, word counts in a given list of files, etc.

$HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduce-examples-2.2.0.jar

Let's have an input directory where we will push a few files and our requirement is to count the total number of words in those files. To calculate the total number of words, we do not need to write our MapReduce, provided the .jar file contains the implementation for word count. You

can try other examples using the same .jar file; just issue the following commandsto check supported MapReduce functional programs by hadoop-mapreduce-examples- 2.2.0.jar file.

$ hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduceexamples-2.2.0.jar

Step 1

Create temporary content files in the input directory. You can create this input directory anywhere you would like to work.

$ mkdir input
$ cp $HADOOP_HOME/*.txt input
$ ls -l input
It will give the following files in your input directory −total 24
-rw-r--r-- 1 root root 15164 Feb 21 1014 LICENSE.txt
-rw-r--r-- 1 root root 101 Feb 21 1014 NOTICE.txt
-rw-r--r-- 1 root root 1366 Feb 21 1014 README.txt

These files have been copied from the Hadoop installation home directory. For your experiment, you can have different and large sets of files.

Step 2

Let's start the Hadoop process to count the total number of words in all the files available in the input directory, as follows −

$ hadoop jar $HADOOP_HOME/share/hadoop/mapreduce/hadoop-mapreduceexamples-2.2.0.jar wordcount input output

Step 3

Step-2 will do the required processing and save the output in output/part-r00000 file, which you can check by using −

$cat output/*

It will list down all the words along with their total counts available in all the files availablein the input directory.

"AS 4
"Contribution" 1
"Contributor" 1
"Derivative 1
"Legal 1
"License" 1
"License"); 1
"Licensor" 1
"NOTICE” 1
"Not 1
"Object" 1
"Source” 1
"Work” 1
"You" 1
"Your") 1
"[]" 1
"control" 1
"printed 1
"submitted" 1
(50%) 1
(BIS), 1
(C) 1 (Don't) 1 (ECCN) 1
(INCLUDING 2
(INCLUDING, 2
.....

Installing Hadoop in Pseudo Distributed Mode

Follow the steps given below to install Hadoop 2.4.1 in pseudo distributed mode.

Step 1 − Setting Up Hadoop

You can set Hadoop environment variables by appending the following commands to **~/.bashrc** file.

export HADOOP_HOME=/usr/local/hadoop
export HADOOP_MAPRED_HOME=$HADOOP_HOME export
HADOOP_COMMON_HOME=$HADOOP_HOME

export HADOOP_HDFS_HOME=$HADOOP_HOME
export YARN_HOME=$HADOOP_HOME
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/nativeexport
PATH=$PATH$HADOOP_HOME/sbin$HADOOP_HOME/bin
export HADOOP_INSTALL=$HADOOP_HOME

Now apply all the changes into the current running system.

$ source ~/.bashrc

Step 2 − Hadoop Configuration

You can find all the Hadoop configuration files in the location "$HADOOP_HOME/etc/hadoop". It is required to make changes in those configuration files according to your Hadoop infrastructure.

$ cd $HADOOP_HOME/etc/hadoop

In order to develop Hadoop programs in java, you have to reset the java environment variables in **hadoop-env.sh** file by replacing **JAVA_HOME** value with the location of javain your system.

export JAVA_HOME=/usr/local/jdk1.7.0_71

The following are the list of files that you have to edit to configure Hadoop.

core-site.xml

The **core-site.xml** file contains information such as the port number used for Hadoop instance, memory allocated for the file system, memory limit for storing the data, and size of Read/Write buffers.

Open the core-site.xml and add the following properties in between <configuration>, </configuration> tags.

```
<configuration>
  <property>
    <name>fs.default.name</name>
    <value>hdfs//localhost9000</value>
  </property>
</configuration>
```

hdfs-site.xml

The **hdfs-site.xml** file contains information such as the value of replication data, namenode path, and datanode paths of your local file systems. It means the place where you want to store the Hadoop infrastructure.

Let us assume the following data. dfs.replication (data replication value) = 1
(In the below given path /hadoop/ is the user name. hadoopinfra/hdfs/namenode is the directory created by hdfs file system.)namenode path =
//home/hadoop/hadoopinfra/hdfs/namenode

(hadoopinfra/hdfs/datanode is the directory created by hdfs file system.)datanode path =
//home/hadoop/hadoopinfra/hdfs/datanode

Open this file and add the following properties in between the <configuration> </configuration> tags in this file.

```
<configuration>
  <property>
    <name>dfs.replication</name>
```

```
    <value>1</value>
  </property>

  <property>
    <name>dfs.name.dir</name>
    <value>file///home/hadoop/hadoopinfra/hdfs/namenode </value>
  </property>

  <property>
    <name>dfs.data.dir</name>
    <value>file///home/hadoop/hadoopinfra/hdfs/datanode </value>
  </property>
</configuration>
```

Note − In the above file, all the property values are user-defined and you can make changes according to your Hadoop infrastructure.

yarn-site.xml

This file is used to configure yarn into Hadoop. Open the yarn-site.xml file and add the following properties in between the <configuration>, </configuration> tags in this file.

```
<configuration>
  <property>
    <name>yarn.nodemanager.aux-services</name>
    <value>mapreduce_shuffle</value>
  </property>
</configuration>
```

mapred-site.xml

This file is used to specify which MapReduce framework we are using. By default, Hadoop contains a template of yarn-site.xml. First of all, it is required to copy the file from **mapred-site.xml.template** to **mapred-site.xml** file using the following command.

```
$ cp mapred-site.xml.template mapred-site.xml
```

Open **mapred-site.xml** file and add the following properties in between the <configuration>, </configuration>tags in this file.

```
<configuration>
  <property>
    <name>mapreduce.framework.name</name>
    <value>yarn</value>
  </property>
</configuration>
```

Verifying Hadoop Installation

The following steps are used to verify the Hadoop installation.

Step 1 − Name Node Setup

Set up the namenode using the command “hdfs namenode -format” as follows.

```
$ cd ~
$ hdfs namenode -format
```

The expected result is as follows.

```
10/24/14 213055 INFO namenode.NameNode STARTUP_MSG
/************************************************************STARTUP_MSG
Starting NameNode
STARTUP_MSG host = localhost/192.168.1.11STARTUP_MSG  args = [-format]
STARTUP_MSG  version = 2.4.1
...
...
10/24/14 213056 INFO common.Storage Storage directory
/home/hadoop/hadoopinfra/hdfs/namenode has been successfully formatted.10/24/14 213056
INFO namenode.NNStorageRetentionManager Going toretain 1 images with txid >= 0
10/24/14 213056 INFO util.ExitUtil Exiting with status 0
```

10/24/14 213056 INFO namenode.NameNode SHUTDOWN_MSG
/**
SHUTDOWN_MSG Shutting down NameNode at localhost/192.168.1.11
**/

Step 2 − Verifying Hadoop dfs

The following command is used to start dfs. Executing this command will start your Hadoop file system.

$ start-dfs.sh
The expected output is as follows −10/24/14 213756
Starting namenodes on [localhost]
localhost starting namenode, logging to /home/hadoop/hadoop2.4.1/logs/hadoop-hadoop-namenode-localhost.out
localhost starting datanode, logging to /home/hadoop/hadoop2.4.1/logs/hadoop-hadoop-datanode-localhost.out
Starting secondary namenodes [0.0.0.0]

Step 3 − Verifying Yarn Script

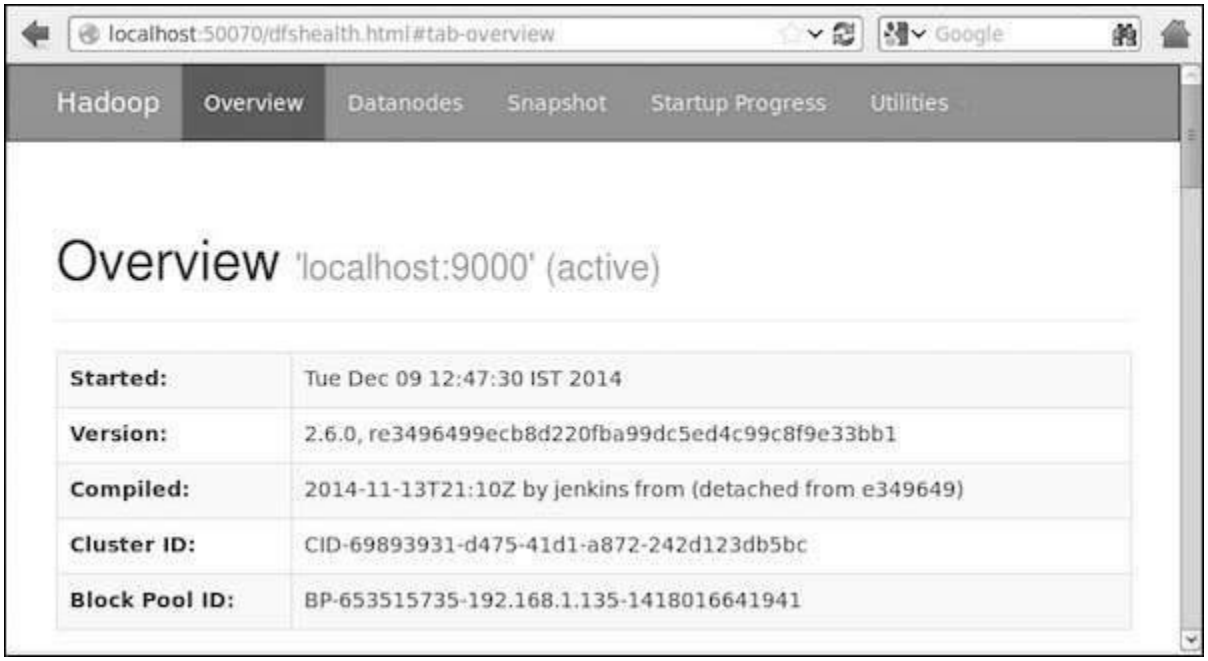
The following command is used to start the yarn script. Executing this command will start your yarn daemons.

$ start-yarn.sh
The expected output as follows −starting yarn daemons
starting resourcemanager, logging to /home/hadoop/hadoop2.4.1/logs/yarn-hadoop-resourcemanager-localhost.out
localhost starting nodemanager, logging to /home/hadoop/hadoop2.4.1/logs/yarn-hadoop-nodemanager-localhost.out

Step 4 − Accessing Hadoop on Browser

The default port number to access Hadoop is 50070. Use the following url to get Hadoop services on browser.

http//localhost50070/



Step 5 − Verify All Applications for Cluster

The default port number to access all applications of cluster is 8088. Use the following url to visit this service.

http//localhost8088/



7. EXPLORING HADOOP COMMANDS

HDFS command is used most of the times when working with Hadoop File System.It includes various shell-like commands that directly interact with the Hadoop Distributed File System (HDFS) as well as other file systems that Hadoop supports.

1) Version Check

To check the version of Hadoop. ubuntu@ubuntu-VirtualBox~$ **hadoop version**Hadoop 2.7.3
Subversion https//git-wip-us.apache.org/repos/asf/hadoop.git -r baa91f7c6bc9cb92be5982de4719c1c8af91ccff

Compiled by root on 2016-08-18T0141ZCompiled with protoc 2.5.0
From source with checksum 2e4ce5f957ea4db193bce3734ff29ff4

This command was run using /home/ubuntu/hadoop-2.7.3/share/hadoop/common/hadoop-common-2.7.3.jar

2) list Command

List all the files/directories for the given hdfs destination path.ubuntu@ubuntu-VirtualBox~ $ **hdfs dfs -ls /**
Found 3 items

Drwxr-xr-x	- ubuntu supergroup	0 2016-11-07 0111 /test
drwxr-xr-x	- ubuntu supergroup	0 2016-11-07 0109 /tmp
drwxr-xr-x	- ubuntu supergroup	0 2016-11-07 0109 /usr

3) df Command

Displays free space at given hdfs destination ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -df hdfs/** Filesystem Size Used Available Use% hdfs//master9000 6206062592 32768 316289024 0%

4) count Command

- Count the number of directories, files and bytes under the paths that match the specified file pattern.

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -count hdfs/** 0 hdfs///

5) fsck Command

HDFS Command to check the health of the Hadoop file system.ubuntu@ubuntu-VirtualBox~$ **hdfs fsck /**
Connecting to namenode via http//master50070/fsck?ugi=ubuntu&path=%2F

FSCK started by ubuntu (authSIMPLE) from /192.168.1.36 for path / at Mon Nov 07012354 GMT+0530 2016

Status HEALTHY Total size 0 B
Total dirs 4

Total files 0

Total symlinks 0

Total blocks (validated)	0
Minimally replicated blocks	0
Over-replicated blocks	0
Under-replicated blocks	0
Mis-replicated blocks	0
Default replication factor	2
Average block replication	0 .
Corrupt blocks	
Missing replicas	0

Number of data-nodes	1
Number of racks	1

FSCK ended at Mon Nov 07 012354 GMT+0530 2016 in 33 millisecondsThe filesystem under path '/' is HEALTHY

6) balancer Command

Run a cluster balancing utility. ubuntu@ubuntu-VirtualBox~$ **hdfs balancer**
16/11/07 012629 INFO balancer.Balancer namenodes = [hdfs//master9000]

16/11/07 012629 INFO balancer.Balancer parameters = Balancer.Parameters[BalancingPolicy.Node, threshold=10.0, max idle iteration = 5, numberof nodes to be excluded = 0, number of nodes to be included = 0]

Time Stamp Iteration# Bytes Already Moved Bytes Left To Move Bytes Being Moved

16/11/07 012638 INFO net.NetworkTopology Adding a new node /default-rack/192.168.1.3650010

16/11/07 012638 INFO balancer.Balancer 0 over-utilized [] 16/11/07 012638 INFO balancer.Balancer 0 underutilized []The cluster is balanced. Exiting...
7 Nov, 2016 12638 AM 0 0 B 0 B -1 B

7 Nov, 2016 12639 AM Balancing took 13.153 seconds

7) mkdir Command

HDFS Command to create the directory in HDFS. ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -mkdir /hadoop**ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -ls /**
Found 5 items

drwxr-xr-x	- ubuntu supergroup	0 2016-11-07 0129 /hadoop
drwxr-xr-x	- ubuntu supergroup	0 2016-11-07 0126 /system
drwxr-xr-x	- ubuntu supergroup	0 2016-11-07 0111 /test
drwxr-xr-x	- ubuntu supergroup	0 2016-11-07 0109 /tmp
drwxr-xr-x	- ubuntu supergroup	0 2016-11-07 0109 /usr

8) put Command

File

Copy file from single src, or multiple srcs from local file system to the destination filesystem.

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -put test /hadoop**ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -ls /hadoop** Found 1 items
-rw-r--r-- 2 ubuntu supergroup 16 2016-11-07 0135 /hadoop/test

Directory

HDFS Command to copy directory from single source, or multiple sources from local file system to the destination file system.

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -put hello /hadoop/**

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -ls /hadoop**

Found 2 items

drwxr-xr-x - ubuntu supergroup 0 2016-11-07 0143 /hadoop/hello

-rw-r--r-- 2 ubuntu supergroup 16 2016-11-07 0135 /hadoop/test

9) du Command

Displays size of files and directories contained in the given directory or the size of a file if its just a file.

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -du /**

59 /hadoop

0   /system

0 /test

0   /tmp

0   /usr

10) rm Command

HDFS Command to remove the file from HDFS. ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -rm /hadoop/test**
16/11/07 015329 INFO fs.TrashPolicyDefault Namenode trash configuration Deletioninterval = 0 minutes, Emptier interval = 0 minutes.
Deleted /hadoop/test

11) expunge Command

HDFS Command that makes the trash empty. ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -expunge**
16/11/07 015554 INFO fs.TrashPolicyDefault Namenode trash configuration Deletioninterval = 0 minutes, Emptier interval = 0 minutes.

12) rm -r Command

HDFS Command to remove the entire directory and all of its content from HDFS.
ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -rm -r /hadoop/hello**
16/11/07 015852 INFO fs.TrashPolicyDefault Namenode trash configuration Deletioninterval = 0 minutes, Emptier interval = 0 minutes.

Deleted /hadoop/hello

13) chmod Command

Change the permissions of files.

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -chmod 777 /hadoop**

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -ls /**

Found 5 items

drwxrwxrwx - ubuntu supergroup 0 2016-11-07 0158 /hadoop

drwxr-xr-x - ubuntu supergroup 0 2016-11-07 0126 /system

drwxr-xr-x - ubuntu supergroup 0 2016-11-07 0111 /test

drwxr-xr-x - ubuntu supergroup 0 2016-11-07 0109 /tmp

drwxr-xr-x - ubuntu supergroup 0 2016-11-07 0109 /usr

14) get Command

HDFS Command to copy files from hdfs to the local file system.

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -get /hadoop/test /home/ubuntu/Desktop/**

ubuntu@ubuntu-VirtualBox~$ **ls -l /home/ubuntu/Desktop/**
total 4

-rw-r--r-- 1 ubuntu ubuntu 16 Nov 8 0047 test

15) cat Command

HDFS Command that copies source paths to stdout. ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -cat /hadoop/test**This is a test.

16) touchz Command

HDFS Command to create a file in HDFS with file size 0 bytes. ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -touchz /hadoop/sample**ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -ls /hadoop**

Found 2 items

-rw-r--r-- 2 ubuntu supergroup 0 2016-11-08 0057 /hadoop/sample

-rw-r--r-- 2 ubuntu supergroup 16 2016-11-08 0045 /hadoop/test

17) text Command

HDFS Command that takes a source file and outputs the file in text format.ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -text /hadoop/test**

This is a test.

18) copyFromLocal Command

HDFS Command to copy the file from Local file system to HDFS.

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -copyFromLocal /home/ubuntu/new /hadoop**

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -ls /hadoop**

Found 3 items

-rw-r—r--	2 ubuntu supergroup	43 2016-11-08 0108 /hadoop/new
-rw-r—r--	2 ubuntu supergroup	0 2016-11-08 0057 /hadoop/sample
-rw-r—r--	2 ubuntu supergroup	16 2016-11-08 0045 /hadoop/test

19) copyToLocal Command

Similar to get command, except that the destination is restricted to a local file reference.
ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -copyToLocal /hadoop/sample /home/ubuntu/**
ubuntu@ubuntu-VirtualBox~$ **ls -l s***
-rw-r--r-- 1 ubuntu ubuntu 0 Nov 8 0112 sample

-rw-rw-r-- 1 ubuntu ubuntu 102436055 Jul 20 0447 sqoop-1.99.7-bin-hadoop200.tar.gz

20) mv Command

HDFS Command to move files from source to destination. This command allows multiple sources as well, in which case the destination needs to be a directory.

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -mv /hadoop/sample /tmp**

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -ls /tmp**

Found 1 items

-rw-r--r-- 2 ubuntu supergroup 0 2016-11-08 0057 /tmp/sample

21) cp Command

HDFS Command to copy files from source to destination. This command allows multiple sources as well, in which case the destination must be a directory.

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -cp /tmp/sample /usr**

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -ls /usr**

Found 1 items

-rw-r--r-- 2 ubuntu supergroup 0 2016-11-08 0122 /usr/sample

22) tail Command

Displays last kilobyte of the file "new" to stdout ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -tail /hadoop/new**This is a new file.
Running HDFS commands.

23) chown Command

HDFS command to change the owner of files.

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -chown rootroot /tmp**

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -ls /**

Found 5 items

drwxrwxrwx - ubuntu supergroup 0 2016-11-08 0117 /hadoop

drwxr-xr-x	- ubuntu supergroup	0 2016-11-07 0126 /system
drwxr-xr-x	- ubuntu supergroup	0 2016-11-07 0111 /test
drwxr-xr-x	- root root	0 2016-11-08 0117 /tmp
drwxr-xr-x	- ubuntu supergroup	0 2016-11-08 0122 /usr

24) setrep Command

Default replication factor to a file is 3. Below HDFS command is used to change replication factor of a file.

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -setrep -w 2 /usr/sample**

Replication 2 set /usr/sample Waiting for /usr/sample ... done

25) distcp Command

Copy a directory from one node in the cluster to another

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -distcp hdfs//namenodeA/apache_hadoop hdfs//namenodeB/hadoop**

26) stat Command

Print statistics about the file/directory at <path> in the specified format. Format accepts filesize in blocks (%b), type (%F), group name of owner (%g), name (%n), block size (%o), replication (%r), user name of owner(%u), and modification date (%y, %Y). %y shows UTC date as “yyyy-MM-dd HHmmss” and %Y shows milliseconds since January 1, 1970 UTC. If the format is not specified, %y is used by default.

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -stat "%F %u%g %b %y %n" /hadoop/test**

regular file ubuntusupergroup 16 2016-11-07 191522 test

27) getfacl Command

Displays the Access Control Lists (ACLs) of files and directories. If a directory has a default ACL, then getfacl also displays the default ACL.

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -getfacl /hadoop**

file /hadoop# owner ubuntu
group supergroup

28) du -s Command

Displays a summary of file lengths.

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -du -s /hadoop**

59 /hadoop

29) checksum Command

Returns the checksum information of a file.

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -checksum /hadoop/new**

/hadoop/new MD5-of-0MD5-of-512CRC32C 000002000000000000000000639a5d8ac275be8d0c2b055d75208265

30) getmerge Command

Takes a source directory and a destination file as input and concatenates files in src into the destination local file.

ubuntu@ubuntu-VirtualBox~$ **cat test**

This is a test.

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -cat /hadoop/new**

This is a new file.

Running HDFS commands.

ubuntu@ubuntu-VirtualBox~$ **hdfs dfs -getmerge /hadoop/new test**

ubuntu@ubuntu-VirtualBox~$ **cat test**

This is a new file.
Running HDFS commands.

8. RACK AWARENESS IN HADOOP HDFS

1. Objective

This Hadoop tutorial will help you in understanding **Hadoop rack awareness** concept, racks in Hadoop environment, why rack awareness is needed, replica placement policy in Hadoop via Rack awareness and advantages of implementing rack awareness in Hadoop **HDFS.**

2. What is Rack Awareness in Hadoop HDFS?

In a large cluster of Hadoop, in order to improve the network traffic while reading/writing HDFS file, namenode chooses the datanode which is closer to the same rack or nearby rack to Read/Write request. Namenode achieves rack information by maintaining the rack id's of each datanode. This concept that chooses closer datanodes based on the rack information is called Rack Awareness in Hadoop.
Rack awareness is having the knowledge of Cluster topology or more specifically how the different data nodes are distributed across the racks of a Hadoop cluster. Default Hadoop installation assumes that all data nodes belong to the same rack.

3. Why Rack Awareness?

In **Big data** Hadoop, rack awareness is required for below reasons

- To improve data **high availability** and reliability.
- **Improve the performance** of the cluster.
- To improve network bandwidth.
- Avoid losing data if entire rack fails though the chance of the rack failure is far less than that of node failure.
- To keep bulk data in the rack when possible.
- An assumption that in-rack id's higher bandwidth, lower latency.

4. Replica Placement via Rack Awareness in Hadoop

Placement of replica is critical for ensuring high reliability and performance of HDFS. Optimizing replica placement via rack awareness distinguishes HDFS from other Distributed

File System. Block Replication in multiple racks in HDFS is done using a policy as follows

“No more than one replica is placed on one node. And no more than two replicas are placed on the same rack. This has a constraint that the number of racks used for block replication should be less than the total number of block replicas”.

For Example

When a new block is created The First replica is placed on the local node. The Second one is placed on a different rack and the third one is placed on a different node at the local rack.
When re-replicating a block, if the number of an existing replica is one, place the second one on the different rack. If the number of an existing replica is two and if the two existing replicas are on the same rack, the third replica is placed on a different rack.

A simple but nonoptimal policy is to place replicas on the different racks. This prevents losing data when an entire rack fails and allows us to use bandwidth from multiple racks while reading the data. This policy evenly distributes the data among replicas in the cluster which makes it easy to balance load in case of component failure. But the biggest drawbackof this policy is that it will increase the cost of write operation because a writer needs to transfer blocks to multiple racks and communication between the two nodes in different rackshas to go through switches.

In most cases, network bandwidth between machines in the same rack is greater than networkbandwidth between machines in different racks. That's why we use replica replacement policy. The chance of the rack failure is far less than that of node failure. It does not impacton data reliability and availability guarantee. However, it does reduce the aggregate network bandwidth used when reading data since a block replica is placed in only two unique racks rather than three.

4.1. What about performance?

- Faster replication operation Since the replicas are placed within the same rack it would use higher bandwidth and lower latency hence making it faster.
- If **YARN** is unable to create a container in the same data node where the queried data is located it would try to create the container in a data node within the same rack. Thiswould be more performant because of the higher bandwidth and lower latency of the data nodes inside the same rack.

5. Advantages of Implementing Rack Awareness

- **Minimize the writing cost and Maximize read speed –** Rack awareness places read/write requests to replicas on the same or nearby rack. Thus minimizing writing cost and maximizing reading speed.
- **Provide maximize network bandwidth and low latency –** Rack awareness maximizes network bandwidth by blocks transfer within a rack. Especially with rack awareness, the YARN is able to optimize **MapReduce** job performance. It assigns tasks to nodes that are 'closer' to their data in terms of network topology.

This is particularly beneficial in cases where tasks cannot be assigned to nodes where their data is stored locally.

- **Data protection against rack failure –** By default, the namenode assigns 2nd & 3rd replicas of a block to nodes in a rack different from the first replica. This providesdata protection even against rack failure; however, this is possible only if Hadoop wasconfigured with knowledge of its rack configuration.

CHAPTER 4

MAP REDUCE

CONTENTS

- Map Reduce Architecture
- Job submission
  - Job Initialization
  - Task Assignment
  - Task execution
  - Progress and status updates
  - Job Completion
- Shuffle and sort on Map and reducer side
- Map Reduce Types
- Input formats
- Output formats
- Sorting
- Map side and Reduce side joins
- Map Reduce Programs
  - Word Count Program
  - Maximum Temperature Program

1. MAP REDUCE ARCHITECTURE

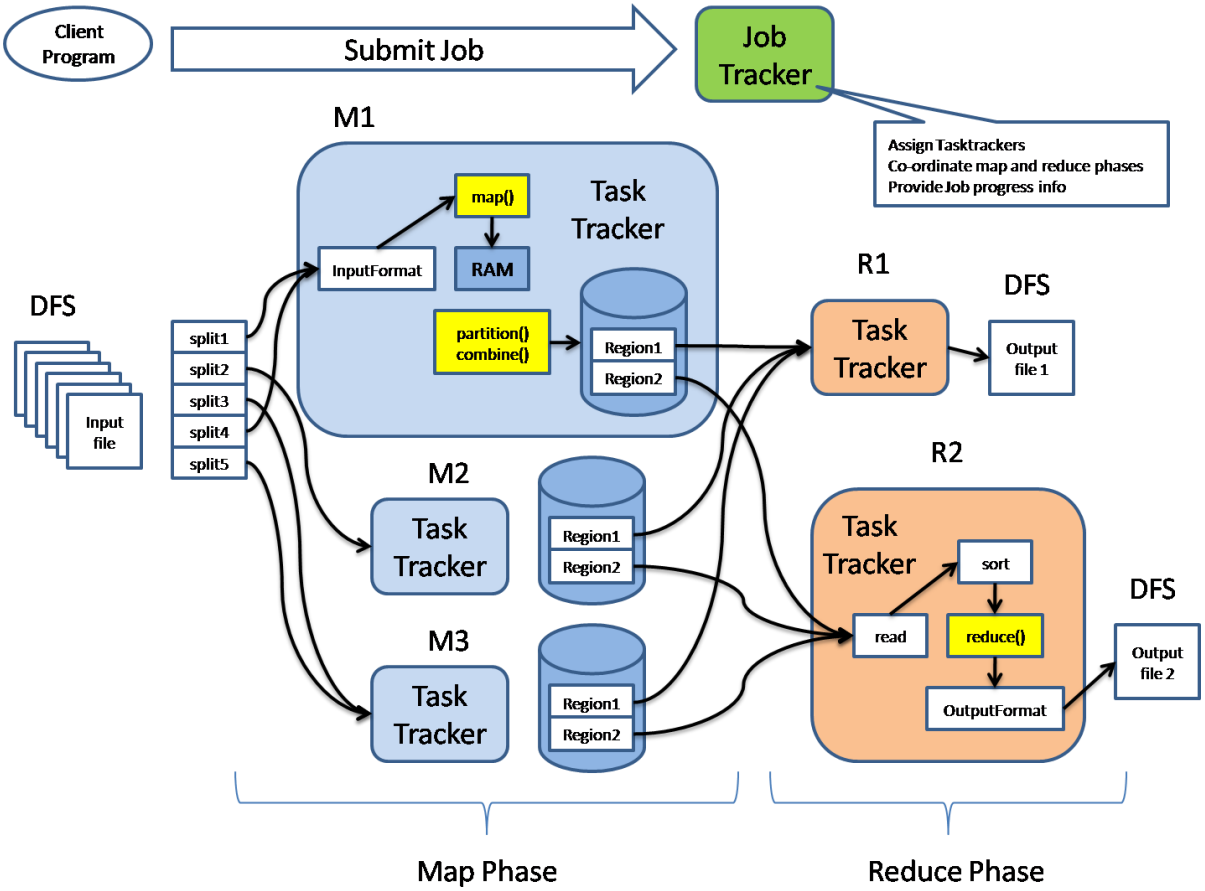
MapReduce is mainly used for parallel processing of large sets of data stored in Hadoop cluster. Initially, it is a hypothesis specially designed by Google to provide parallelism, data distribution and fault-tolerance. MR processes data in the form of key-value pairs. A key- value (KV) pair is a mapping element between two linked data items - key and its value.

The key (K) acts as an identifier to the value. An example of a key-value (KV) pair is a pair where the key is the node Id and the value is its properties including neighbor nodes, predecessor node, etc. MR API provides the following features like batch processing, parallel processing of huge amounts of data and high availability.

For processing large sets of data MR comes into the picture. The programmers will write MR

applications that could be suitable for their business scenarios. Programmers have to understand the MR working flow and according to the flow, applications will be developed and deployed across Hadoop clusters. Hadoop built on Java APIs and it provides some MR APIs that is going to deal with parallel computing across nodes.

The MR work flow undergoes different phases and the end result will be stored in hdfs with replications. Job tracker is going to take care of all MR jobs that are running on various nodes present in the Hadoop cluster. Job tracker plays vital role in scheduling jobs and it will keep track of the entire map and reduce jobs. Actual map and reduce tasks are performed by Task tracker.



Map reduce architecture consists of mainly two processing stages. First one is the map stage and the second one is reduced stage. The actual MR process happens in task tracker. In between map and reduce stages, Intermediate process will take place. Intermediate process will do operations like shuffle and sorting of the mapper output data. The Intermediate data isgoing to get stored in local file system.

Mapper Phase

In Mapper Phase the input data is going to split into 2 components, Key and Value. The key is writable and comparable in the processing stage. Value is writable only during the processing stage. Suppose, client submits input data to Hadoop system, the Job trackerassigns tasks to task tracker. The input data that is going to get split into several input splits.

Input splits are the logical splits in nature. Record reader converts these input splits in Key-Value (KV) pair. This is the actual input data format for the mapped input for further processing of data inside Task tracker. The input format type varies from one type ofapplication to another. So the programmer has to observe input data and to code according.

Suppose we take Text input format; the key is going to be byte offset and value will be the entire line. Partition and combiner logics come in to map coding logic only to perform special data operations. Data localization occurs only in mapper nodes.

Combiner is also called as mini reducer. The reducer code is placed in the mapper as a combiner. When mapper output is a huge amount of data, it will require high network bandwidth. To solve this bandwidth issue, we will place the reduced code in mapper as combiner for better performance. Default partition used in this process is Hash partition.

A partition module in Hadoop plays a very important role to partition the data received from either different mappers or combiners. Petitioner reduces the pressure that builds on reducer and gives more performance. There is a customized partition which can be performed on any relevant data on different basis or conditions.

Also, it has static and dynamic partitions which play a very important role in Hadoop as well as hive. The partitioner would split the data into numbers of folders using reducers at the end of map reduce phase. According to the business requirement developer will design this partition code. This partitioner runs in between Mapper and Reducer. It is very efficient for query purpose.

Intermediate Process

The mapper output data undergoes shuffle and sorting in intermediate process. The intermediate data is going to get stored in local file system without having replications in Hadoop nodes. This intermediate data is the data that is generated after some computations based on certain logics. Hadoop uses a Round-Robin algorithm to write the intermediate data to local disk. There are many other sorting factors to reach the conditions to write the data to local disks.

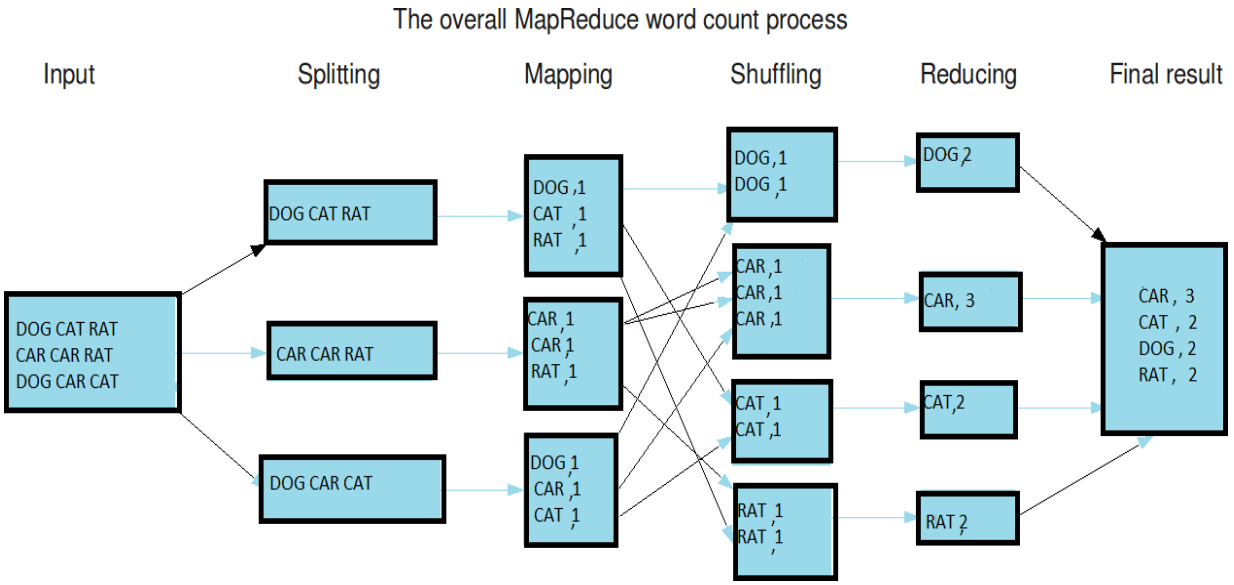
Reducer Phase

Shuffled and sorted data is going to pass as input to the reducer. In this phase, all incoming data is going to combine and same actual key value pairs is going to write into hdfs system. Record writer writes data from reducer to HDFS. The reducer is not so mandatory for searching and mapping purpose.

Reducer logic is mainly used to start the operations on mapper data which is sorted and finally it gives the reducer outputs like part-r-0001etc, Options are provided to set thenumber of reducers for each job that the user wanted to run. In the configuration file mapped- site.xml, we have to set some properties which will enable to set the number of reducers for the particular task.

Speculative Execution plays an important role during job processing. If two or more mappers are working on the same data and if one mapper is running slow then Job tracker assigns tasks to the next mapper to run the program fast. The execution will be on FIFO (First In First Out).

MapReduce word count Example



Suppose the text file having the data like as shown in Input part in the above figure. Assume that, it is the input data for our MR task. We have to find out the word count at end of MR Job. The internal data flow can be shown in the above example diagram. The line splits in splitting phase and gives a key value pair to input by record reader.

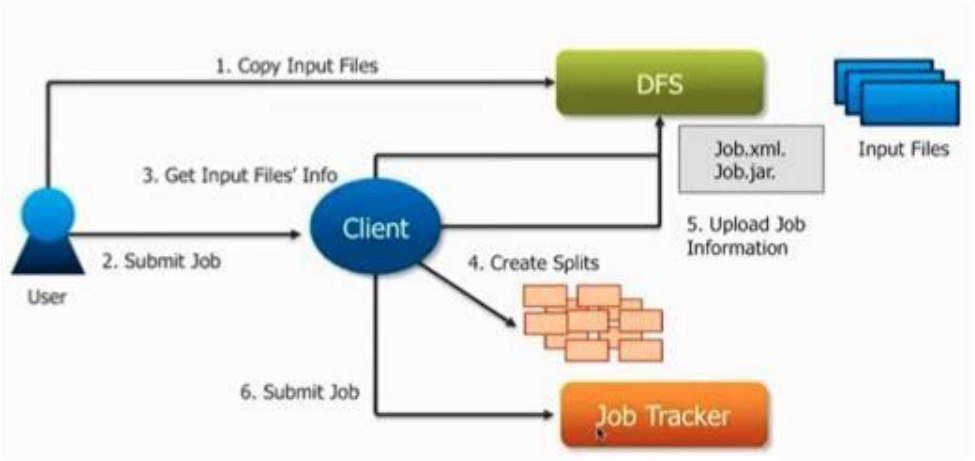
Here, three mappers are running parallel and each mapper task is going to generate output for each input row that comes as input to it. After mapper phase, the data is going to shuffle and sort. All the grouping will be done here and the value is passed as input to Reducer phase. The reducers then finally combine each key-value pair and pass those values to HDFS via record writer.

2. MAP REDUCE JOBS

Job submission- From the time a user fires any job from a Client/Edge node or Cluster node, until the time thejob actually gets submitted to the JobTracker for execution.

Users submitting a job communicate with the cluster via JobClient, which is the interface for the user-job to interact with the cluster. JobClient provides a lot of facilities, such as job submission, progress tracking, accessing of component-tasks' reports/logs, Map-Reduce cluster status information, etc.

MR Flow overview



The above figure gives a good high-level overview for the flow in MR1 in terms of how a job gets submitted to JobTracker. Below are the steps which are followed when any MR job is submitted by the user until it gets submitted to JobTracker

- User copies input file to distributed file system
- User submits job
- Job client get input files info
- Creates splits
- Uploads job info i.e., Job.jar and Job.xml
- Validation of Job Output directory is done via HDFS API call; then client submits job to JobTracker using RPC call

Once the job is submitted to JobTracker, it assumes it is JobTracker's responsibility to distribute the job to the TT's, schedule tasks and monitor them, and provide status and diagnostic information back to the job-client. Details of a job submission on the JobTracker side is out of scope for this post, but I plan to write a dedicated post in the future which details job flow for both JobTracker and TaskTracker.

Now that we have understood the flow completely, let's associate the above steps with the log lines when a job does get submitted. I spun up a cluster to demonstrate this

Environment

On the client node, where I plan to fire a WordCount job for demonstration purposes, I changed the log level of log4j.logger.org.apache.hadoop.mapred.JobClient class to DEBUG by editing "/opt/mapr/hadoop/hadoop-0.20.2/conf/log4j.properties" file

Logging levels

log4j.logger.org.apache.hadoop.security.JniBasedUnixGroupsMapping=WARN
log4j.logger.org.apache.hadoop.util.NativeCodeLoader=WARN
log4j.logger.org.apache.hadoop.mapred.JobTracker=INFO
log4j.logger.org.apache.hadoop.mapred.TaskTracker=INFO

log4j.logger.org.apache.hadoop.mapred.JobClient=DEBUG
log4j.logger.org.apache.zookeeper=INFO
log4j.logger.org.apache.hadoop.mapred.MapTask=WARN
log4j.logger.org.apache.hadoop.mapred.ReduceTask=WARN
#log4j.logger.org.apache.hadoop.fs.FSNamesystem=DEBUG

With the above DEBUG enabled, it appears that we didn't get enough log messages which would actually list every step that we discussed earlier, so we had to modify code in the below jar to print custom debug log lines in order to understand and validate the flow.
/opt/mapr/hadoop/hadoop-0.20.2/lib/hadoop-0.20.2-dev-core.jar

- As a first step, I copied the input file to the Distributed File System. The file "/myvolume/in" is roughly 1.5 MB in size, on which I will run the WordCount job.

[root@ip-10-255-68-164 conf]# hadoop fs -du /myvolume/in
Found 1 items
1573079 maprfs/myvolume/in
[root@ip-10-255-68-164 conf]#

- Now we submit the WordCount job to the JobClient as shown below. Here I just added a custom split size while executing the job to make sure our job will run two map tasks in parallel, since we will get two splits for our input file, i.e., inputfile/custom split size.

hadoop jar /opt/mapr/hadoop/hadoop-0.20.2/hadoop-0.20.2-dev-examples.jar wordcount -Dmapred.max.split.size=786432 /myvolume/in /myvolume/out >> /tmp/JobClient-WC-split1 2>&1

Note When the JobClient initiates, you will see the messages below, which are due to the fact that MapR supports JobTracker high availability. It connects to ZooKeeper to find which is currently the active JobTracker for communication, and gets the JobId for the current job by making an RPC call to JobTracker.

15/06/27 073833 INFO zookeeper.ZooKeeper Initiating client connection, connectString=ec2-54-177-98-44.us-west-1.compute.amazonaws.com5181,ec2-184-169-213-71.us-west-1.compute.amazonaws.com5181,ec2-184-169-212-210.us-west-1.compute.amazonaws.com5181 sessionTimeout=30000 watcher=com.mapr.fs.JobTrackerWatcher@7b05d0ae
15/06/27 073833 INFO zookeeper.Login successfully logged in.
15/06/27 073833 INFO client.ZooKeeperSaslClient Client will use SIMPLE-SECURITYas SASL mechanism.
15/06/27 073833 INFO zookeeper.ClientCnxn Opening socket connection to server ip-10- 255-0-66.us-west-1.compute.internal/10.255.0.665181\. Will attempt to SASL-authenticate using Login Context section 'Client_simple'
15/06/27 073833 INFO zookeeper.ClientCnxn Socket connection established to ip-10-255- 0-66.us-west-1.compute.internal/10.255.0.665181, initiating session
15/06/27 073833 INFO zookeeper.ClientCnxn Session establishment complete on server ip-10-255-0-66.us-west-1.compute.internal/10.255.0.665181, sessionid = 0x24dd4a5bb6a0896, negotiated timeout = 30000
15/06/27 073833 INFO fs.JobTrackerWatcher Current running JobTracker is ip-10-128- 202-14.us-west-1.compute.internal/10.128.202.149001

Custom Debug Log Lines### got the jobId 7 for the job submitted jobsubmit dir is maprfs/var/mapr/cluster/mapred/jobTracker/staging/root/.staging/job_201506202152_0007

- Now the JobClient checks if there are any custom library jars, then inputs files specified during job execution and creates a Job directory, libjars directory, archives directory and files directory under JobTracker volume to place temporary files during job execution (the distributed cache which can be used during job processing).

Custom Debug Log Lines### files null libjars null archives null
15/06/27 073833 DEBUG mapred.JobClient default FileSystem maprfs///
Custom Debug Log Lines### submitJobDir /var/mapr/cluster/mapred/jobTracker/staging/root/.staging/job_201506202152_0007
Custom Debug Log Lines### filesDir /var/mapr/cluster/mapred/jobTracker/staging/root/.staging/job_201506202152_0007/files archivesDir /var/mapr/cluster/mapred/jobTracker/staging/root/.staging/job_201506202152_0007/archives libjarsDir /var/mapr/cluster/mapred/jobTracker/staging/root/.staging/job_201506202152_0007/libjars

- Now the JobClient starts creating splits for the input file. It generated two splits, since we choose custom split size and have one input file, which is roughly two times the split size. Finally, this split meta information is written to a file under the temp job directory as well (under the JobTracker volume).

15/06/27 073833 DEBUG mapred.JobClient Creating splits at maprfs/var/mapr/cluster/mapred/jobTracker/staging/root/.staging/job_201506202152_0007
Custom Debug Log Lines### creating splits at maprfs/var/mapr/cluster/mapred/jobTracker/staging/root/.staging/job_201506202152_0007
Custom Debug Log Lines### writing splits and calculating number of map
15/06/27 073833 INFO input.FileInputFormat Total input paths to process 1
15/06/27 073833 WARN snappy.LoadSnappy Snappy native library is available
15/06/27 073833 INFO snappy.LoadSnappy Snappy native library loaded
Custom Debug Log Lines### splits generated number of splits is 2
Custom Debug Log Lines### splits are sorted into order based on size so that the biggest go first
Custom Debug Log Lines### split file maprfs/var/mapr/cluster/mapred/jobTracker/staging/root/.staging/job_201506202152_0007/job.split no of replication set is 10
Custom Debug Log Lines### writing new splits
Custom Debug Log Lines### wiriting split meta info
Custom Debug Log Lines### splits meta-info file for job tracker maprfs/var/mapr/cluster/mapred/jobTracker/staging/root/.staging/job_201506202152_0007/job.splitmetainfo

- Job jar and job.xml are also copied to the shared job directory (under JobTracker volume) for it to be available when Jobtracker starts job execution.

Custom Debug Log Lines### copying jar file /var/mapr/cluster/mapred/jobTracker/staging/root/.staging/job_201506202152_0007/job.jar

Custom Debug Log Lines### submitjobfile maprfs/var/mapr/cluster/mapred/jobTracker/staging/root/.staging/job_201506202152_0007/job.xml

- Finally, the JobClient checks if the output directory exists. If it does, job initialization fails to prevent output directory from being overwritten. If it doesn't exist, it will be created and the job is submitted to JobTracker.

Note Checking if the output directory exists is not done on client side; it is done via an HDFS interface call from the client.

15/06/27 073833 INFO mapred.JobClient Creating job's output directory at /myvolume/out
15/06/27 073833 INFO mapred.JobClient Creating job's user history location directory at /myvolume/out/_logs
15/06/27 073834 INFO mapred.JobClient root, realuser null
15/06/27 073834 DEBUG mapred.JobClient Printing tokens for job job_201506202152_0007
Custom Debug Log Lines### submitting the job to JobTracker

Job Submission, Job Initialization, Task Assignment, Task execution, Progress and status updates, Job Completion

You can run a mapreduce job with a single method call submit() on a Job object or you can also call waitForCompletion(), which submits the job if it hasn't been submitted already, then waits for it to finish.

The below is the structure at high level

1. The client, which submits the MapReduce job.
2. The YARN resource manager, which coordinates the allocation of compute resources on the cluster.

3. The YARN node managers, which launch and monitor the compute containers on machines in the cluster.

4. The MapReduce application master, which coordinates the tasks running the Map-Reduce job. The application master and the MapReduce tasks run in containers that are scheduled by the resource manager and managed by the node managers

5. The distributed filesystem, which is used for sharing job files between the other entities.

Job Submission

The submit() method on Job creates an internal JobSubmitter instance and calls submitJobInternal() on it Having submitted the job, waitFor Completion() polls the job's progress once per second and reports the progress to the console if it has changed since the last report. When the job completes successfully, the job counters are displayed. Otherwise, the error that caused the job to fail is logged to the console.

The job submission process implemented by JobSubmitter does the following

1. Asks the resource manager for a new application ID, used for the MapReduce job ID.

2. Checks the output specification of the job. For example, if the output directory has not been specified or it already exists, the job is not submitted and an error is thrown to the MapReduce program.

3. Computes the input splits for the job. If the splits cannot be computed (because the input paths don't exist, for example), the job is not submitted and an error is thrown to the MapReduce program.

4. Copies the resources needed to run the job, including the job JAR file, the configuration file, and the computed input splits, to the shared filesystem in a directory named after the job ID. The job JAR is copied with a high replication factor controlled by the mapreduce.client.submit.file.replication property, which defaults to 10 so that there are lots of copies across the cluster for the node managers to access when they run tasks for the job.

The client running the job calculates the splits for the job by calling getSplits() on the inputformat class, then sends them to the application master, which uses their storage locations to schedule map tasks that will process them on the cluster. The map task passes thesplit to the createRecordReader() method on InputFormat to obtain a RecordReader for that split. A RecordReader is little more than an iterator over records, and the map task uses oneto generate record key-value pairs, which it passes to the map function.

5. Submits the job by calling submitApplication() on the resource manager

Job Initialization

1. Resource manager receives a call to its submitApplication() it hands off the request to the YARN scheduler.
2. The scheduler allocates a container, and the resource manager then launches the application master's process there, under thenode manager's management.

3. The application master for MapReduce jobs is a Java application whose main class is MRAppMaster. It initializes the job by creating a number of bookkeeping objects to keep track of the job's progress, as it will receive progress and completion reports from thetasks.

4. Next, it retrieves the input splits computed in the client from the shared filesystem. It then creates a map task object for each split, as well as a number of reduce task objects determined by the mapreduce.job.reduces property which is set by the setNumReduceTasks() method on Job. Tasks are given IDs at this point.

5. The application master must decide how to run the tasks that make up the MapReduce job. If the job is small, the application master may choose to run the tasks in the same JVM as itself. This happens when it judges that the overhead of allocating and running tasks in new containers outweighs the gain to be had in running them in parallel, compared to runningthem sequentially on one node. Such a job is said to be uberized, or run as an uber task.

6. Finally, before any tasks can be run, the application master calls the setupJob() method on the OutputCommitter. For FileOutputCommitter, which is the default, it will create the final output directory for the job and the temporary working space for the task output.

Note By default, a small job is one that has less than 10 mappers,only one reducer, and an input size that is less than the size of one HDFS block. And these values may be changed fora job by mapreduce.job.ubertask.maxmaps, mapreduce.job.ubertask.maxreduces, and map reduce.job.ubertask.maxbytes. Uber tasks must be enabled explicitly for an individual job, or across the cluster by setting mapreduce.job.ubertask.enable to true.

Task Assignment

1. If the job does not qualify for running as an uber task, then the application master requests containers for all the map and reduce tasks in the job from the resource manager. Requests for map tasks are made first and with a higher priority than those for reduce tasks, since all the map tasks must complete before the sort phase of the reduce can start. Requests for reduce tasks are not made until 5% of map tasks have completed.

2. Reduce tasks can run anywhere in the cluster, but requests for map tasks have data locality constraints that the scheduler tries to honor
In the optimal case, the task is data local—that is, running on the same node that the split resides on. Alternatively, the task may be rack local on the same rack, but not the same node, as the split. Some tasks are neither data local nor rack local and retrieve their data from a different rack than the one they are running on. For a particular job run, you can determine the number of tasks that ran at each locality level by looking at the job's counters which is DATA_LOCAL_MAPS.

3. Requests also specify memory requirements and CPUs for tasks. By default, each map and reduce task is allocated 1,024 MB of memory and one virtual core. The values are configurable on a per-job basis via the following properties mapreduce.map.memory.mb, mapreduce.reduce.memory.mb, mapreduce.map.cpu.vcores and mapreduce.reduce.cpu.vcores.

Task Execution

1. Once a task has been assigned resources for a container on a particular node by the resource manager's scheduler, the application master starts the container by contacting the node manager.

2. The task is executed by a Java application whose main class is YarnChild. Before it can run the task, it localizes the resources that the task needs, including the job configuration and JAR file, and any files from the distributed cache.

3. Finally, it runs the map or reduce task.

Note The YarnChild runs in a dedicated JVM, so that any bugs in the user-defined map and reduce functions or even in YarnChild don't affect the node manager by causing it to crash or hang.Each task can perform setup and commit actions, which are run in the same JVM as the task itself and are determined by the OutputCommitter for the job . For file-based jobs, the commit action moves the task output from a temporary location to its final location. The commit protocol ensures that when speculative execution is enabled, only one of the duplicate tasks is committed and the other is aborted.

Progress and Status Updates

When a task is running, it keeps track of its progress that is the proportion of the task completed. For map tasks, this is the proportion of the input that has been processed. For reduce tasks, it's a little more complex, but the system can still estimate the proportion of the reduce input processed. It does this by dividing the total progress into three parts, corresponding to the three phases of the shuffle.

Progress reporting is important, as Hadoop will not fail a task that's making progress. All of the following operations constitute progress

1. Reading an input record in a mapper or reducer.

2. Writing an output record in a mapper or reducer.

3. Setting the status description via Reporter's or TaskAttemptContext's setStatus() method.

4. Incrementing a counter using Reporter's incrCounter() method or Counter's increment() method.

5. Calling Reporter's or TaskAttemptContext's progress () method.

Note As the map or reduce task runs, the child process communicates with its parent application master through the umbilical interface. The task reports its progress and status including counters back to its application master, which has an aggregate view of the job, every three seconds over the umbilical interface.

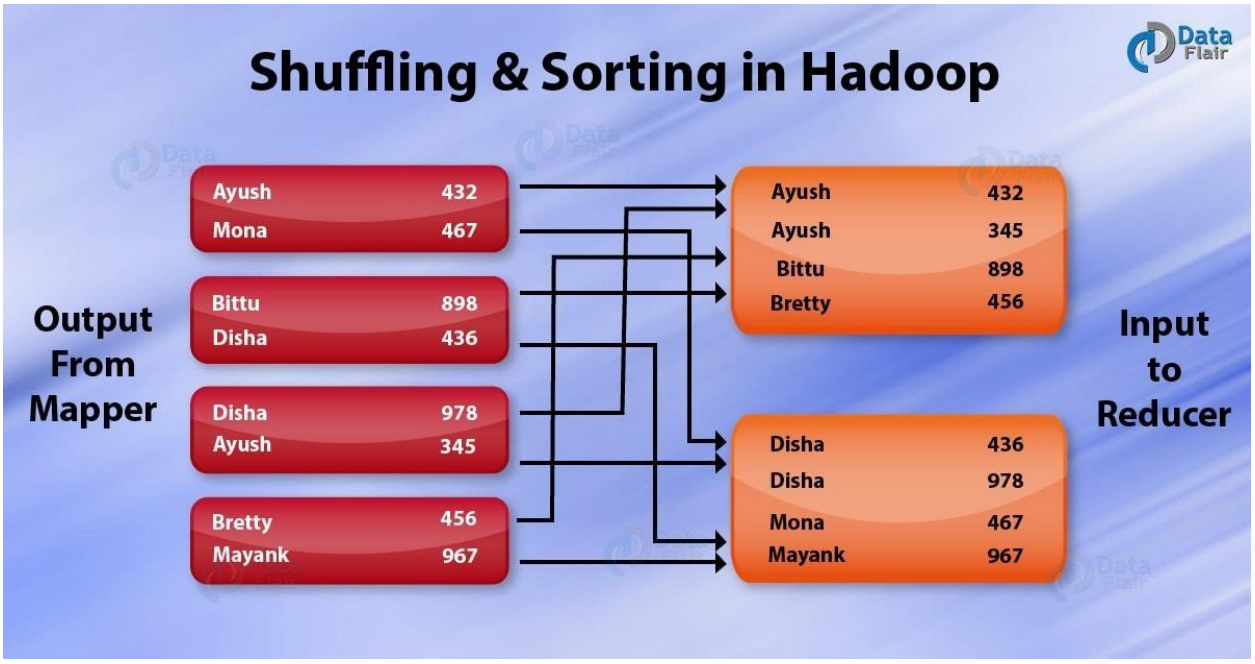
Job Completion

When the application master receives a notification that the last task for a job is complete, it changes the status for the job to successful. Then, when the Job polls for status, it learns that the job has completed successfully, so it prints a message to tell the user and then returns from the waitForCompletion() method. Job statistics and counters are printed to the consoleat this point.

Finally, on job completion, the application master and the task containers clean up their working state so intermediate output is deleted, and the OutputCommitter's commit Job() method is called. Job information is archived by the job history server to enable later interrogation by users if desired.

3. SHUFFLE AND SORT ON MAP AND REDUCER SIDE

In **Hadoop**, the process by which the intermediate output from **mappers** is transferred to the **reducer** is called Shuffling. Reducer gets 1 or more keys and associated values on the basis of reducers. Intermediated **key-value** generated by mapper is sorted automatically by key.



When you run a MapReduce job and mappers start producing output internally lots of processing is done by the Hadoop framework before the reducers get their input. Hadoop framework also guarantees that the map output is sorted by keys. This whole internal processing of sorting map output and transfering it to reducers is known as shuffle phase in Hadoop framework.

The tasks done internally by Hadoop framework with in the shuffle phase are as follows-

1. Data from mappers is partitioned as per the number of reducers.
2. Data is also sorted by keys with in a partition.
3. Output from Maps is written to disk as many temporary files.
4. Once the map task is finished all the files written to the disk are merged to create a single file.
5. Data from a particular partition (from all mappers) is transferred to a reducer that is supposed to process that particular partition.
6. If data transferred to a reducer exceeded the memory limit, then it is copied to a disk.
7. Once reducer has got its portion of data from all the mappers data is again merged while still maintaining the sort order of keys to create reduce task input.

6. As you can see some of the shuffle phase tasks happen at the nodes where mappers are running and some of them at the nodes where reducers are running.

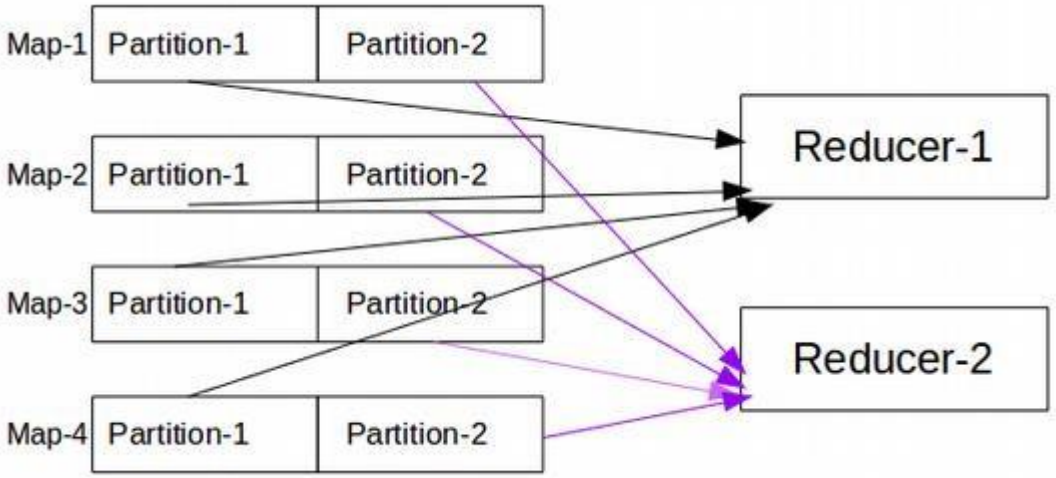
Shuffle phase process at mappers side

When the map task starts producing output it is not directly written to disk instead there is a memory buffer (size 100 MB by default) where map output is kept. This size is configurable and parameter that is used is – **mapreduce.task.io.sort.mb**

When that data from memory is spilled to disk is controlled by the configuration parameter **mapreduce.map.sort.spill.percent** (default is 80% of the memory buffer). Once this threshold of 80% is reached, a thread will begin to spill the contents to disk in the background.

Before writing to the disk the *Mapper outputs are sorted and then partitioned per Reducer*. The total number of partitions is the same as the number of reduce tasks for the job. For

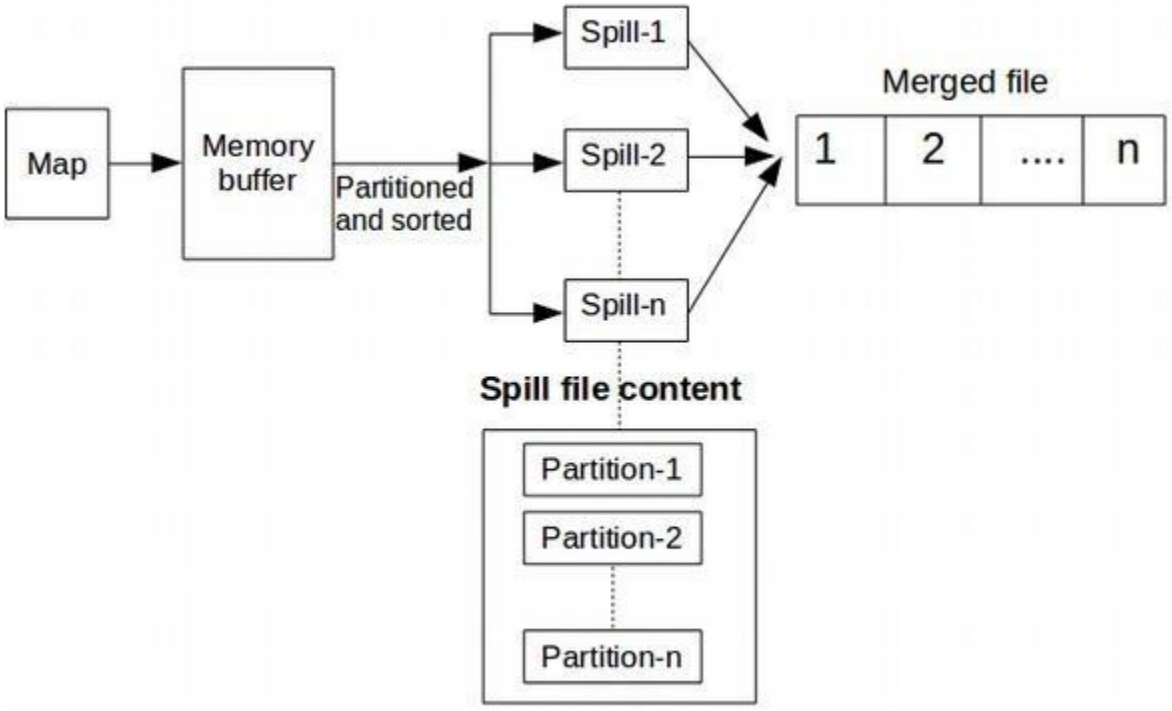
example, let's say there are 4 mappers and 2 reducers for a MapReduce job. Then output of all of these mappers will be divided into 2 partitions one for each reducer.



If there is a Combiner that is also executed in order to reduce the size of data written to the disk.

This process of keeping data into memory until threshold is reached, partitioning and sorting, creating a new spill file every time threshold is reached and writing data to the disk is repeated until all the records for the particular map tasks are processed. Before the Map taskis finished all these spill files are merged, keeping the data partitioned and sorted by keyswith in each partition, to create a single merged file.

Following image illustrates the shuffle phase process at the Map end.



Shuffle phase process at Reducer side

By this time, you have the Map output ready and stored on a local disk of the node where Map task was executed. Now the relevant partition of the output of all the mappers has to be transferred to the nodes where reducers are running.

Reducers don't wait for all the map tasks to finish to start copying the data, as soon as a Map task is finished data transfer from that node is started. **For example,** if there are 10 mappers running, framework won't wait for all the 10 mappers to finish to start map output transfer. As soon as a map task finish transfer of data starts.

Data copied from mappers is kept is memory buffer at the reducer side too. The size of the buffer is configured using the following parameter.

mapreduce.reduce.shuffle.input.buffer.percent- The percentage of memory- relative to the maximum heapsize as typically specified in mapreduce.reduce.java.opts- that can be allocated to storing map outputs during the shuffle. Default is 70%.

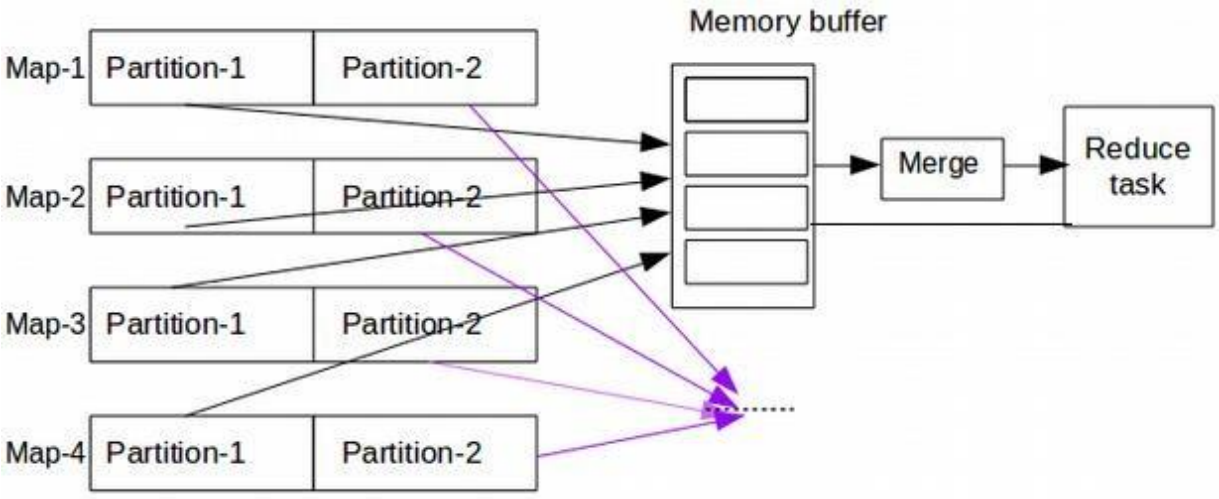
When the buffer reaches a certain threshold map output data is merged and written to disk.

This merging of Map outputs is known as **sort phase**. During this phase the framework groups Reducer inputs by keys since different mappers may have produced the same key as output.

The threshold for triggering the merge to disk is configured using the following parameter.

mapreduce.reduce.merge.inmem.thresholds- The number of sorted map outputs fetched into memory before being merged to disk. In practice, this is usually set very high (1000) or disabled (0), since merging in-memory segments is often less expensive than merging from disk.

The merged file, which is the combination of data written to the disk as well as the data still kept in memory constitutes the input for Reduce task.



Points to note-

1. The Mapper outputs are sorted and then partitioned per Reducer.
2. The total number of partitions is the same as the number of reduce tasks for the job.
3. Reducer has 3 primary phases shuffle, sort and reduce.
4. Input to the Reducer is the sorted output of the mappers.
5. In shuffle phase the framework fetches the relevant partition of the output of all the mappers, via HTTP.
6. In sort phase the framework groups Reducer inputs by keys from different map outputs.
7. The shuffle and sort phases occur simultaneously; while map-outputs are being fetched they are merged.

Shuffling in MapReduce

The process of transferring data from the mappers to reducers is known as shuffling i.e. the process by which the system performs the sort and transfers the map output to the reducer as input. So, MapReduce shuffle phase is necessary for the reducers, otherwise, they would not have any input (or input from every mapper). As shuffling can start even before the map phase has finished so this saves some time and completes the tasks in lesser time.

Sorting in MapReduce

The keys generated by the mapper are automatically sorted by MapReduce Framework, i.e. Before starting of reducer, all intermediate **key-value pairs** in MapReduce that are generated by mapper get sorted by key and not by value. Values passed to each reducer are not sorted; they can be in any order.

Sorting in Hadoop helps reducer to easily distinguish when a new reduce task should start. This saves time for the reducer. Reducer starts a new reduce task when the next key in the sorted input data is different than the previous. Each reduce task takes key-value pairs asinput and generates key-value pair as output.

Note that shuffling and sorting in Hadoop MapReduce is not performed at all if you specify zero reducers (setNumReduceTasks(0)). Then, the MapReduce job stops at the map phase, and the map phase does not include any kind of sorting (so even the map phase is faster).

Secondary Sorting in MapReduce

If we want to sort reducer's values, then the secondary sorting technique is used as it enables us to sort the values (in ascending or descending order) passed to each reducer.

4. MAPREDUCE TYPES

The first thing that comes into mind while writing a MapReduce program is the types we you are going to use in the code for Mapper and Reducer class.There are few points that should be followed for writing and understanding Mapreduce program.Here is a recap for the data types used in MapReduce (in case you have missed the **MapReduce Introduction** post).

Broadly the data types used in MapRduce are as follows.

- LongWritable-Corresponds to Java Long
- Text -Corresponds to Java String
- IntWritable -Corresponds to Java Integer
- NullWritable - Corrresponds to Null Values

Having a quick overview, we can jump over to the key thing that is data type in MapReduce. Now MapReduce has a simple model of data processing inputs and outputs for the map and reduce functions are key-value pairs

- The map and reduce functions in MapReduce have the following general form map (K1, V1) → list(K2, V2) reduce (K2, list(V2)) → list(K3, V3)
    - K1-Input Key
    - V1-Input value
    - K2-Output Key
    - V2-Output value
- In general,the map input key and value types (K1 and V1) are different from the map output types (K2 and V2). However, the reduce input must have the same types as the map output, although the reduce output types may be different again (K3 and V3).
- As said in above pont even though the map output types and the reduce input types must match, this is not enforced by the Java compiler. If the reduce output types may be different from the map output types (K2 and V2) then we have to specify in the code the types of both the map and reduce function else error will be thrown.So if k2 and k3 are the same, we don't need to call **setMapoutputKeyClass()**.Similarly, if v2 and v3 are the same, we only need to use **setOutputValueClass()**
- NullWritable is used when the user wants to pass either key or value (generally key) of map/reduce method as null.
- If a combine function is used, then it is the same form as the reduce function (and is an implementation of Reducer), except its output types are the intermediate key and value types (K2 and V2), so they can feed the reduce function map (K1, V1) → list(K2, V2) combine (K2, list(V2)) → list(K2, V2) reduce (K2, list(V2)) → list(K3, V3) Often the combine and reduce functions are the same, in which case K3 is the same as K2, and V3 is the same as V2.
- The partition function operates on the intermediate key and value types (K2 and V2) and returns the partition index. In practice, the partition is determined solely by the key (the value is ignored) partition (K2, V2) → integer

Default MapReduce JobNo Mapper, No Reducer

Ever tried to run MapReduce program without setting a mapper or a reducer? Here is the minimal MapReduce program.

```
public class MinimalMapReduce extends Configured implements Tool
{

    public int run(String[] args) throws Exception
    {
        Configuration conf = new Configuration();
        Job job = new Job(conf, "MinimalMapReduce");

        job.setJarByClass(MinimalMapReduce.class);
        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        return (job.waitForCompletion(true) ? 0 : 1);
    }

    public static void main (String[] args) throws Exception
    {
        int ecode = ToolRunner.run(new MinimalMapReduce(),args);
        System.exit(ecode);
    }
}
```

Run it over a small data and check the output. Here is little data which I used and the final result.You can take a larger data set.

```
1       Bruce   NewYork
2       Tom     London
3       Tony    Venice
4       John    Manchester
```

```
0       1       Bruce   NewYork
16      2       Tommy   NewYork
32      3       James   Venice
35      4       John    Manchester
```

Notice the result file we get after running the above code on the given data. It added an extra column with some numbers as data. What happened is the that the newly added column contains the key for every line. The number is the offset of the line from the first line i.e. how far the beginning of the first line is placed from the first line(0 of course)similarly how many characters away is the second line from first. Count the characters, it will be 16 and so on.

```
public interface Mapper<K1, V1, K2, V2> extends JobConfigurable,
    Closeable {
  void map(K1 key, V1 value, OutputCollector<K2, V2> output,
    Reporter reporter) throws IOException;
}

public interface Reducer<K2, V2, K3, V3> extends JobConfigurable,
    Closeable {
  void reduce(K2 key, Iterator<V2> values,
```

```
    OutputCollector<K3, V3> output, Reporter reporter)throws
      IOException;
}
```

The *OutputCollector* is the generalized interface of the Map-Reduce framework to facilitate collection of data output either by the *Mapper* or the *Reducer*. These outputs are nothing but intermediate output of the job. Therefore, they must be parameterized with their types. The *Reporter* facilitates the Map-Reduce application to report progress and update counters and status information. If, however, the combine function is used, it has the same form as the reduce function and the output is fed to the reduce function. This may be illustrated as follows

map (K1, V1) -> list (K2, V2)
combine (K2, list(V2)) -> list (K2, V2)
reduce (K2, list(V2)) -> list (K3, V3)

Note that the combine and reduce functions use the same type, except in the variable names where K3 is K2 and V3 is V2.

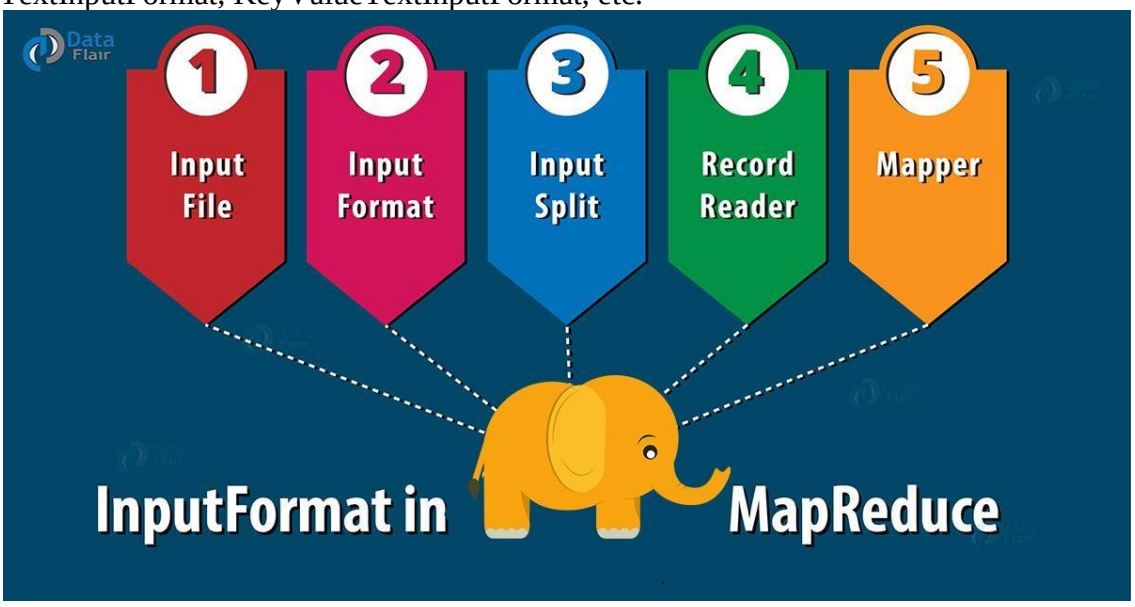
The partition function operates on the intermediate key-value types. It controls the partitioning of the keys of the intermediate map outputs. The key derives the partition using a typical hash function. The total number of partitions is the same as the number of reducetasks for the job. The partition is determined only by the key ignoring the value.

```
public interface Partitioner<K2, V2> extends JobConfigurable {
  int getPartition(K2 key, V2 value, int numberOfPartition);
}
```

This is the key essence of MapReduce types in short.

5. INPUT FORMATS

Hadoop InputFormat checks the Input-Specification of the job. InputFormat split the Input file into InputSplit and assign to individual Mapper. In this Hadoop InputFormat Tutorial, we will learn what is InputFormat in Hadoop **MapReduce**, different methods to get the data to the mapper and different types of InputFormat in Hadoop like FileInputFormat in Hadoop, TextInputFormat, KeyValueTextInputFormat, etc.



An Hadoop InputFormat is the first component in Map-Reduce, it is responsible for creating the input splits and dividing them into records. If you are not familiar with MapReduce Job Flow, so follow our **Hadoop MapReduce Data flow tutorial** for more understanding.

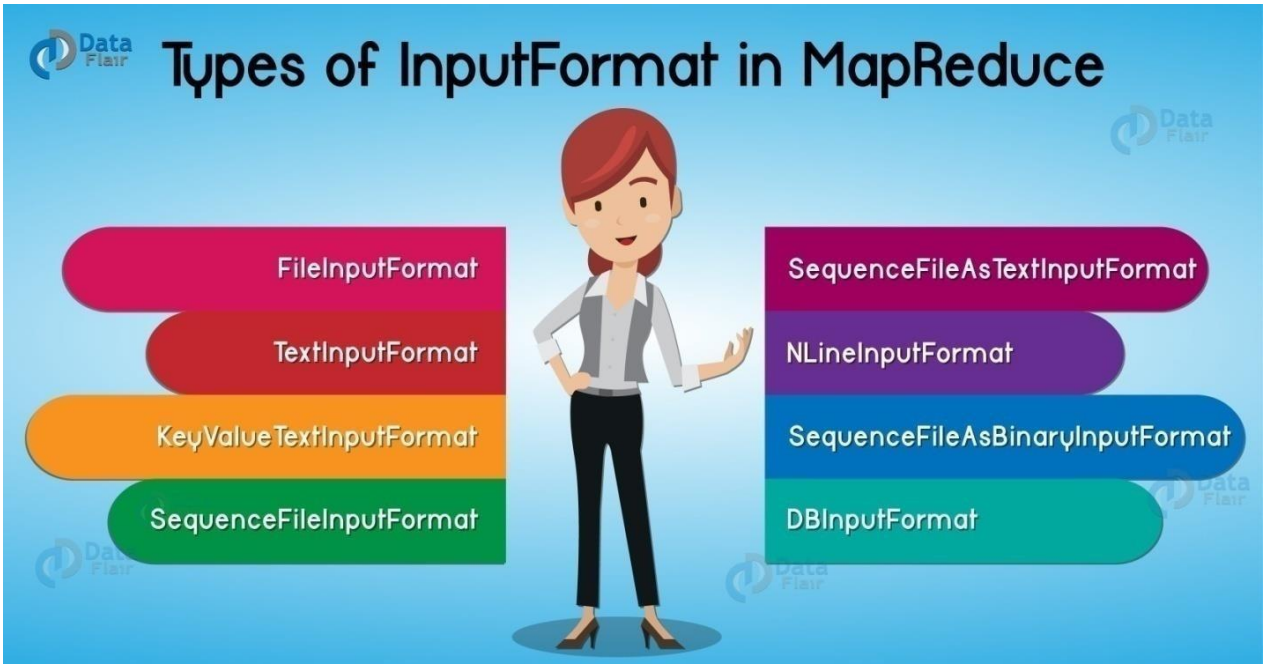
Initially, the data for a MapReduce task is stored in input files, and input files typically reside in **HDFS**. Although these files format is arbitrary, line-based log files and binary format can be used. Using InputFormat we define how these input files are split and read. The InputFormat class is one of the fundamental classes in the Hadoop MapReduce framework which provides the following functionality

- The files or other objects that should be used for input is selected by the InputFormat.
- InputFormat defines the Data splits, which defines both the size of individual **Map tasks** and its potential execution server.
- InputFormat defines the **RecordReader**, which is responsible for reading actual records from the input files

We have 2 methods to get the data to **mapper** in MapReduce getsplits() and createRecordReader() as shown below

```
1. public abstract class InputFormat<K, V>
2.  {
3.   public abstract List<InputSplit> getSplits(JobContext context)
4.   throws IOException, InterruptedException;
5.   public abstract RecordReader<K, V>
6.   createRecordReader(InputSplit split,
7.   TaskAttemptContext context) throws IOException,
8.   InterruptedException;
9.  }
```

Types of Input Format in MapReduce



FileInputFormat in Hadoop

It is the base class for all file-based InputFormats. Hadoop FileInputFormat specifies input directory where data files are located. When we start a Hadoop job, FileInputFormat is provided with a path containing files to read. FileInputFormat will read all files and divides these files into one or more InputSplits.

TextInputFormat

It is the default InputFormat of MapReduce. TextInputFormat treats each line of each input file as a separate record and performs no parsing. This is useful for unformatted data or line- based records like log files.

- **Key –** It is the byte offset of the beginning of the line within the file (not whole file just one split), so it will be unique if combined with the file name.
- **Value –** It is the contents of the line, excluding line terminators.

KeyValueTextInputFormat

It is similar to TextInputFormat as it also treats each line of input as a separate record. While TextInputFormat treats entire line as the value, but the KeyValueTextInputFormat breaks the line itself into key and value by a tab character ('/t'). Here Key is everything up to the tab character while the value is the remaining part of the line after tab character.

SequenceFileInputFormat

Hadoop **SequenceFileInputForma**t is an InputFormat which reads sequence files. Sequence files are binary files that stores sequences of binary **key-value pairs**. Sequence files block-compress and provide direct serialization and deserialization of several arbitrary data types (not just text). Here Key & Value both are user-defined.

SequenceFileAsTextInputFormat

Hadoop **SequenceFileAsTextInputFormat** is another form of SequenceFileInputFormat which converts the sequence file key values to Text objects. By calling **'tostring()'** conversion is performed on the keys and values. This InputFormat makes sequence files suitable input for streaming.

SequenceFileAsBinaryInputFormat

Hadoop **SequenceFileAsBinaryInputFormat** is a SequenceFileInputFormat using which we can extract the sequence file's keys and values as an opaque binary object.

NLineInputFormat

Hadoop **NLineInputFormat** is another form of TextInputFormat where the keys are byte offset of the line and values are contents of the line. Each mapper receives a variable number of lines of input with TextInputFormat and KeyValueTextInputFormat and the number depends on the size of the split and the length of the lines. And if we want our mapper to receive a fixed number of lines of input, then we use NLineInputFormat. N is the number

of lines of input that each mapper receives. By default (N=1), each mapper receives exactly one line of input. If N=2, then each split contains two lines. One mapper willreceive the first two Key-Value pairs and another mapper will receive the second two key- value pairs.

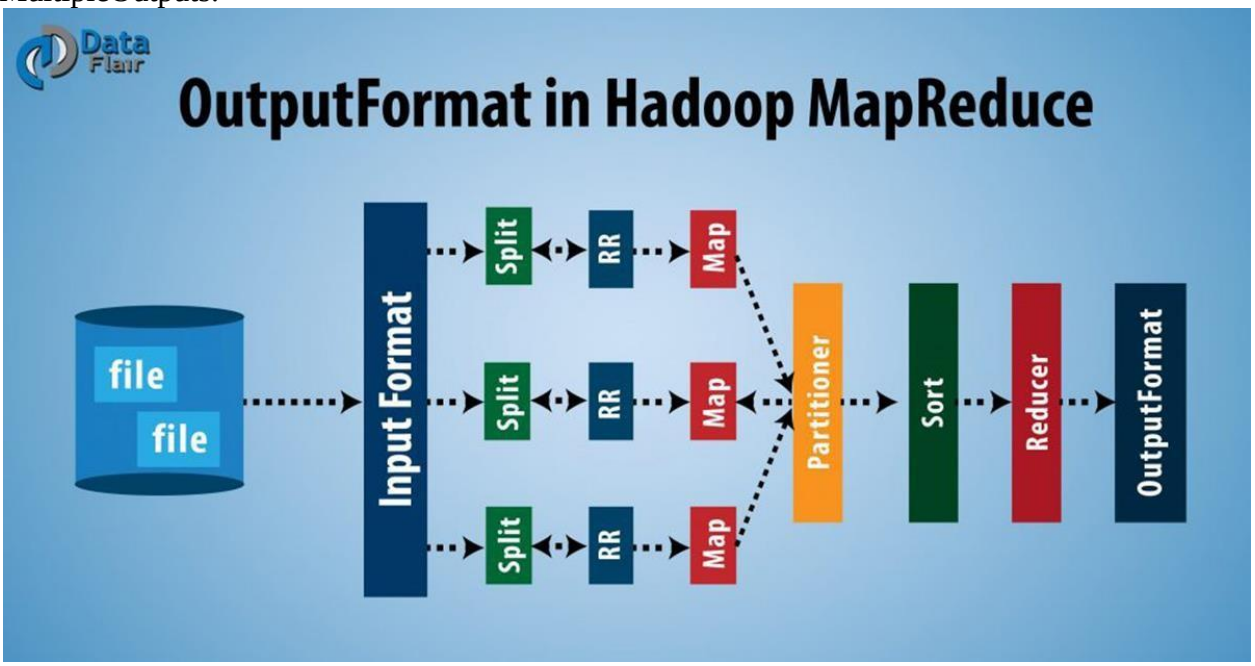
DBInputFormat

Hadoop **DBInputFormat** is an InputFormat that reads data from a relational database, using JDBC. As it doesn't have portioning capabilities, so we need to careful not to swamp the database from which we are reading too many mappers. So it is best for loading relatively small datasets, perhaps for joining with large datasets from HDFS using MultipleInputs. HereKey is LongWritables while Value is DBWritables.

6. OUTPUT FORMATS

The Hadoop Output Format checks the Output-Specification of the job. It determines how RecordWriter implementation is used to write output to output files. In this blog, we are going to see what is Hadoop Output Format, what is Hadoop RecordWriter, how RecordWriter is used in Hadoop?

In this Hadoop Reducer Output Format guide, will also discuss various types of Output Format in Hadoop like textOutputFormat, sequenceFileOutputFormat, mapFileOutputFormat, sequenceFileAsBinaryOutputFormat, DBOutputFormat, LazyOutputForma, and MultipleOutputs.



let us first see what is a RecordWriter in MapReduce and what is its role in MapReduce?

i. Hadoop RecordWriter

As we know, **Reducer** takes as input a set of an intermediate **key-value pair** produced by the **mapper** and runs a reducer function on them to generate output that is again zero or more key-value pairs. RecordWriter writes these output key-value pairs from the Reducer phase to output files.

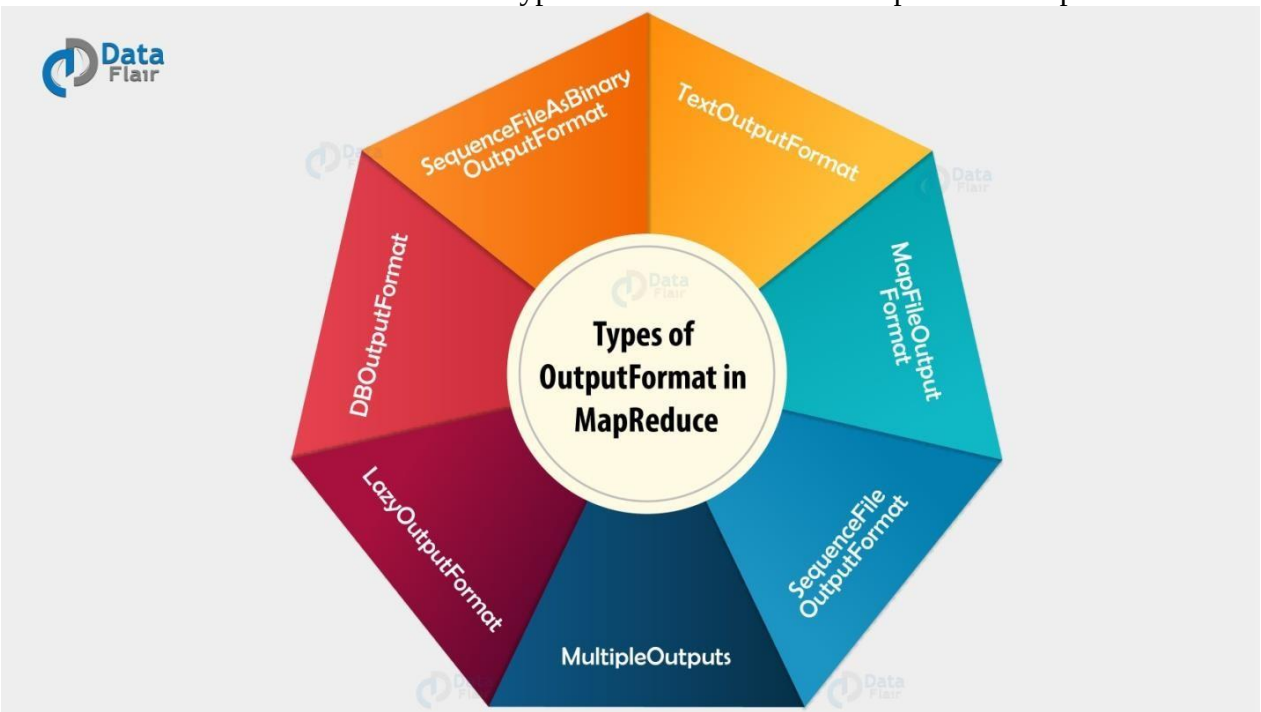
ii. Hadoop Output Format

As we saw above, Hadoop RecordWriter takes output data from Reducer and writes this data to output files. The way these output key-value pairs are written in output files by RecordWriter is determined by the Output Format. The Output Format and **InputFormat** functions are alike. OutputFormat instances provided by Hadoop are used to write to files on the **HDFS** or local disk. OutputFormat describes the output-specification for a **Map-Reduce job**. On the basis of output specification;

- MapReduce job checks that the output directory does not already exist.
- OutputFormat provides the RecordWriter implementation to be used to write the output files of the job. Output files are stored in a FileSystem.

FileOutputFormat.setOutputPath() method is used to set the output directory. Every Reducer writes a separate file in a common output directory.

There are various types of Hadoop OutputFormat.



i. TextOutputFormat

MapReduce default Hadoop reducer Output Format is **TextOutputFormat**, which writes (key, value) pairs on individual lines of text files and its keys and values can be of any type since TextOutputFormat turns them to string by calling toString() on them. Each key-value pair is separated by a tab character, which can be changed using ***MapReduce.output.textoutputformat.separator*** property. KeyValueTextOutputFormat is used for reading these output text files since it breaks lines into key-value pairs based on a configurable separator.

ii. SequenceFileOutputFormat

It is an Output Format which writes sequences files for its output and it is intermediate format use between MapReduce jobs, which rapidly serialize arbitrary data types to the file; and the corresponding SequenceFileInputFormat will deserialize the file into the same types and

presents the data to the next mapper in the same manner as it was emitted by the previous reducer, since these are compact and readily compressible. Compression is controlled by the static methods on SequenceFileOutputFormat.

iii. SequenceFileAsBinaryOutputFormat

It is another form of SequenceFileInputFormat which writes keys and values to sequence file in binary format.

iv. MapFileOutputFormat

It is another form of FileOutputFormat in Hadoop Output Format, which is used to write output as map files. The key in a MapFile must be added in order, so we need to ensure that reducer emits keys in sorted order.
Any doubt yet in Hadoop Oputput Format? Please Ask.MultipleOutputs

It allows writing data to files whose names are derived from the output keys and values, or in fact from an arbitrary string.

v. LazyOutputFormat

Sometimes FileOutputFormat will create output files, even if they are empty. LazyOutputFormat is a wrapper OutputFormat which ensures that the output file will be created only when the record is emitted for a given partition.

vi. DBOutputFormat

DBOutputFormat in Hadoop is an Output Format for writing to relational databases and **HBase**. It sends the reduce output to a SQL table. It accepts key-value pairs, where the key has a type extending DBwritable. Returned RecordWriter writes only the key to the database with a batch SQL query.

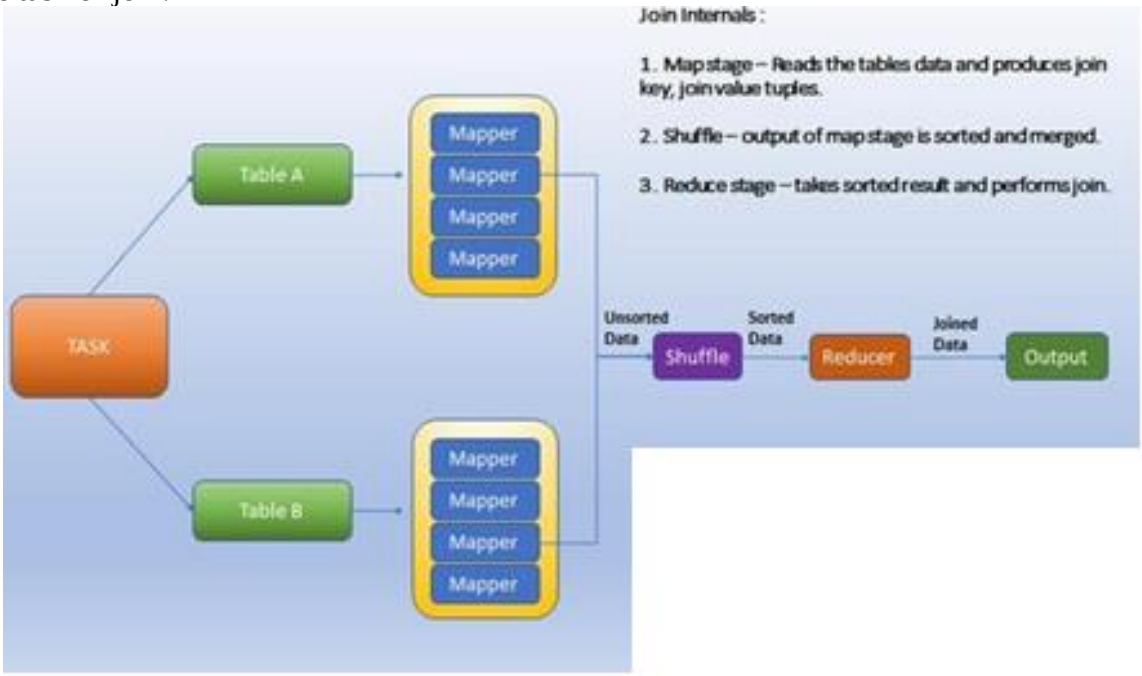
7. MAP SIDE JOIN AND REDUCE SIDE JOIN

Two different large data can be joined in map reduce programming also. Joins in Map phase refers as Map side join, while join at reduce side called as reduce side join. Lets go in detail, Why we would require to join the data in map reduce. If one Dataset A has master data and B has sort of transactional data(A & B are just for reference). we need to join them on a coexisting common key for a result. It is important to realize that we can share data with side data sharing techniques(passing key value pair in job configuration /distribution caching) if master data set is small. we will use map-reduce join only when we have both dataset is too big to use data sharing techniques.

Joins at Map Reduce is not recommended way. Same problem can be addressed through high level frameworks like Hive or cascading. even if you are in situation then we can use below mentioned method to join.

Whenever, we apply join operation, the job will be assigned to a Map Reduce task which consists of two stages- a ***'Map stage***' and a '***Reduce stage***'. A mapper's job during Map Stage is to *"read"* the data from join tables and to *"return"* the **'join key'** and **'join value'** pair into

an intermediate file. Further, in the shuffle stage, this intermediate file is then sorted and merged. The reducer's job during reduce stage is to take this sorted result as input and complete the task of join.



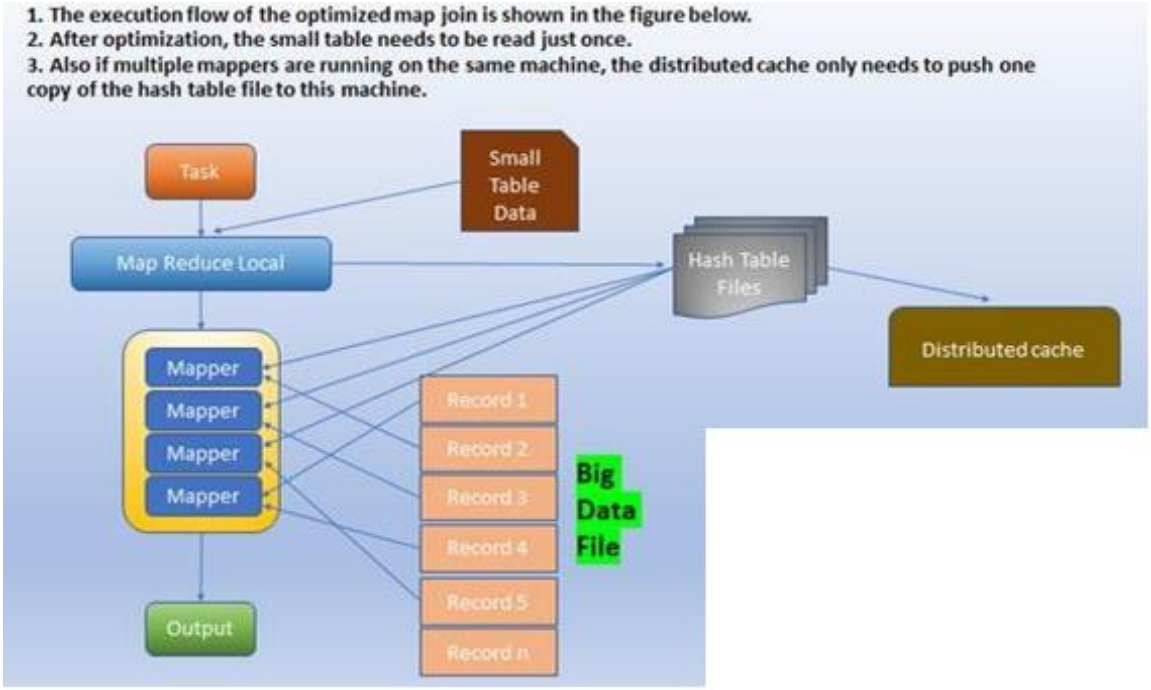
- Map-side Join is similar to a join but all the task will be performed by the mapper alone.
- There will be no reducer stage in Map side join.
- The Map-side Join will be mostly suitable for small tables to optimize the tasks.
- There are two ways to enable it. First is by using a hint, which looks like /*+ MAPJOIN(aliasname), MAPJOIN(anothertable) */

EG SELECT /*+ MAPJOIN(c) */ * FROM orders o JOIN cities c ON (o.city_id = http//c.id);

How will the map-side join optimize the task?

Assume that we have two tables of which one of them is a small table. When we submit a map reduce task, a Map Reduce local task will be created before the original join Map Reduce task which will read data of the small table from HDFS and store it into an in- memory hash table. After reading, it serializes the in-memory hash table into a hash table file.

In the next stage, when the original join Map Reduce task is running, it moves the data in the hash table file to the Hadoop distributed cache, which populates these files to each mapper's local disk. So, all the mappers can load this persistent hash table file back into the memory and do the join work as before. The execution flow of the optimized map join is shown in the figure below. After optimization, the small table needs to be read just once. Also, if multiple mappers are running on the same machine, the distributed cache only needs to push one copy of the hash table file to this machine.



Advantages of using map side join

- Map-side join helps in minimizing the cost that is incurred for sorting and merging in the *shuffle* and *reduce* stages.
- Map-side join also helps in improving the performance of the task by decreasing the time to finish the task.

Disadvantages of Map-side join

- Map side join is adequate only when one of the tables on which you perform map-side join operation is small enough to fit into the memory. Hence it is not suitable to perform map-side join on the tables which are huge data in both of them.

Map side Join

Map side join is a process where joins between two tables are performed in the map phase without the involvement of reduce phase. Map side join allows a table to get loaded into memory ensuring a very fast join operation, performed entirely within a mapper and that too without having to use both map and reduce phases.

Joining at map side performs the join before data reached to map. function It expects a strong prerequisite before joining data at map side. Both joining techniques comes with it's own kind of pros and cons. Map side join could be more efficient to reduce side but strict format requirement is very tough to meet natively. however, if we would prepare this kind of data through some other MR jobs, will lose the expected performance over reduce side join.

- Data should be partitioned and sorted in particular way.
- Each input data should be divided in same number of partitions.
- Must be sorted with same key.
- All the records for a particular key must reside in the same partition.

Reduce Side Join

Reduce side join also called as Repartitioned join or Repartitioned sort merge join and also it is mostly used join type. This type of join would be performed at reduce side. i.e it will have to go through sort and shuffle phase which would incur network overhead. to make itsimple we are going to add the steps needs to be performed for reduce side join. Reduce side join uses few terms like data source, tag and group key lets be familiar with it.

- **Data Source** is referring to data source files, probably taken from RDBMS
- **Tag** would be used to tag every record with it's source name, so that it's source can be identified at any given point of time be it is in map/reduce phase. why it is required will cover it later.
- **Group key** is referring column to be used as join key between two data sources.

As we know we are going to join this data on reduce side we must prepare in a way that it can be used for joining in reduce phase. let's have a look what are the steps needs to be performed.

Map Phase

Expectation from routine map function is emit, (Key, value), while to joining at reduce side join we would design map in a way so that it could emit, (Key, Source Tag+Value) of every record for each data source. This output will then go for sort and shuffle phase, as we know these operation would based on key, so it will club all the values from all source at one place regarding a particular key. and this data would reach to reducer

Reduce Phase

Reducer will create a cross product of every record of map output for one key and will handover to combine function.

Combine function

whether this reduce function is going to perform inner join or outer join would be decided in combine function. And desired ouput format will also be decided at this place

Simple Example for Map Reduce Joins

Let us create two tables

- **Emp** contains details of an Employee such as Employee name, Employee ID and the Department she belongs to.

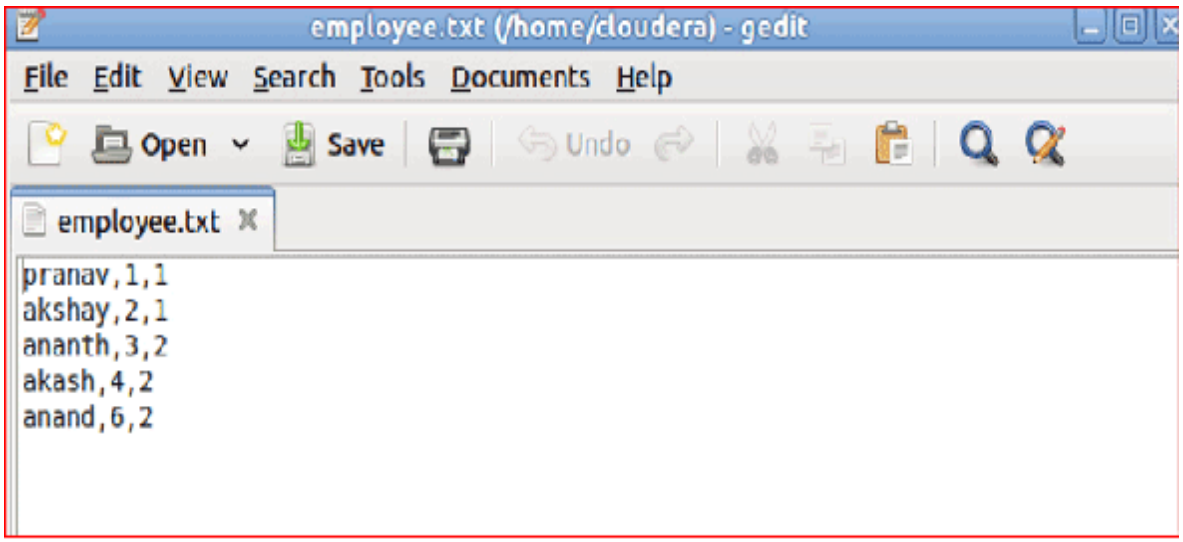
```
hive> create table emp ( name string, id bigint, deptid bigint) row format delim
ited fields terminated by',';
OK
Time taken: 0.328 seconds
```

- **Dept** contains the details like the Name of the Department, Department ID and so on.

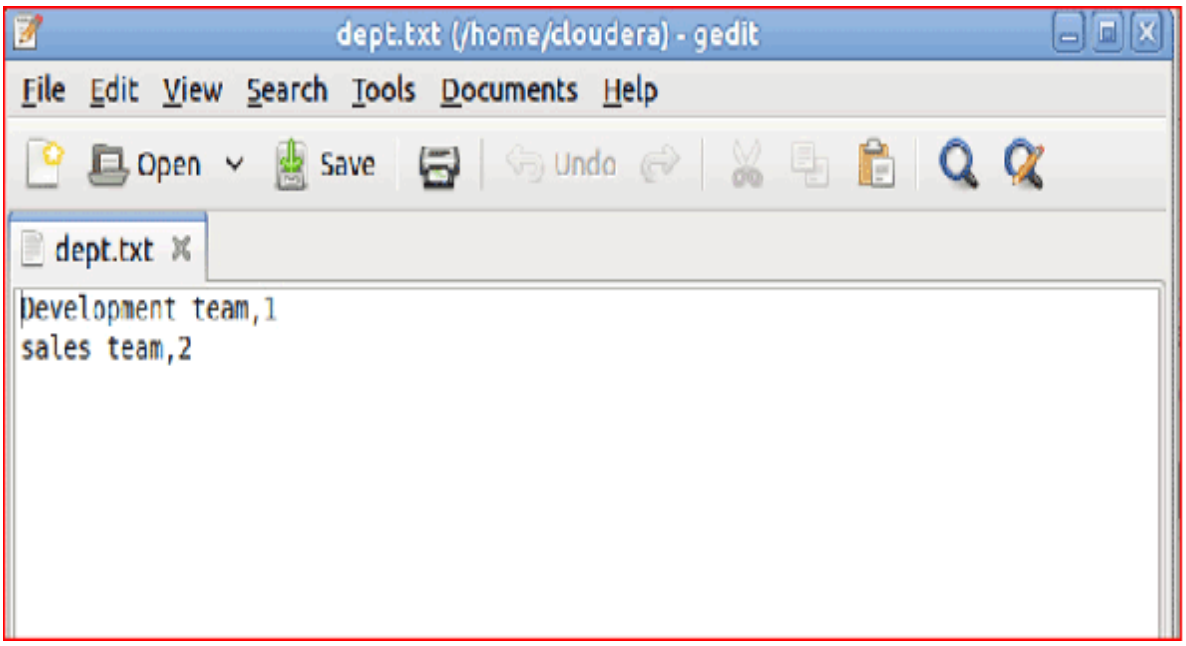
```
hive> create table dept( deptname string, deptid bigint) row format delimited fi
elds terminated by ',';
OK
Time taken: 0.077 seconds
```

Create two input files as shown in the following image to load the data into the tables created.

employee.txt



dept.txt



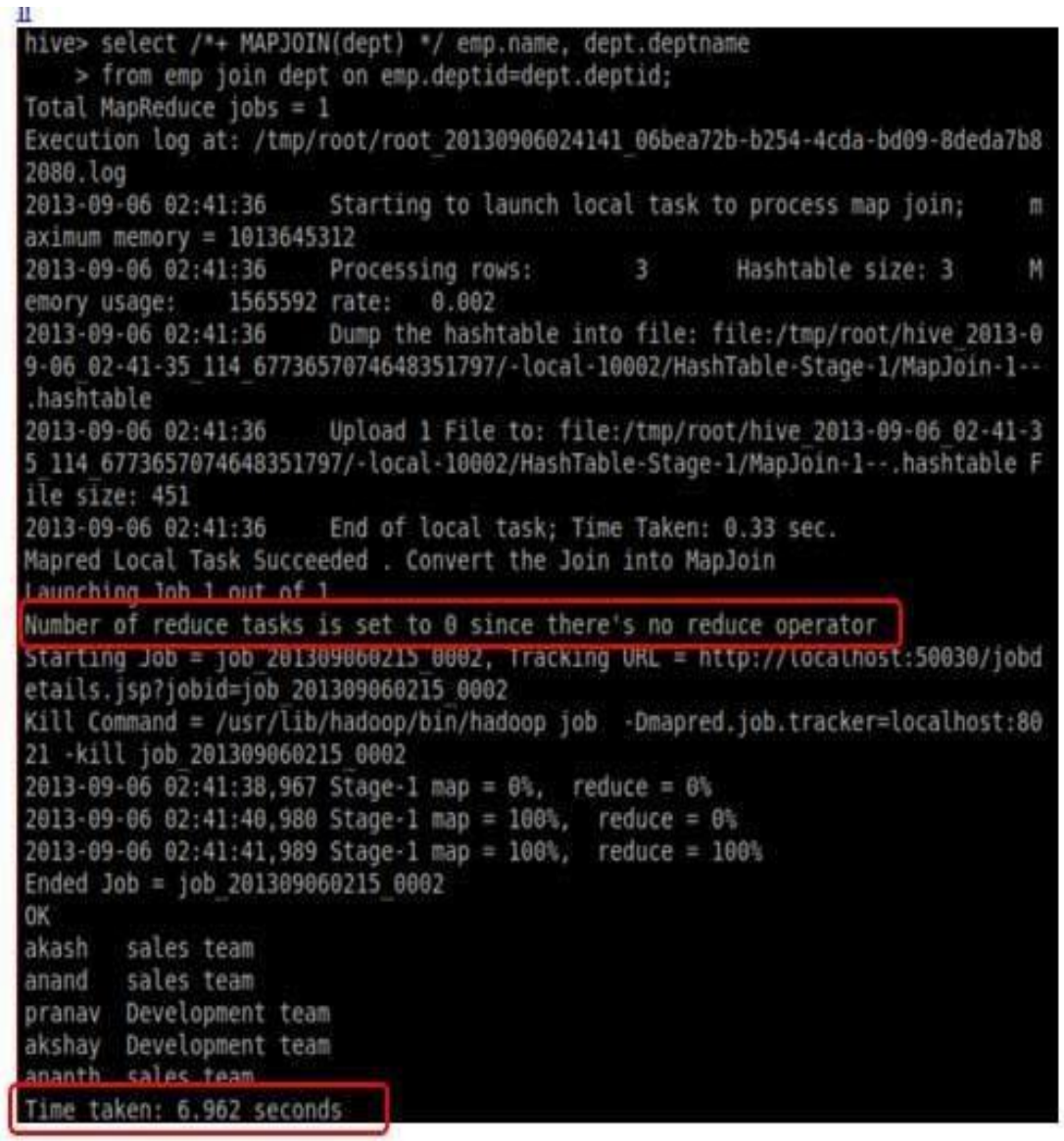
Now, let us load the data into the tables.

```
hive> load data local inpath '/home/cloudera/employee.txt' overwrite into table
emp;
Copying data from file:/home/cloudera/employee.txt
Copying file: file:/home/cloudera/employee.txt
Loading data to table default.emp
Deleted hdfs://localhost/user/hive/warehouse/emp
OK
Time taken: 0.538 seconds
```

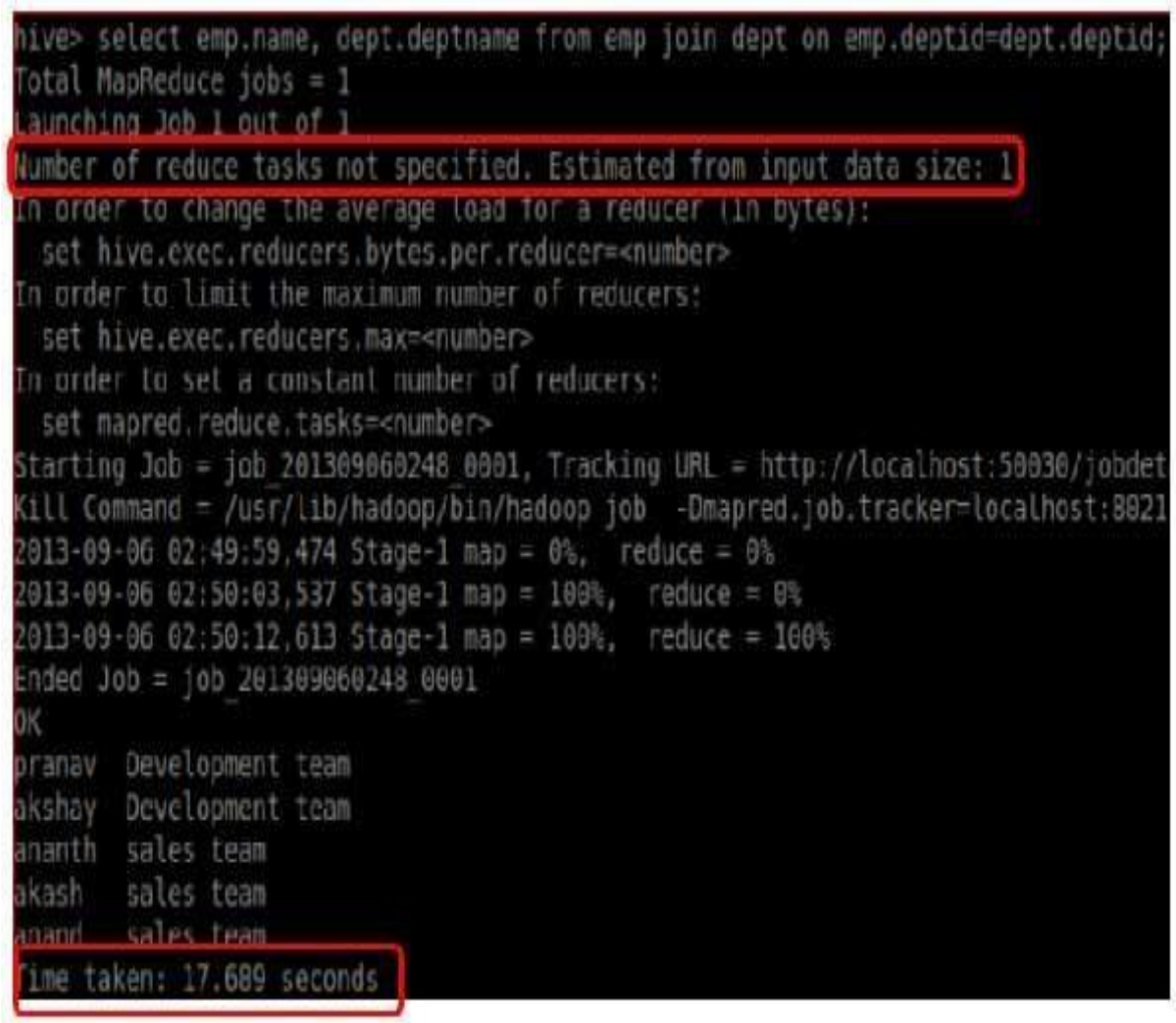
```
hive> load data local inpath '/home/cloudera/dept.txt' overwrite into table dept
;
Copying data from file:/home/cloudera/dept.txt
Copying file: file:/home/cloudera/dept.txt
Loading data to table default.dept
Deleted hdfs://localhost/user/hive/warehouse/dept
OK
Time taken: 0.209 seconds
```

Let us perform the **Map-side Join** on the two tables to extract the list of departments in which each employee is working.

Here, the **second table dept** is a small table. Remember, always the number of department will be less than the number of employees in an organization.



perform the same task with the help of normal Reduce-side join.



While executing both the joins, you can find the two differences

- Map-reduce join has completed the job in less time when compared with the time taken in normal join.
- Map-reduce join has completed its job without the help of any reducer whereas normal join executed this job with the help of one reducer.

Hence, **Map-side Join** is your best bet when one of the tables is small enough to fit in memory to complete the job in a short span of time.

In **Real-time environment**, you will be have data-sets with huge amount of data. So performing analysis and retrieving the data will be time consuming if one of the data-sets is of a smaller size. In such cases *Map-side join* will help to complete the job in less time.

8. MAP REDUCE PROGRAMS

MapReduce Word Count Example

In MapReduce word count example, we find out the frequency of each word. Here, the role of Mapper is to map the keys to the existing values and the role of Reducer is to aggregate the keys of common values. So, everything is represented in the form of Key-value pair.

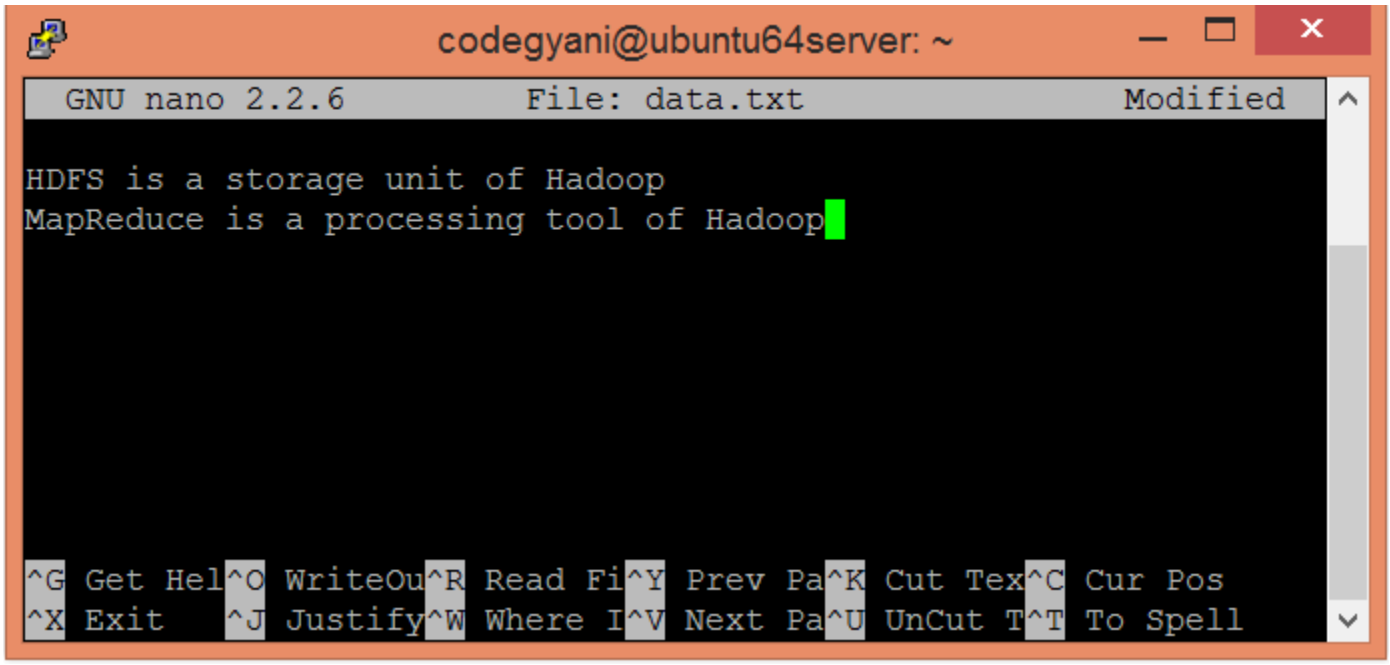
Pre-requisite

- **Java Installation -** Check whether the Java is installed or not using the following command.
  java -version
- **Hadoop Installation -** Check whether the Hadoop is installed or not using the following command.
  hadoop version

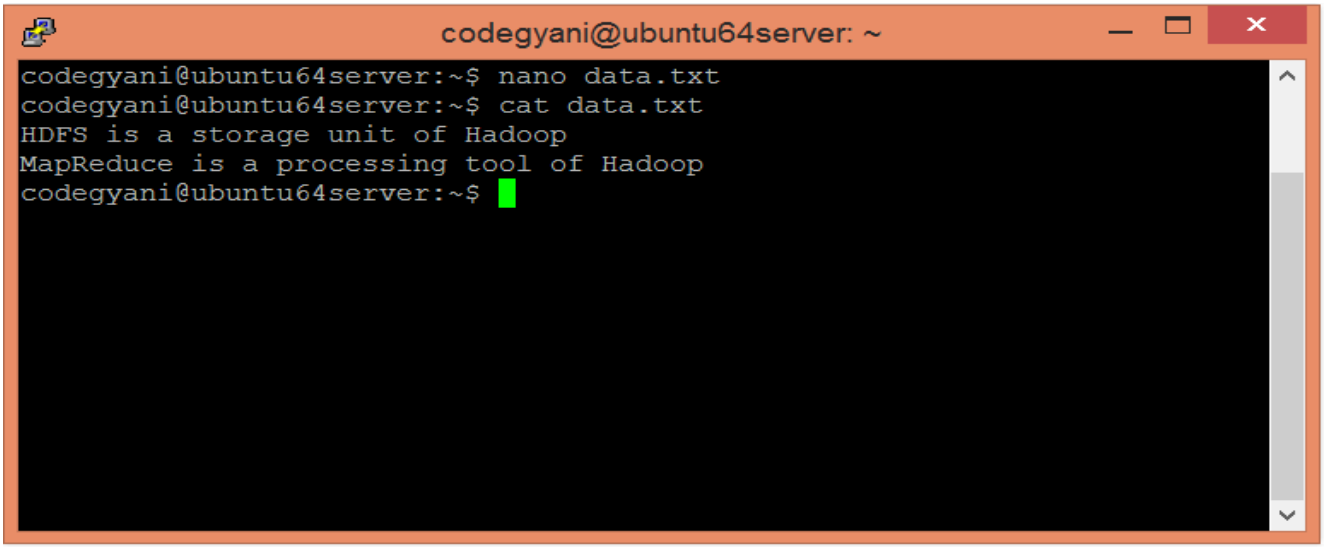
If any of them is not installed in your system, follow the below link to install it.

Steps to execute MapReduce word count example

- Create a text file in your local machine and write some text into it.
  $ nano data.txt

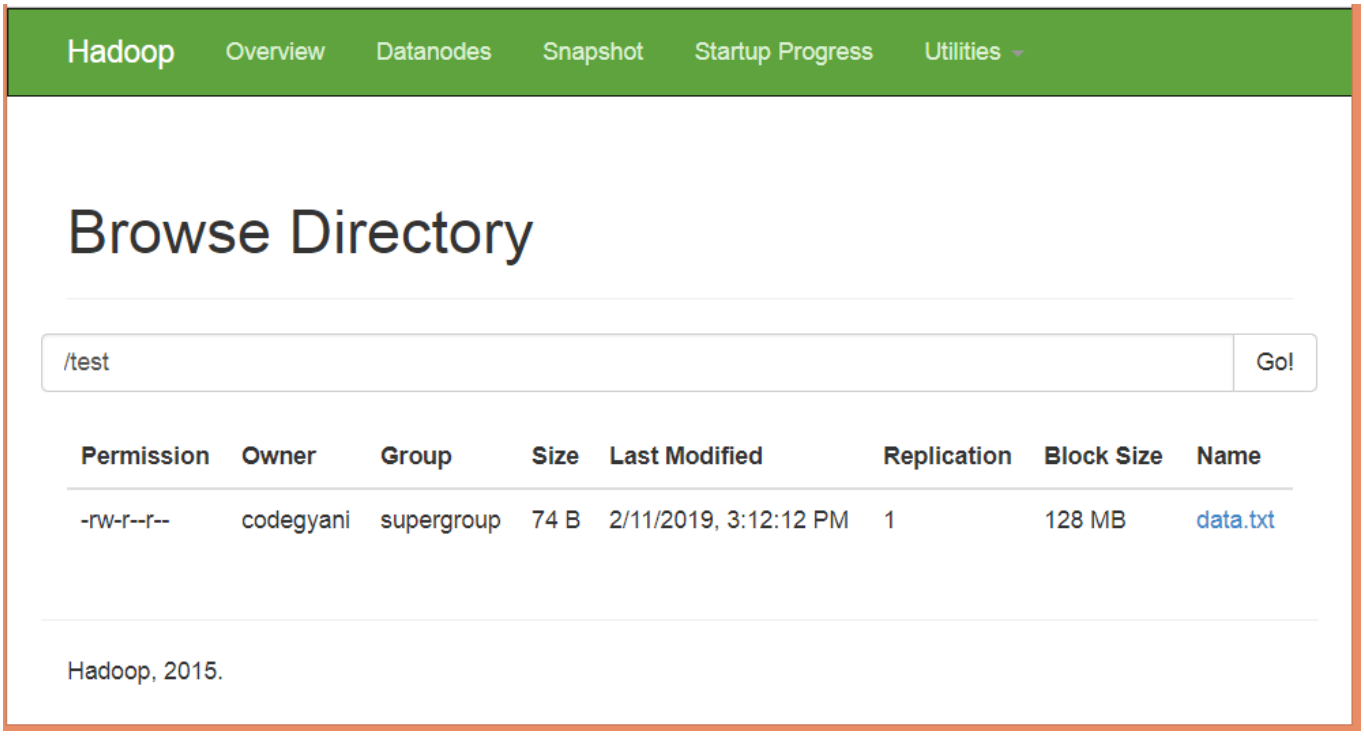


- Check the text written in the data.txt file.
  $ cat data.txt



In this example, we find out the frequency of each word exists in this text file.

- Create a directory in HDFS, where to kept text file.
  $ hdfs dfs -mkdir /test
- Upload the data.txt file on HDFS in the specific directory.
  $ hdfs dfs -put /home/codegyani/data.txt /test



- Write the MapReduce program using eclipse.

File WC_Mapper.java

```
1. package com.javatpoint;
2.
3.  import java.io.IOException;
```

```
4. import java.util.StringTokenizer;
5. import org.apache.hadoop.io.IntWritable;
6. import org.apache.hadoop.io.LongWritable;
7. import org.apache.hadoop.io.Text;
8. import org.apache.hadoop.mapred.MapReduceBase;
9. import org.apache.hadoop.mapred.Mapper;
10. import org.apache.hadoop.mapred.OutputCollector;
11. import org.apache.hadoop.mapred.Reporter;
12. public class WC_Mapper extends MapReduceBase implements Mapper<LongWritabl
    e,Text,Text,IntWritable>{
13.    private final static IntWritable one = new IntWritable(1);
14.    private Text word = new Text();
15.    public void map(LongWritable key, Text value,OutputCollector<Text,IntWritable>
     output,
16.          Reporter reporter) throws IOException{
17.       String line = value.toString();
18.       StringTokenizer  tokenizer = new StringTokenizer(line);
19.       while (tokenizer.hasMoreTokens()){
20.          word.set(tokenizer.nextToken());
21.          output.collect(word, one);
22.       }
23.    }
24.
25. }
```

File WC_Reducer.java

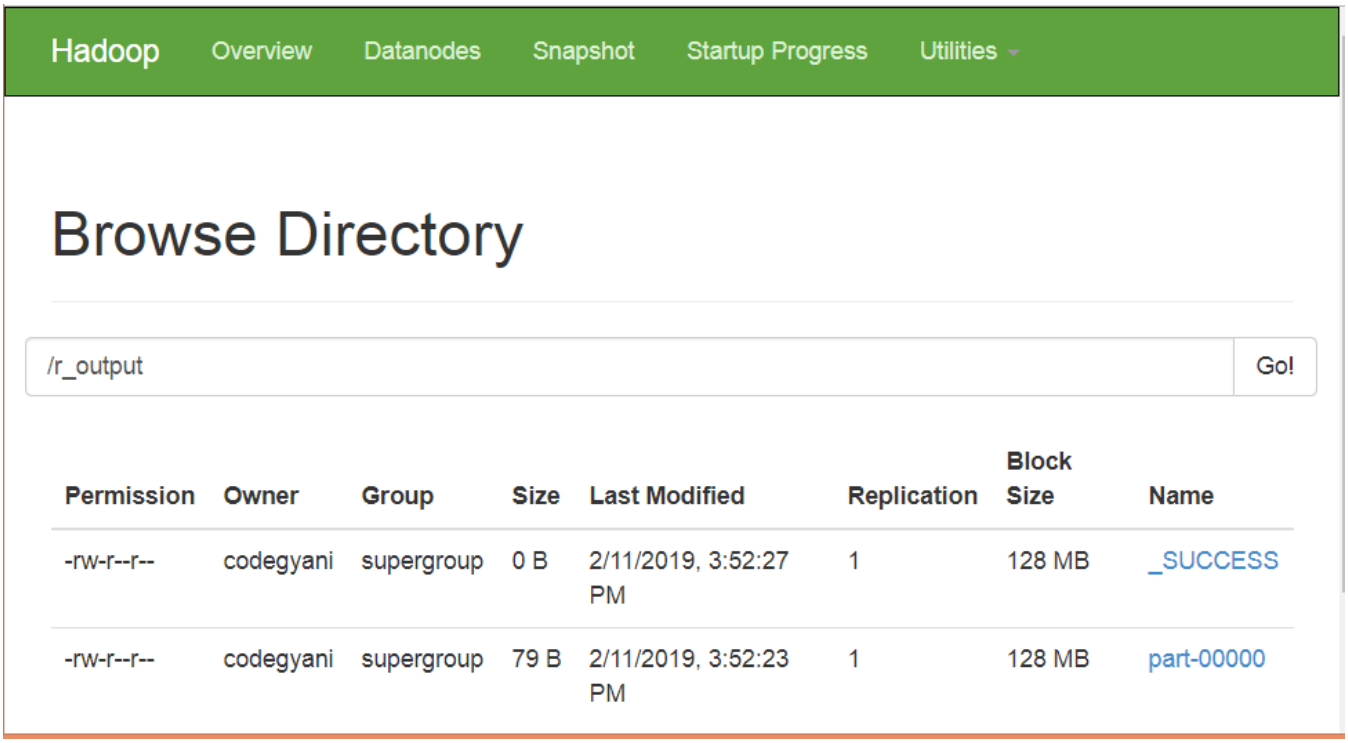
```
1. package com.javatpoint;
2.    import java.io.IOException;
3.    import java.util.Iterator;
4.    import org.apache.hadoop.io.IntWritable;
5.    import org.apache.hadoop.io.Text;
6.    import org.apache.hadoop.mapred.MapReduceBase;
7.    import org.apache.hadoop.mapred.OutputCollector;
8.    import org.apache.hadoop.mapred.Reducer;
9.    import org.apache.hadoop.mapred.Reporter;
10.
11.   public class WC_Reducer extends MapReduceBase implements Reducer<Text,Int
    Writable,Text,IntWritable> {
12.   public void reduce(Text key, Iterator<IntWritable> values,OutputCollector<Text,In
    tWritable> output,
13.    Reporter reporter) throws IOException {
14.   int sum=0;
15.   while (values.hasNext()) {
16.   sum+=values.next().get();
17.   }
18.   output.collect(key,new IntWritable(sum));
19.   }
20.   }
```

File WC_Runner.java

```
1. package com.javatpoint;
2.
3.     import java.io.IOException;
4.     import org.apache.hadoop.fs.Path;
5.     import org.apache.hadoop.io.IntWritable;
6.     import org.apache.hadoop.io.Text;
7.     import org.apache.hadoop.mapred.FileInputFormat;
8.     import org.apache.hadoop.mapred.FileOutputFormat;
9.     import org.apache.hadoop.mapred.JobClient;
10.    import org.apache.hadoop.mapred.JobConf;
11.    import org.apache.hadoop.mapred.TextInputFormat;
12.    import org.apache.hadoop.mapred.TextOutputFormat;
13.    public class WC_Runner {
14.        public static void main(String[] args) throws IOException{
15.            JobConf conf = new JobConf(WC_Runner.class);
16.            conf.setJobName("WordCount");
17.            conf.setOutputKeyClass(Text.class);
18.            conf.setOutputValueClass(IntWritable.class);
19.            conf.setMapperClass(WC_Mapper.class);
20.            conf.setCombinerClass(WC_Reducer.class);
21.            conf.setReducerClass(WC_Reducer.class);
22.            conf.setInputFormat(TextInputFormat.class);
23.            conf.setOutputFormat(TextOutputFormat.class);
24.            FileInputFormat.setInputPaths(conf,new Path(args[0]));
25.            FileOutputFormat.setOutputPath(conf,new Path(args[1]));
26.            JobClient.runJob(conf);
27.        }
28.    }
```

Download the source code.

- Create the jar file of this program and name it **countworddemo.jar.**
- Run the jar file hadoop jar /home/codegyani/wordcountdemo.jar com.javatpoint.WC_Runner /test/data.txt /r_output
- The output is stored in /r_output/part-00000



- Now execute the command to see the output.
  hdfs dfs -cat /r_output/part-00000



Maximum Temperature Program

Problem Statement Find the max temperature of each city using MapReduce

Input

```
Kolkata,56
Jaipur,45
Delhi,43
Mumbai,34
Goa,45
Kolkata,35
Jaipur,34
Delhi,32
```

Output

```
Kolkata  56
Jaipur    45
Delhi     43
Mumbai    34
```

Map

```
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

public class Map
      extends Mapper<LongWritable, Text, Text, IntWritable>{

   private IntWritable max = new IntWritable();
   private Text word = new Text();

   @Override
   protected void map(LongWritable key, Text value, Context context)
         throws IOException, InterruptedException {

      StringTokenizer line = new StringTokenizer(value.toString(),",\t");

      word.set(line.nextToken());
      max.set(Integer.parseInt(line.nextToken()));

      context.write(word,max);

      }
   }
```

Reduce

```
import java.io.IOException;
```

```
import java.util.Iterator;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class Reduce
      extends Reducer<Text, IntWritable, Text, IntWritable>{

   private int max_temp = Integer.MIN_VALUE;
   private int temp = 0;

   @Override
   protected void reduce(Text key, Iterable<IntWritable> values,
         Context context)
         throws IOException, InterruptedException {

      Iterator<IntWritable> itr = values.iterator();

      while (itr.hasNext()) {

         temp = itr.next().get();
         if( temp > max_temp)
         {
            max_temp = temp;
         }
      }

      context.write(key, new IntWritable(max_temp));
   }
}
```

Driver Class

```
import  org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class MaxTempDriver {
   public static void main(String[] args) throws Exception {

      // Create a new job
```

```
        Job job = new Job();

        // Set job name to locate it in the
        distributed environment
        job.setJarByClass(MaxTempDriver.c
        lass); job.setJobName("Max
        Temperature");

        // Set input and output Path, note that we use the default input format
        // which is TextInputFormat (each
        record is a line of input)
        FileInputFormat.addInputPath(job,
        new Path(args[0]));
        FileOutputFormat.setOutputPath(job,
        new Path(args[1]));

        // Set Mapper
        and Reducer
        class
        job.setMapperCl
        ass(Map.class);
        job.setCombiner
        Class(Reduce.cla
        ss);
        job.setReducerCl
        ass(Reduce.class)
        ;

        // Set Output key and value
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable
        .class);

        System.exit(job.waitForCompletion(true) ? 0  1);
    }
}
```

CHAPTER 5

BIG DATA ANALYTICS – CASE STUDIES

CONTENTS

- Netflix on AWS
- AccuWeather on Microsoft Azure
- China Eastern Airlines on Oracle Cloud
- Etsy on Google Cloud
- mLogica on Sap Hana Cloud

1. NETFLIX ON AWS

Netflix is one of the largest media and technology enterprises in the world, with thousands of shows that its hosts for streaming as well as its growing media production division. Netflix stores billions of data sets in its systems related to audio-visual data, consumer metrics, and recommendation engines. The company required a solution that would allow it to store, manage, and optimize viewers' data. As its studio has grown, Netflix also needed a platform that would enable quicker and more efficient collaboration on projects.
"Amazon Kinesis Streams processes multiple terabytes of log data each day. Yet, events show up in our analytics in seconds," says John Bennett, senior software engineer at Netflix.
"We can discover and respond to issues in real-time, ensuring high availability and a great customer experience."

Industries: Entertainment, media streaming

Use cases: Computing power, storage scaling, database and analytics management, recommendation engines powered through AI/ML, video transcoding, cloud collaboration space for production, traffic flow processing, scaled email and communication capabilities

Outcomes:

- Now using over 100,000 server instances on AWS for different operational functions
- Used AWS to build a studio in the cloud for content production that improves collaborative capabilities
- Produced entire seasons of shows via the cloud during COVID-19 lockdowns
- Scaled and optimized mass email capabilities with Amazon Simple Email Service (Amazon SES)

- Netflix's Amazon Kinesis Streams-based solution now processes billions of traffic flows daily

2. ACCUWEATHER ON MICROSOFT AZURE

AccuWeather is one of the oldest and most trusted providers of weather forecast data. The weather company provides an API that other companies can use to embed their weather content into their own systems. AccuWeather wanted to move its data processes to the cloud. However, the traditional GRIB 2 data format for weather data is not supported by most data management platforms. With Microsoft Azure, Azure Data Lake Storage, and Azure Databricks (AI), AccuWeather was able to find a solution that would convert the GRIB 2 data, analyse it in more depth than before, and store this data in a scalable way.

"With some types of severe weather forecasts, it can be a life-or-death scenario," says Christopher Patti, CTO at AccuWeather.

"With Azure, we're agile enough to process and deliver severe weather warnings rapidly and offer customers more time to respond, which is important when seconds count and lives are on the line."

Industries: Media, weather forecasting, professional services

Use cases: Making legacy and traditional data formats usable for AI-powered analysis, API migration to Azure, data lakes for storage, more precise reporting and scaling

Outcomes:

- GRIB 2 weather data made operational for AI-powered next-generation forecasting engine, via Azure Databricks
- Delta lake storage layer helps to create data pipelines and more accessibility
- Improved speed, accuracy, and localization of forecasts via machine learning
- Real-time measurement of API key usage and performance
- Ability to extract weather-related data from smart-city systems and self-driving vehicles

3. CHINA EASTERN AIRLINES ON ORACLE CLOUD

China Eastern Airlines is one of the largest airlines in the world that is working to improve safety, efficiency, and overall customer experience through big data analytics. With Oracle's cloud setup and a large portfolio of analytics tools, it now has access to more in-flight, aircraft, and customer metrics.

“By processing and analysing over 100 TB of complex daily flight data with Oracle Big Data Appliance, we gained the ability to easily identify and predict potential faults and enhanced flight safety,” says Wang Xuanwu, head of China Eastern Airlines’ data lab.

“The solution also helped to cut fuel consumption and increase customer experience.”

Industries: Airline, travel, transportation

Use cases: Increased flight safety and fuel efficiency, reduced operational costs, big data analytics

Outcomes:

- Optimized big data analysis to analyse flight angle, take-off speed, and landing speed, maximizing predictive analytics for engine and flight safety
- Multi-dimensional analysis on over 60 attributes provides advanced metrics and recommendations to improve aircraft fuel use
- Advanced spatial analytics on the travellers’ experience, with metrics covering in-flight cabin service, baggage, ground service, marketing, flight operation, website, and call centre
- Using Oracle Big Data Appliance to integrate Hadoop data from aircraft sensors, unifying and simplifying the process for evaluating device health across an aircraft
- Central interface for daily management of real-time flight data

4. ETSY ON GOOGLE CLOUD

Etsy is an e-commerce site for independent artisan sellers. With its goal to create a buying and selling space that puts the individual first, Etsy wanted to advance its platform to the cloud to keep up with needed innovations. But it didn’t want to lose the personal touches or values that drew customers in the first place. Etsy chose Google for cloud migration and big data management for several primary reasons: Google’s advanced features that back scalability, its commitment to sustainability, and the collaborative spirit of the Google team.

Mike Fisher, CTO at Etsy, explains how Google’s problem-solving approach won them over.

“We found that Google would come into meetings, pull their chairs up, meet us halfway, and say, ‘We don’t do that, but let’s figure out a way that we can do that for you.'”

Industries: Retail, E-commerce

Use cases: Data centre migration to the cloud, accessing collaboration tools, leveraging machine learning (ML) and artificial intelligence (AI), sustainability efforts

Outcomes:

- 5.5 petabytes of data migrated from existing data center to Google Cloud
- >50% savings in compute energy, minimizing total carbon footprint and energy usage
- 42% reduced compute costs and improved cost predictability through virtual machine (VM), solid state drive (SSD), and storage optimizations
- Democratization of cost data for Etsy engineers
- 15% of Etsy engineers moved from system infrastructure management to customer experience, search, and recommendation optimization

5. MLOGICA ON SAP HANA CLOUD

mLogica is a technology and product consulting firm that wanted to move to the cloud, in order to better support its customers' big data storage and analytics needs. Although it held on to its existing data analytics platform, CAP*M, mLogica relied on SAP HANA Cloud to move from on-premises infrastructure to a more scalable cloud structure.

"More and more of our clients are moving to the cloud, and our solutions need to keep pace with this trend," says Michael Kane, VP of strategic alliances and marketing, mLogica

"With CAP*M on SAP HANA Cloud, we can future-proof clients' data setups."

Industry: Professional services

Use cases: Manage growing pools of data from multiple client accounts, improve slow upload speeds for customers, move to the cloud to avoid maintenance of on-premises infrastructure, integrate the company's existing big data analytics platform into the cloud

Outcomes:

- SAP HANA Cloud launched as the cloud platform for CAP*M, mLogica's big data analytics tool, to improve scalability
- Data analysis now enabled on a petabyte scale
- Simplified database administration and eliminated additional hardware and maintenance needs
- Increased control over total cost of ownership
- Migrated existing customer data setups through SAP IQ into SAP HANA, without having to adjust those setups for a successful migration

ABOUT AUTHORS

John T Mesia Dhas received his Ph.D. in Computer Science and Engineering from Vel Tech University, Chennai, India. He has 16 years of Experience in the field of Education and Industry, currently he is working as Associate Professor with Computer Science and Engineering Department of T John Institute of Technology, Bangalore under VTU, India.

He is also doing researches in Software Engineering and Data Science fields. He has published more than 25 research articles in conferences and Journals.

T. S. Shiny Angel received her Ph.D. in Computer Science and Engineering from SRM University, Chennai, India. She has 20 years of Experience in the field of Education and Industry, currently she is working as Associate Professor with Software Engineering Department of SRM Institute of Science and Technology (formerly known as SRM University), Chennai, Tamil Nadu, India.

She is also doing researches in Software Engineering, Machine Learning and Data Analytics fields. She has published more than 45 research papers in conferences and Journals.

Adarsh T K received his Ph.D. in Computer Science and Engineering from SRM University, Chennai, India. He has 16 years of Experience in the field of Education and Industry, currently he is working as Associate Professor with Computer Science and Engineering Department of T John Institute of Technology, Bangalore under VTU, India.

He is also doing researches in Internet of Things, Machine Learning and Data Analytics fields. He has published more than 25 research papers in conferences and Journals.

ISBN: **978-93-5627-419-8**
Price: Rs. 450/-



OTHER BOOKS

S. No	Title	ISBN
1	C LOGIC PROGRAMMING	978-93-5416-366-1
2	MODERN METRICS (MM): THE FUNCTIONAL SIZE ESTIMATOR FOR MODERN SOFTWARE	978-93-5408-510-9
3	PYTHON 3.7.1 Vol - I	978-93-5416-045-5
4	SOFTWARE SIZING APPROACHES	978-93-5437-820-1
5	DBMS PRACTICAL PROGRAMS	978-93-5437-572-9
6	SERVICE ORIENTED ARCHITECTURE	978-93-5416-496-5
7	ANDROID APPLICATIONS DEVELOPMENT PRACTICAL APPROACH	978-93-5445-403-5
8	MOBILE APPLICATIONS DEVELOPMENT	978-93-5445-406-6
9	XML HAND BOOK	978-93-5493-336-3
10	PARALLEL COMPUTING IN ENGINEERING APPLICATIONS	978-93-5578-655-5
11	A TO Z STEP BY STEP APPROACHES FOR INDIAN PATENT	978-93-5607-5740
12	INTRODUCTION TO BIG DATA ANALYTICS	978-93-5627-419-8

For free E-Books: jtmdhasres@gmail.com

ISBN: **978-93-5627-419-8**
Price: Rs. 450/-



View publication stats